

Agent Kung-fu

Ten Layers of Agent Engineering, from Prompts to First Principles

史春奇 (Chunqi SHI)

2026-09-27

Table of contents

1	The Ten-Level Roadmap	3
1.1	Welcome	3
1.2	Ten levels at a glance	4
1.3	Ten Architectures in Detail: How the Object of Control Migrates	5
1.4	How to read this book	17
1.5	What this book does not promise	18
2	Introduction: Agent Upgrades Are Migrations of Control	19
2.1	Model capability is not system capability	19
2.2	The Ten-Level Roadmap: Two Layer Domains and Seven Stages	19
2.3	Acting on an Unknown Map: Why Local Intelligence Needs Global Control	20
2.4	From Prompting a Model to Governing a Lifecycle: Anthropic's Year-by-Year Shift	25
2.5	A Typical AGI Evolution Narrative and This Book's Position	28
2.6	How Three Agent Paths Feed into the AGI Question	31
2.7	Two parallel pipelines	33
2.8	Offline mode does not mean synthetic data	33
2.9	One question throughout the book	34
2.10	Reading conventions	34
3	Environment and Reproducibility	35
3.1	Objective	35
3.2	1. Python project	35
3.3	2. Quarto and PDF	35
3.4	3. Live Ollama path	36
3.5	4. Three entry points with distinct purposes	36
3.6	Check	36
4	Sakila: One Question, Ten Control Surfaces	38
4.1	Data source and security boundary	38
4.2	Data profile	38
4.3	The baseline query used throughout the book	39
4.4	Why holding the question constant matters	39
4.5	Validate the guard first	40
5	Experimental Method: Comparing Ten Levels Fairly	41
5.1	Research question	41
5.2	Controlled variables	41
5.3	Two experimental tracks	41
5.4	Observed measures	42
5.5	Passing criteria	42
5.6	Current validation coverage and gaps	43
5.7	How to interpret the results	43

I	Part I — From Prompts to Tools	45
6	Level 1: Let Prompts Direct the Workflow	46
6.1	Capability Acceptance Experiment	46
6.2	From QA to Tasks: Why the Baseline Is Not Enough	46
6.3	Mental Model	49
6.4	Implementation Roadmap	49
6.5	Running Level 1 on the Shared Question	50
6.6	SQL Guard Rejection Drill	51
6.7	Reading Level 1's Limits Through an Evaluation Framework	52
6.8	When to Stop at This Level	54
6.9	Upgrade Trigger	55
6.10	End-of-Chapter Exercises	55
7	Level 2: Move Capabilities from Prose into Code	56
7.1	Capability Acceptance Experiment	56
7.2	From "Please Do This" to "You Can Only Do This"	56
7.3	SkillRegistry Is a Permission Catalogue	57
7.4	Running Code-Owned Skills	57
7.5	The Real Difference Between Offline and Live Modes	58
7.6	OpenClaw: A General-Purpose Agent Engineering Breakthrough, Not Proof of General Intelligence	58
7.7	What This Level Still Cannot Guarantee	61
7.8	When to Stop at This Level	61
7.9	Upgrade Trigger	61
7.10	End-of-Chapter Exercises	61
II	Part II — From Protocols to Improvement Loops	62
8	Level 3: Make Boundaries Traceable	63
8.1	Capability Acceptance Experiment	63
8.2	The Problem Beyond Tool Calls	63
8.3	Sequence of a Single Request	64
8.4	Harness: Stability Is a System Output, Not a Model Property	64
8.5	Tracing an Offline Run	68
8.6	The Real Boundary of the Codex Workers	69
8.7	What This Level Adds—and What It Does Not	71
8.8	When to Stop at This Level	71
8.9	End-of-Chapter Exercises	72
9	Level 4: Constrain Agent Behavior with a Harness	73
9.1	Capability Acceptance Experiment	73
9.2	The problem: traceability is not control	73
9.3	Architecture: one bundle, two subagents, one execution host	73
9.4	The Polars skill is a runtime input, not a decorative label	76
9.5	Host allowlisting: models plan, code authorizes	76
9.6	One-collect Polars computation	76
9.7	Experiment: the offline lesson still runs the real data path	77
9.8	Ten checks: the generator does not grade itself	78
9.9	Live mode: OmniGenT client connects to the Codex harness	78
9.10	Conclusion and promotion criterion	79
9.11	End-of-chapter exercises	79

10 Level 5: Make Failure the Input to the Next Iteration	80
10.1 Capability Acceptance Experiment	80
10.2 The problem: how can an agent explore the full data space and stop at the goal?	80
10.3 From Item Repair to PRD and SDLC Loops	81
10.4 Experiment: complete “explore—calculate—stop” in 17 iterations	83
10.5 How the live loop works	84
10.6 The evaluator determines the flywheel’s direction	85
10.7 Conclusion and promotion criterion	86
10.8 End-of-chapter exercises	86
 III Part III — From Data Contracts to Knowledge Production	 87
11 Level 6: Encode Rules and Relationships in Types and Graphs	88
11.1 Capability Acceptance Experiment	88
11.2 The problem: feedback knows the result is poor, but not how tables connect . . .	88
11.3 From a Skill Catalogue to a Feature Graph	89
11.4 Experiment 1: a typed manifest	92
11.5 Experiment 2: recover relationships from DDL	93
11.6 Graphs, lineage, and ontologies are different	94
11.7 Deterministic gates and DeepEval	95
11.8 Conclusion and promotion criterion	95
11.9 End-of-chapter exercises	95
 12 Level 7: Let the System Remember Yesterday	 96
12.1 Capability Acceptance Experiment	96
12.2 The problem: the graph can be rebuilt, but every explanation must be rebuilt too	96
12.3 Three Open-Source Architecture Paths for Knowledge Production	97
12.4 Why Knowledge-Driven Production Is Entering Complex Problem Solving . . .	99
12.5 Experiment: generate a knowledge snapshot from the database	100
12.6 Why Markdown is a valuable intermediate layer	102
12.7 “LLM Wiki” needs a precise explanation	102
12.8 New risks introduced by accumulated knowledge	102
12.9 Conclusion and promotion criterion	103
12.10 End-of-chapter exercises	103
 13 Level 8: From Recording Knowledge to Producing Knowledge	 104
13.1 Capability Acceptance Experiment	104
13.2 The Open-Source Implementation Landscape of Ontology Engineering	104
13.3 Trend: From Generating a File to Governing a Knowledge-Production Lifecycle .	107
13.4 Epistemic Reorganization: An Ontology Is Not Knowledge Itself but an Explicit Commitment About What Can Be Said	108
13.5 The problem: a page states a relationship, but the system does not know its type	109
13.6 Experiment: generate and parse the canonical OWL	110
13.7 Schema graphs, ontologies, and business truth	111
13.8 Canonical and candidate layers must remain separate	112
13.9 Evaluation	112
13.10 Conclusion and promotion criterion	112
13.11 End-of-chapter exercises	112

IV Part IV — From Problems to Falsifiable Innovation	113
14 Level 9: Ask Whether the Effect Is Identifiable Before Asking How to Optimize	114
14.1 Capability Acceptance Experiment	114
14.2 The problem: what can we infer from Store 2's higher revenue?	114
14.3 Experiment: broaden the descriptive evidence without crossing its boundary	115
14.4 DAG: turn assumptions into objects of debate	115
14.5 Trust levels for this level's artifacts	116
14.6 From an identification gap to the next experiment	116
14.7 Conclusion and promotion criterion	117
14.8 End-of-chapter exercises	117
15 Level 10: Derive Falsifiable Innovation from Invariants	118
15.1 Capability Acceptance Experiment	118
15.2 The problem: the DAG tells us what we do not know—what comes next?	118
15.3 Experiment: inspect the first-principles structure	119
15.4 From a mechanism to a falsifiable experiment	119
15.5 Implementation facts: how Level 10 inherits Level 9	120
15.6 How first-principles reasoning differs from unconstrained brainstorming	121
15.7 Conclusion: the highest level still receives no additional privilege	121
15.8 End-of-chapter exercises	121
16 Conclusion: Choose the Lowest Sufficient Control Level	122
16.1 The ten levels are a selection matrix, not a ranking	122
16.2 The migration of control	123
16.3 Ten engineering principles	124
16.4 A recommended learning path	124
16.5 Chinese–English synchronized maintenance policy	125
16.6 Glossary	125
16.7 The final reader test	126
References	126

1 The Ten-Level Roadmap

1.1 Welcome

Large language models can write SQL, but “being able to write SQL” is not the same as having a reliable agent system. Whether a system is ready for production usually depends less on making the model slightly smarter than on having clear answers to questions such as these:

- Who decides the next action?
- Which capabilities may the model invoke?
- Who arbitrates when multiple participants disagree?
- How does failure feed the next iteration instead of being silently ignored?
- How is experience retained as reusable knowledge?
- When may the system only describe an association, and when may it propose a causal intervention?

This book answers these questions through one consistent case study: using the local `ornith-1.5:9b` model to analyze the Sakila film-rental database. The ten-level architecture is not a collection of ten chat scripts with different names. It is a sequence of ten **migrations of control**: from prompts to code, protocols, governance, feedback, data relationships, persistent knowledge, constrained knowledge production, causal questions, and finally falsifiable experiments.

What you will be able to do

After finishing the book, you will be able to choose the lowest level that is sufficient for a concrete task, explain why an upgrade is needed, identify the new control surface introduced by that upgrade, and state which risks remain unresolved.

1.2 Ten levels at a glance

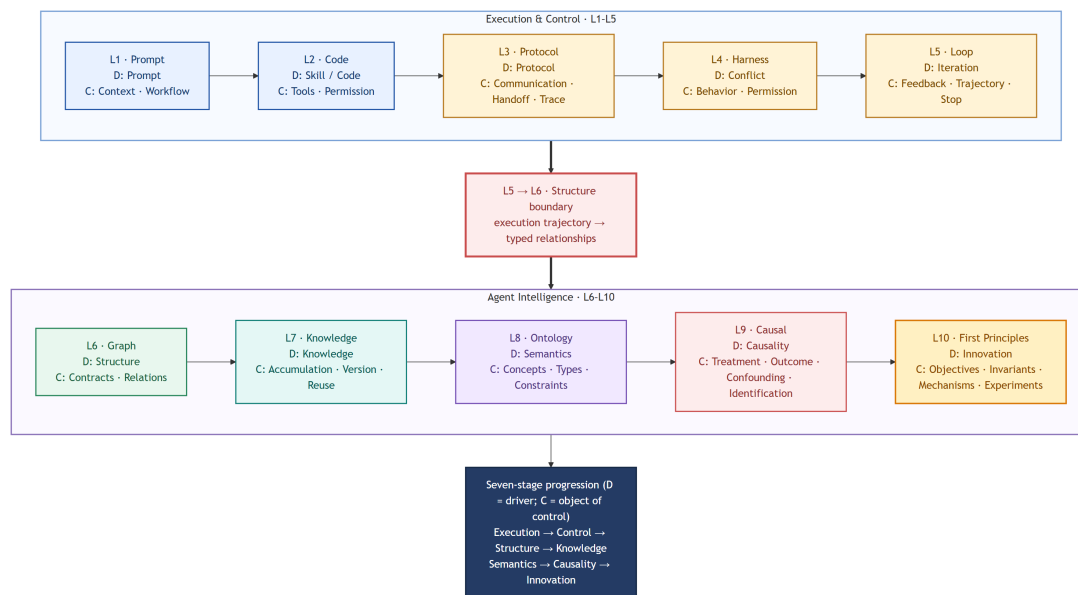


Figure 1.1: The ten-level agent roadmap: the first five levels establish execution and control; the next five make structure, knowledge, semantics, causality, and innovation explicit objects of control

Table 1.1: Dominant drivers, objects of control, and example core technologies across the ten levels

Layer	Suggested name	Core driver	What does the Agent control?	Example core technologies
L1	Prompt Agent	Prompt	Token / Context	LiteLLM + LangGraph + Jinja2
L2	Code Agent	Skill / Code	Tool / Code	LiteLLM + OpenAI Agents
L3	Protocol Agent	Protocol	Agent communication	Codex Python SDK + A2A/MCP/JSON-RPC
L4	Harness Agent	Conflict	Agent behavior	Harness + OmniGenT
L5	Loop Agent	Iteration	Agent trajectory	Ralph Loop
L6	Graph Engineering Agent	Structure	Data / Collaboration	Harness + Graph Engineering
L7	Knowledge Driven Agent	Knowledge	Knowledge accumulation	LLM Wiki
L8	Ontology Driven Agent	Semantics	Concepts / Types / Relations	Ontology
L9	Causal Driven Agent	Causality	Problem / Cause / Effect	Causal model
L10	First-Principles Driven Agent	Innovation	Assumptions / Laws / Principles	First Principles

Table Table 1.1 is the book's compact canonical classification. The C labels inside the roadmap expand those same objects of control into implementable sub-objects; they are not a competing level taxonomy. When citing a level's standard name, dominant driver, or object of control, use the table.

1.3 Ten Architectures in Detail: How the Object of Control Migrates

The next ten figures reconstruct the reference sketches as auditable control architectures rather than copying their pixels. Blue denotes inputs or execution endpoints, gold denotes control mechanisms, red denotes validation and decision gates, green denotes data, evidence, or outputs, purple denotes knowledge and semantic assets, and gray denotes external boundaries. An arrow represents an explicit contract or evidence flow, not an implemented integration. Unless a level's executable demo explicitly instantiates and verifies an integration, every product name in this section is an architectural example.

1.3.1 L1 Prompt Agent: Let prompts direct a fixed workflow

At L1, the Prompt carries task semantics and the model proposes a candidate. The fixed workflow and code guard still own execution order and permission, so this level controls context and flow rather than open-ended action.

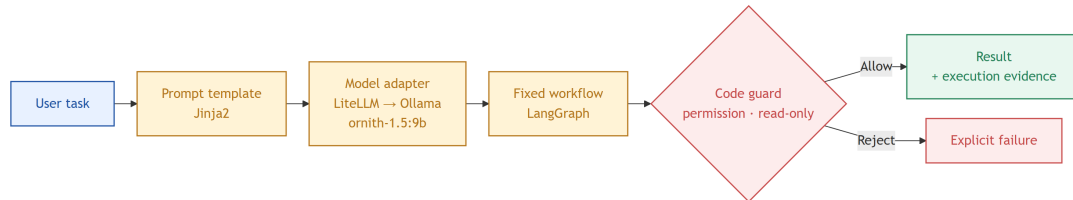


Figure 1.2: L1 Prompt Agent: prompts carry task semantics while a fixed workflow and code guard retain execution control

1.3.2 L2 Code Agent: Move capabilities from text into code

L2 uses an SDK to construct role-specific Agents and coded Skills to make inputs, outputs, permissions, and errors explicit. Python, Search, and SQL are examples of authorized tools; the three branches are not yet cross-Agent protocol collaboration.

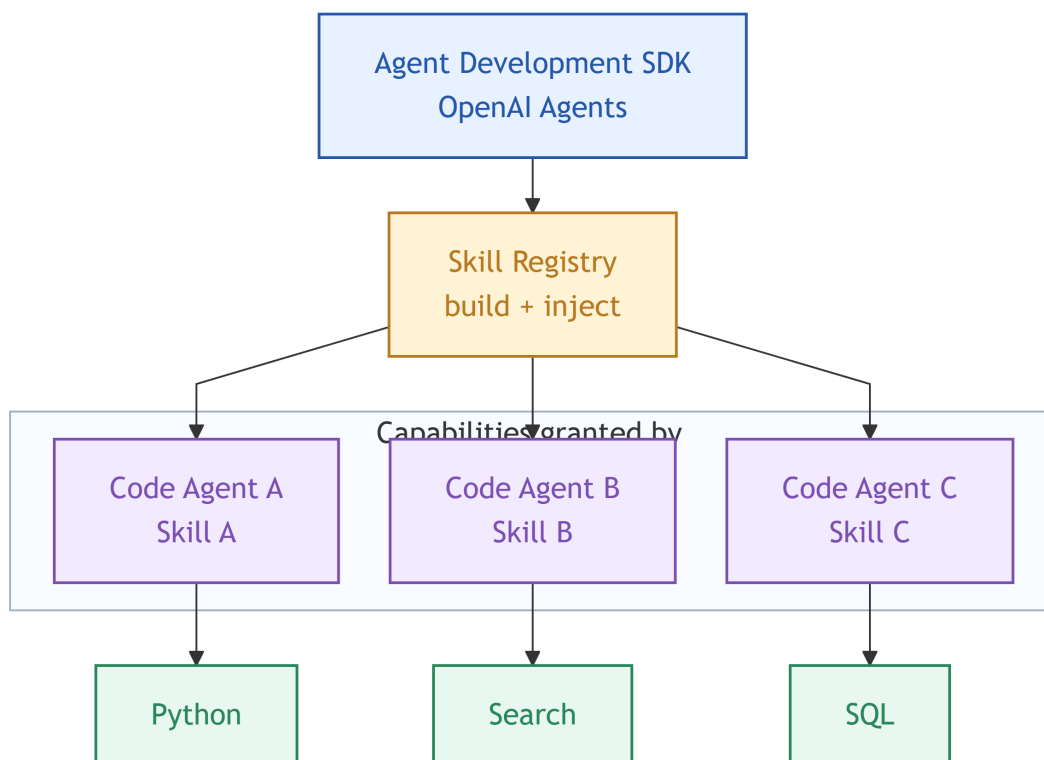


Figure 1.3: L2 Code Agent: a development SDK injects coded Skills into role-specific Agents and maps them to explicitly authorized tools

1.3.3 L3 Protocol Agent: Let dynamic Codex worker threads collaborate through protocols

L3 no longer uses the OpenAI Agents SDK as its runtime. `agent.md` declares named Reader and Calculator workers, Pydantic first turns those declarations into a validated contract, and a Codex worker factory then creates one thread per worker through the Codex Python SDK. Both threads are ephemeral, read-only, and deny every approval request; they produce structured actions through the local app-server JSON-RPC boundary instead of executing arbitrary code directly. Worker names remain in the host Trace because ephemeral threads do not permit metadata updates such as renaming. A host-side allowlist permits only data reads or Polars calculations, after which the evidence is recorded as MCP events. A2A delegation, JSON-RPC turns, and MCP results share one correlation ID, so the complete interaction can be reconstructed as a single Trace.

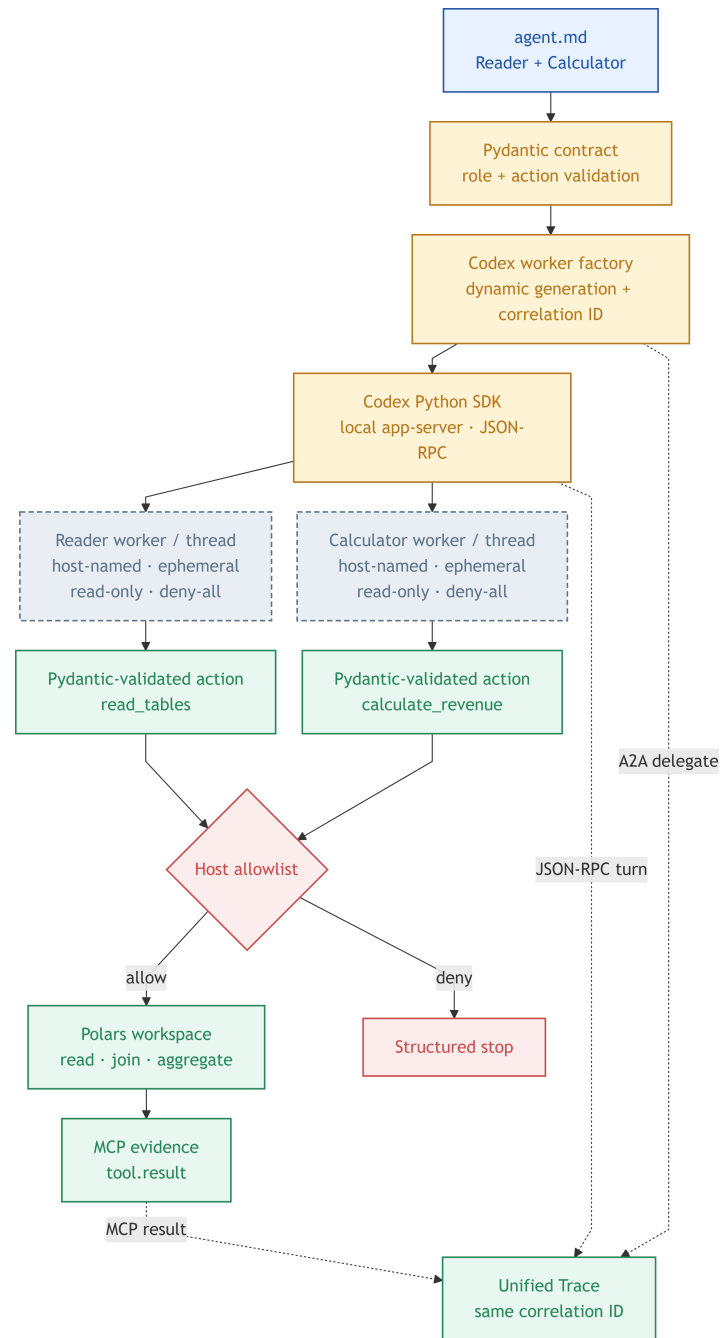


Figure 1.4: L3 Protocol Agent: agent.md dynamically generates two constrained Codex worker threads, a host allowlist executes their actions, and A2A, JSON-RPC, and MCP form a correlated Trace

1.3.4 L4 Harness Agent: Put behavioral conflicts behind an execution boundary

L4 uses an OmniGenT bundle to generate exactly two Codex subagents: a Reader proposes the table-read action, then a Calculator proposes the revenue action. The root and both child configurations declare `sandbox: auto`, an empty `write_paths`, `allow_network: false`, and `start_in_scratch: true`; OmniGenT 0.11's Codex executor uses `approvalPolicy=never`. These settings provide defense in depth. A Windows Job Object isolates the process tree and resources but does not enforce filesystem or network policy, so the portable hard boundary remains typed model proposals followed by a fixed host allowlist. The host then checks delegation evidence, action count, order, parameters, and skill digest, and accepts the result only after ten deterministic checks and SQL-baseline comparison.

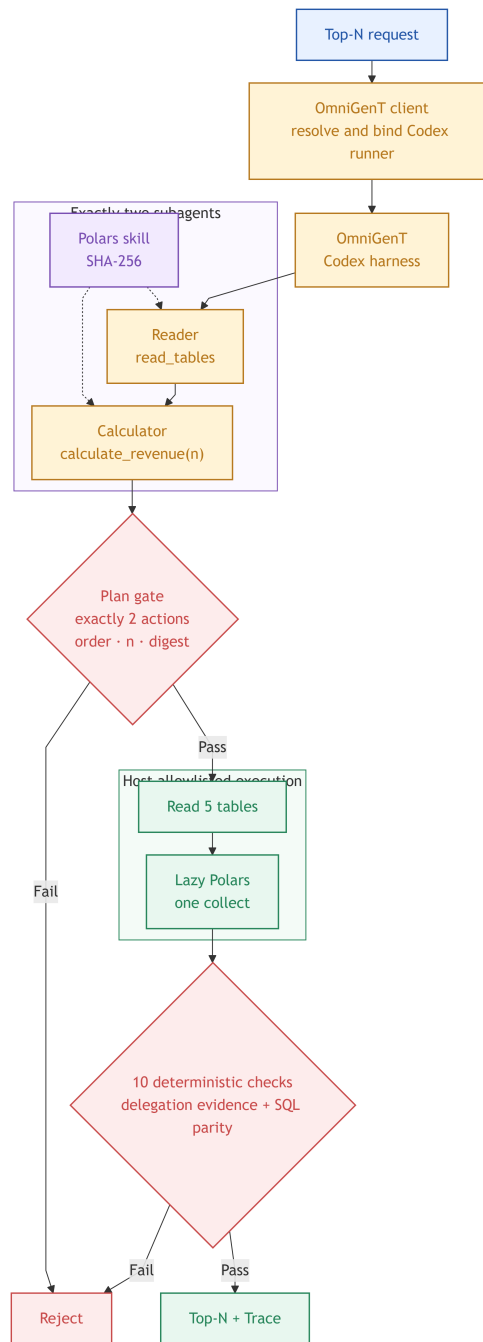


Figure 1.5: L4 Harness Agent: OmniGenT generates two Codex subagents; the host allowlists execution and evaluates it independently

1.3.5 L5 Loop Agent: Make failure the next-round input

L5 contains two nested loops: verification failure returns evidence to Execute; after one requirement passes, unfinished PRD work returns the next item to Plan. Attempts are counted independently for each requirement. Once that item's limit is reached, the system escalates the evidence and unmet requirement to a human owner, stops in a structured state, and preserves the unfinished PRD.

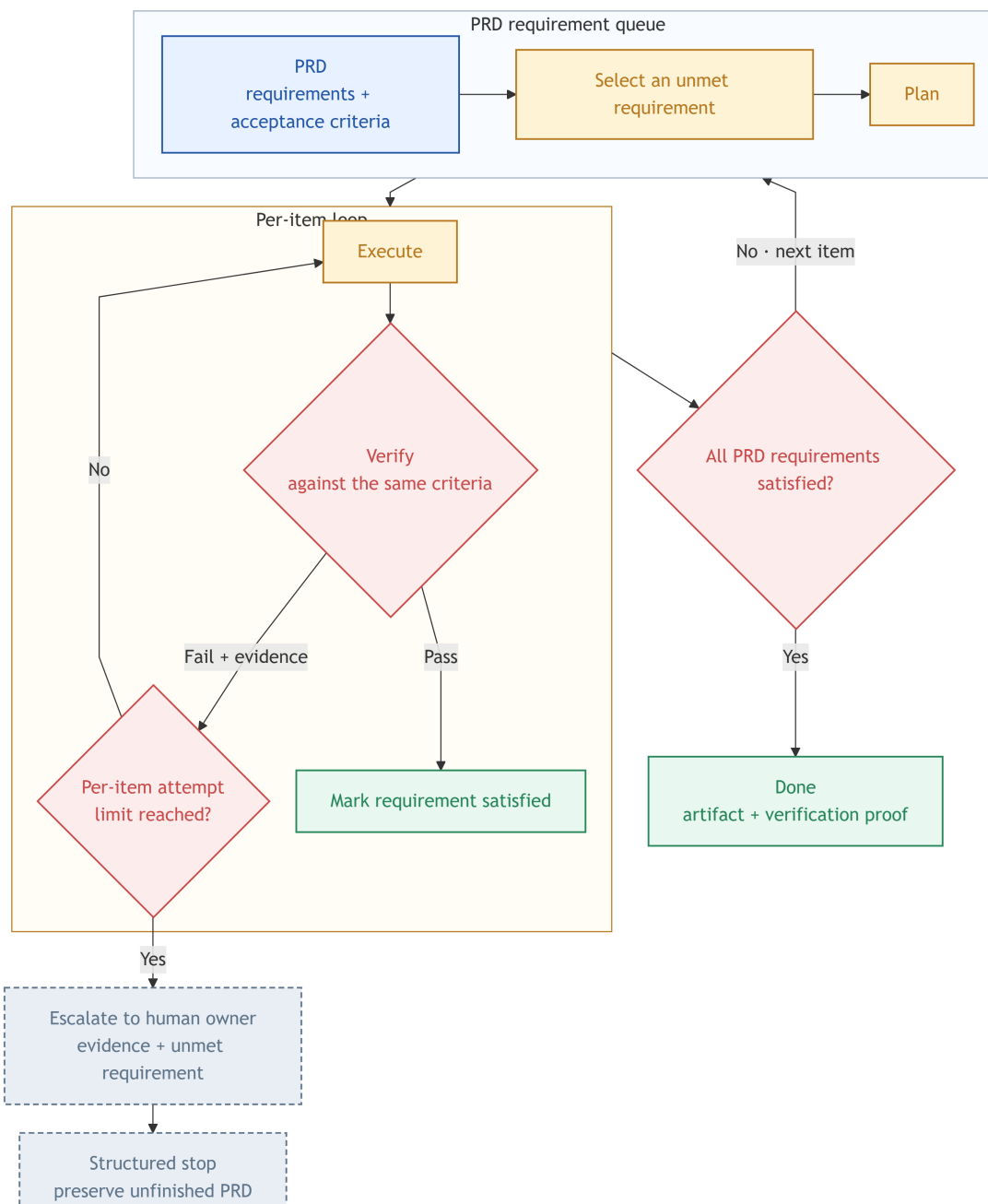


Figure 1.6: L5 Loop Agent: the PRD drives the requirement loop, while failed verification drives a local repair loop

1.3.6 L6 Graph Engineering Agent: Encode collaboration in a relation graph

L6 is not merely about drawing a process as a graph. It makes node dependencies, exchange protocols, data contracts, and quality gates executable relationships, allowing multiple engineering Loops to collaborate around one typed graph instead of sharing only natural-language context.

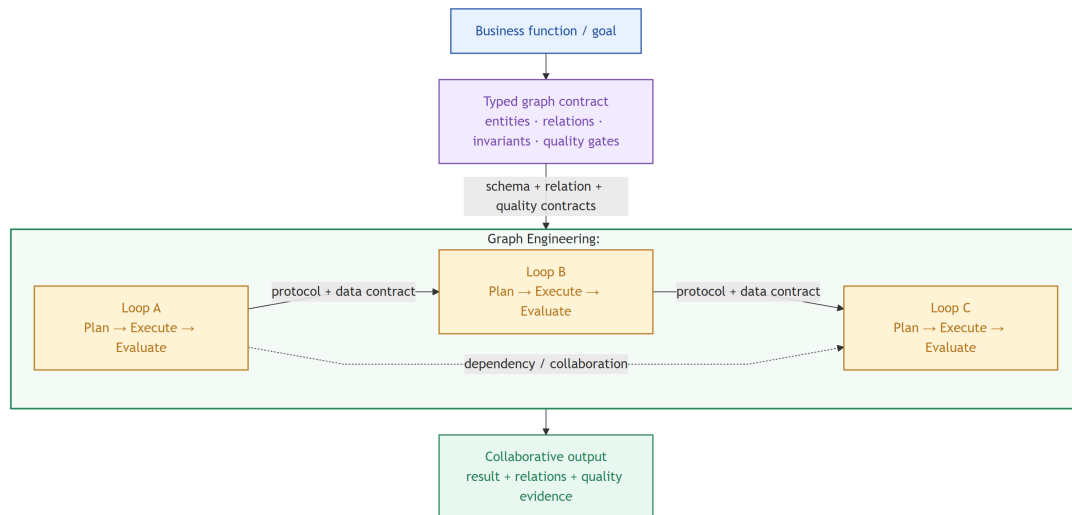


Figure 1.7: L6 Graph Engineering Agent: explicit relationships, protocols, and data contracts orchestrate multiple engineering loops

1.3.7 L7 Knowledge Driven Agent: Let the system remember yesterday

L7 makes cross-run knowledge explicit. The figure unfolds feedback across time: the previous Wiki version is retrieved into the current collaboration graph, and this run's trajectories, decisions, and evaluations become the next Wiki version. Memory is therefore not confused with hidden state inside one run.

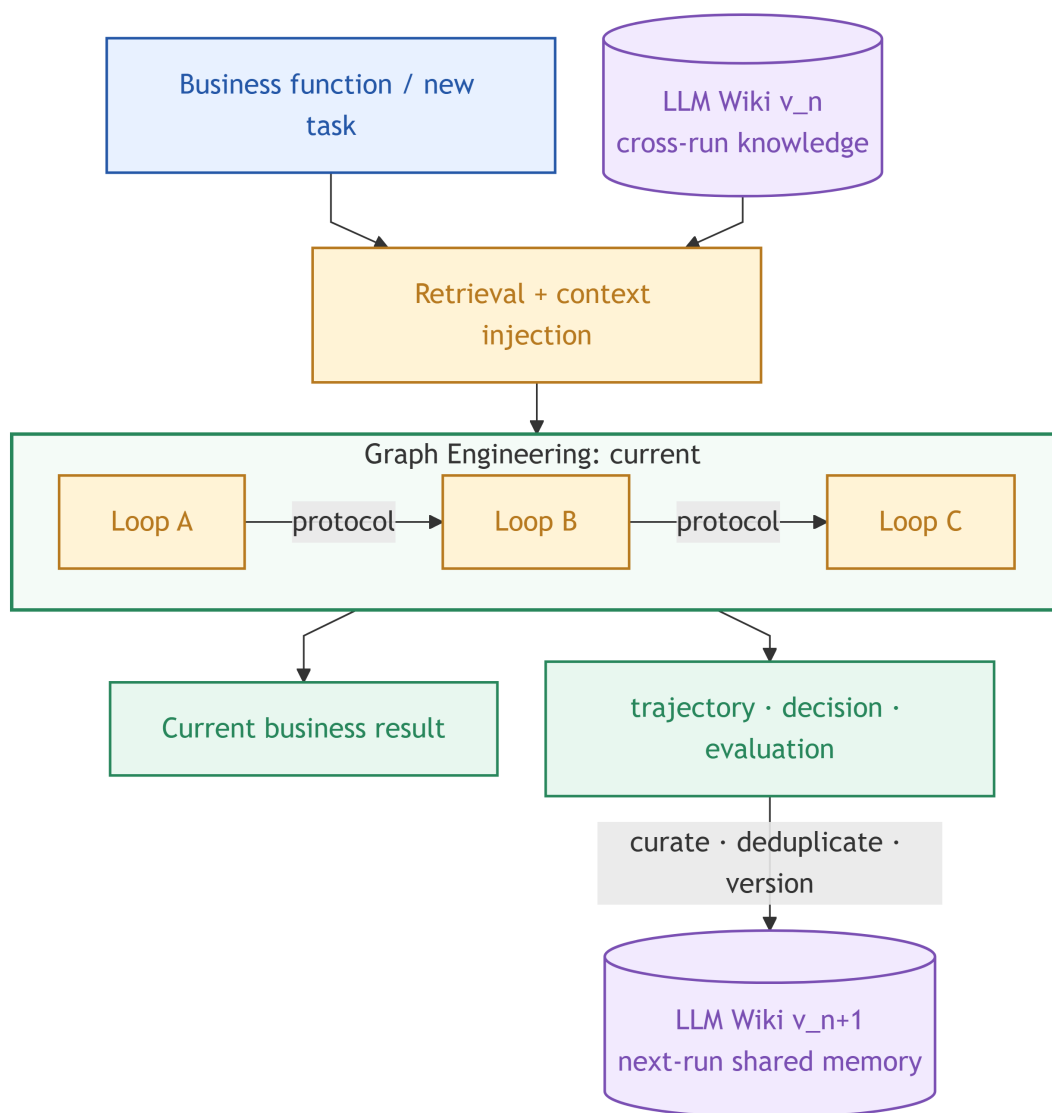


Figure 1.8: L7 Knowledge Driven Agent: execution experience becomes retrievable, reusable, and versioned shared memory

1.3.8 L8 Ontology Driven Agent: Move from knowledge recording to knowledge production

L8 places the Ontology before both the Wiki and the execution graph. Concepts, types, relations, and constraints define the acceptable shape of knowledge before candidate facts may enter the next version. Semantic consistency is an admission condition, not a guarantee that a claim is true.

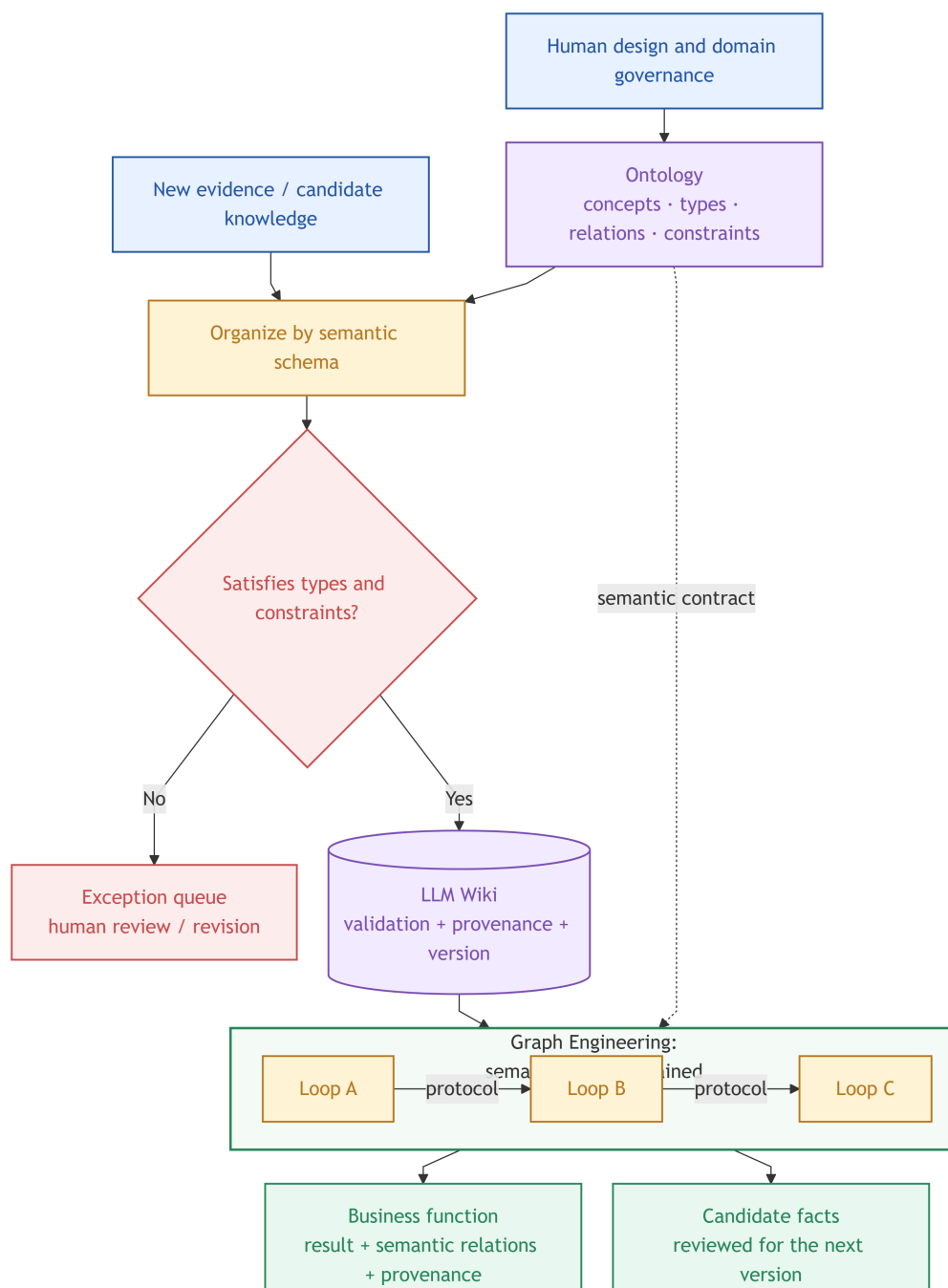


Figure 1.9: L8 Ontology Driven Agent: concepts, relations, and constraints guide knowledge organization and graph-engineering execution

1.3.9 L9 Causal Driven Agent: Identify the problem before claiming a causal answer

L9's evaluation gate checks treatment, outcome, confounding, temporal order, and identification assumptions instead of merely comparing the answer's surface form. The figure packages one run's conclusion as versioned evidence for a later independent run: an unidentified question records gaps and required evidence; only adequate assumptions permit a conditional causal candidate.

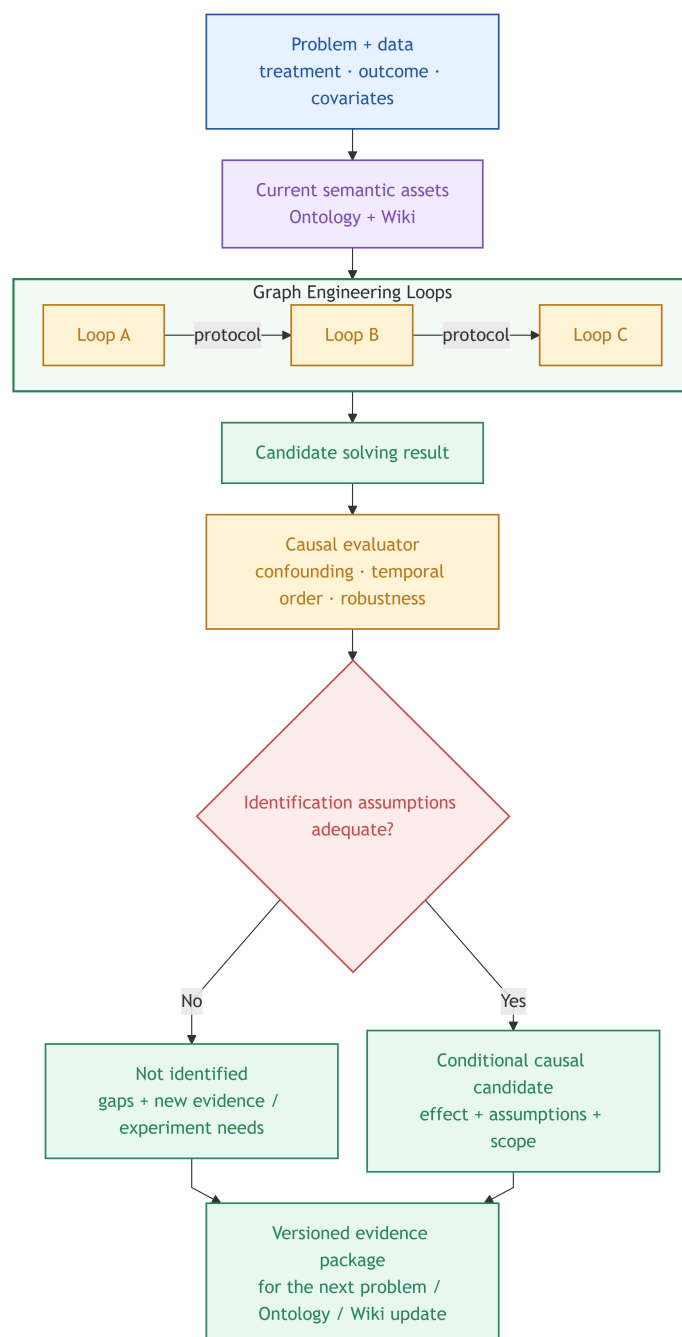


Figure 1.10: L9 Causal Driven Agent: a causal question and identification gate drive the joint evolution of the Ontology, Wiki, and solving evidence

1.3.10 L10 First-Principles Driven Agent: Redefine the problem and propose falsifiable innovation

L10 does not accept the inherited problem definition directly. It reframes the objective and success criteria, separates facts from conventions, searches for invariants, and derives a mechanism before asking the Ontology, Wiki, and Graph Engineering system to run a falsifiable experiment. Failed evidence becomes the input to the next clean-sheet reconstruction.

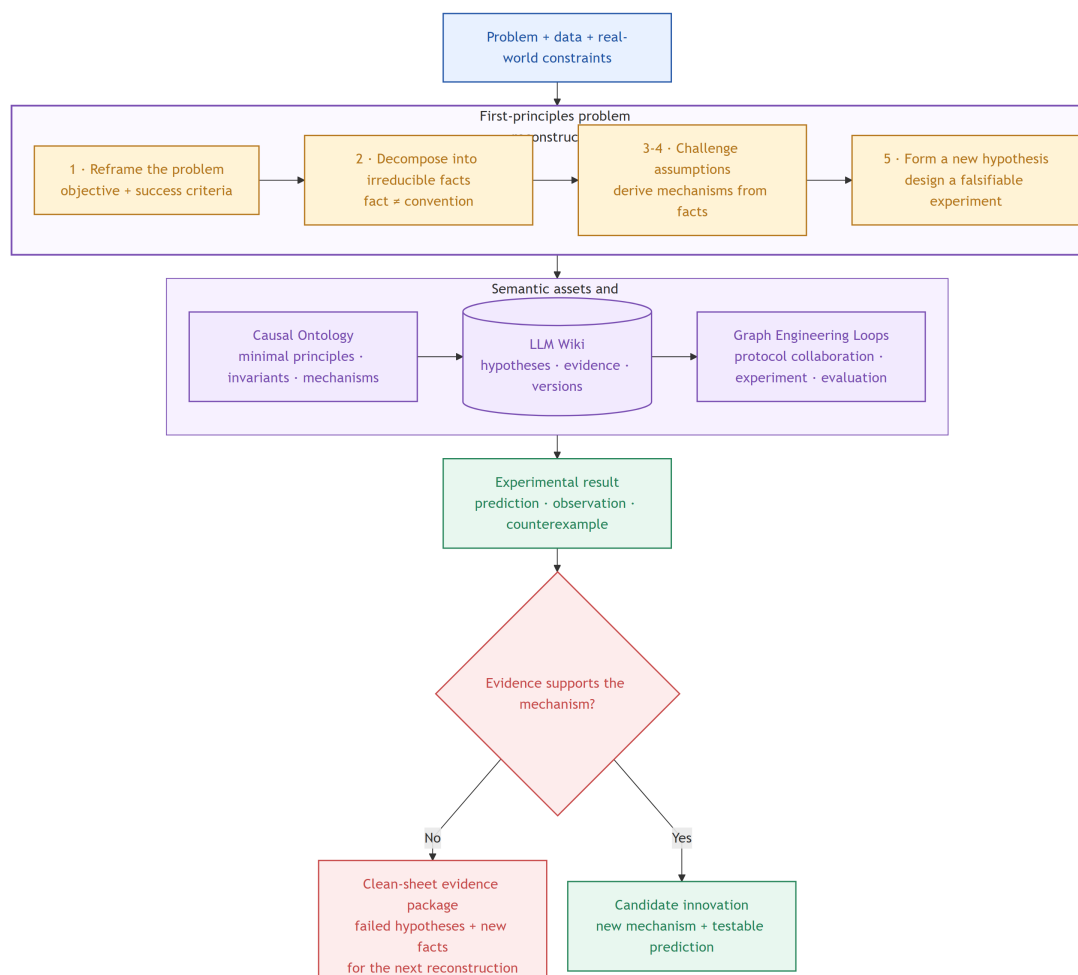


Figure 1.11: L10 First-Principles Driven Agent: irreducible facts and invariants lead to falsifiable experiments and candidate innovation

1.4 How to read this book

On a first reading, proceed from beginning to end. Every level addresses the same store-comparison question, so the differences come from the control surface rather than from a changing case study. On a second reading, you can go directly to the selection matrix in Chapter 16 and then return to the relevant chapter.

Every chapter answers five recurring questions:

1. Why is the previous level insufficient?
2. Which kind of control does this level relocate, and where does it go?
3. How does the repository implementation actually run?
4. How does the offline teaching path differ from the live-service path?
5. What conditions justify moving to the next level?

1.5 What this book does not promise

This book is not a production security certification, and it does not treat framework names as proof of capability. This matters especially at Levels 9–10: Sakila contains observational transaction data, with neither randomized assignment nor sufficiently rich time-varying covariates. The book therefore does not claim that any causal effect has been identified. A DAG or experiment design merely expresses “what we still do not know” more precisely.

2 Introduction: Agent Upgrades Are Migrations of Control

2.1 Model capability is not system capability

Connecting a stronger model to a database will often improve natural-language understanding and SQL generation. It does not automatically create permission boundaries, conflict arbitration, stopping conditions, knowledge versioning, or causal identification. Model capability answers “how well might it perform?” System capability answers “what is it allowed to do, what happens when it fails, and can its evidence be audited?”

This project separates the latter into ten levels. A higher level does not mean that every task should use it. A read-only SQL assistant will often need no more than Level 2. Level 3 becomes worthwhile only when cross-process collaboration must be auditable, and Level 7 is necessary only when the system must accumulate knowledge across runs.

! The ten levels are not a simple stack

Each chapter emphasizes one dominant mechanism. The implementation reuses the data kernel from earlier levels, but it does not guarantee that every level fully stacks every preceding third-party runtime. For example, Level 6 focuses on a typed harness and schema graph; it does not launch the OpenAI Agents Runner again.

2.2 The Ten-Level Roadmap: Two Layer Domains and Seven Stages

When read only through tool names, the ten levels can look like ten Agent Frameworks stacked on top of one another. A more accurate reading follows **how the dominant object of control moves upward**: from prompt context, tools, and communication to behavior, trajectory, and feedback, and then to data relationships, knowledge, semantics, causality, and foundational assumptions. Components may be reused, but a distinct engineering question defines each level.

The roadmap therefore has two domains. **L1–L5 form the Agent Execution & Control Layer**: L1–L2 first make an intention executable, while L3–L5 make multi-agent action communicable, arbitrable, and responsive to feedback. **L6–L10 form the Agent Intelligence Layer**: instead

of managing only one execution, the system makes structure, knowledge, semantics, causal models, and foundational assumptions inspectable, revisable, and testable objects.

L5 → L6 is the roadmap's pivotal boundary. L5 can already replan a trajectory from failure, but its primary object is still "how should this run proceed?" L6 begins to state explicitly which types, constraints, and relationships exist in the database schema. That stable substrate allows later levels to accumulate knowledge, constrain concepts, pose causal questions, and finally inspect the assumptions behind the problem. "Agent Intelligence" is a **systems-engineering domain name** here; it is not a five-step scale of model intelligence and is not evidence that AGI has been achieved. Nor is it a capability certification for a model, developer, or organization.

The ten levels compress into a seven-stage semantic progression. **Execution** turns intention into a run. **Control** gives that run boundaries and feedback. **Structure** makes data relationships explicit. **Knowledge** allows results to accumulate across runs. **Semantics** keeps concepts and constraints coherent. **Causality** reframes correlation as an identification problem. **Innovation** returns to objectives, invariants, and principles to propose falsifiable candidate mechanisms. Later stages reuse the earlier evidence contracts relevant to their problem; they do not replay every preceding runtime. "Higher level" therefore means neither a simple cumulative stack nor "necessarily more correct."

The ten levels in one sentence

L1 makes an LLM follow a workflow; L2 lets an agent use code-based skills to invoke authorized tools; L3 lets agents collaborate through protocols; L4 lets the harness arbitrate competing proposals; L5 lets agents iterate through feedback; L6 lets agents operate on typed relationships; L7 lets agents accumulate knowledge; L8 lets agents use an ontology to organize verifiable candidate semantic assets; L9 lets agents model causal problems and identification gaps; and L10 lets agents redefine problems from first principles and propose falsifiable candidate mechanisms for innovation.

2.3 Acting on an Unknown Map: Why Local Intelligence Needs Global Control

The central difficulty of a real-world agent is not adding another model; it is connecting local inference to durable business semantics, permissions, and accountability. That yields this book's control problem: **the real world does not reveal its complete state to an agent; a model can compress only local observations, while goals, objects, permissions, actions, and accountability must persist in a higher-level structure.**

Public sources support only a narrow but auditable conclusion. Vendor announcements in 2024 say that Claude 3/3.5 became available through Palantir AIP and AWS infrastructure in a US government environment meeting IL6 requirements. AWS describes the intended uses as processing large volumes of data, identifying patterns, streamlining document work, and informing

critical decisions (Palantir Technologies 2024; Lian and Krpata 2024). Anthropic had previously confirmed only that Claude 3 Haiku and Sonnet were available to government users through AWS Marketplace and GovCloud (Anthropic 2024b). These sources support **product availability and a public architecture**, not an independent effectiveness study; specific operational outcomes and database-strategy claims without public primary evidence are not treated as verified facts.

2.3.1 The Cooperation Model Allocates Control Planes; It Does Not Merely Combine Three Vendors

Palantir's official documentation describes the Ontology as a development foundation connecting data, objects, properties, relationships, actions, functions, models, and applications, with governed context available to LLMs and agents (Palantir Technologies 2026). **Ontology and LLM are therefore not alternatives.** An LLM is useful for proposing interpretations, candidate relationships, and next steps from ambiguous language and local evidence. Ontology/AIP aligns those candidates with real business objects, permitted actions, and traceable workflows. AWS carries compute, identity, logging, and security boundaries. Humans retain mission purpose, risk tolerance, forbidden zones, escalation conditions, and irreversible decisions. The diagram is this book's synthesis of public material; it is not a claim that the current Ollama/Sakila runtime integrates Palantir, AWS, or Claude.

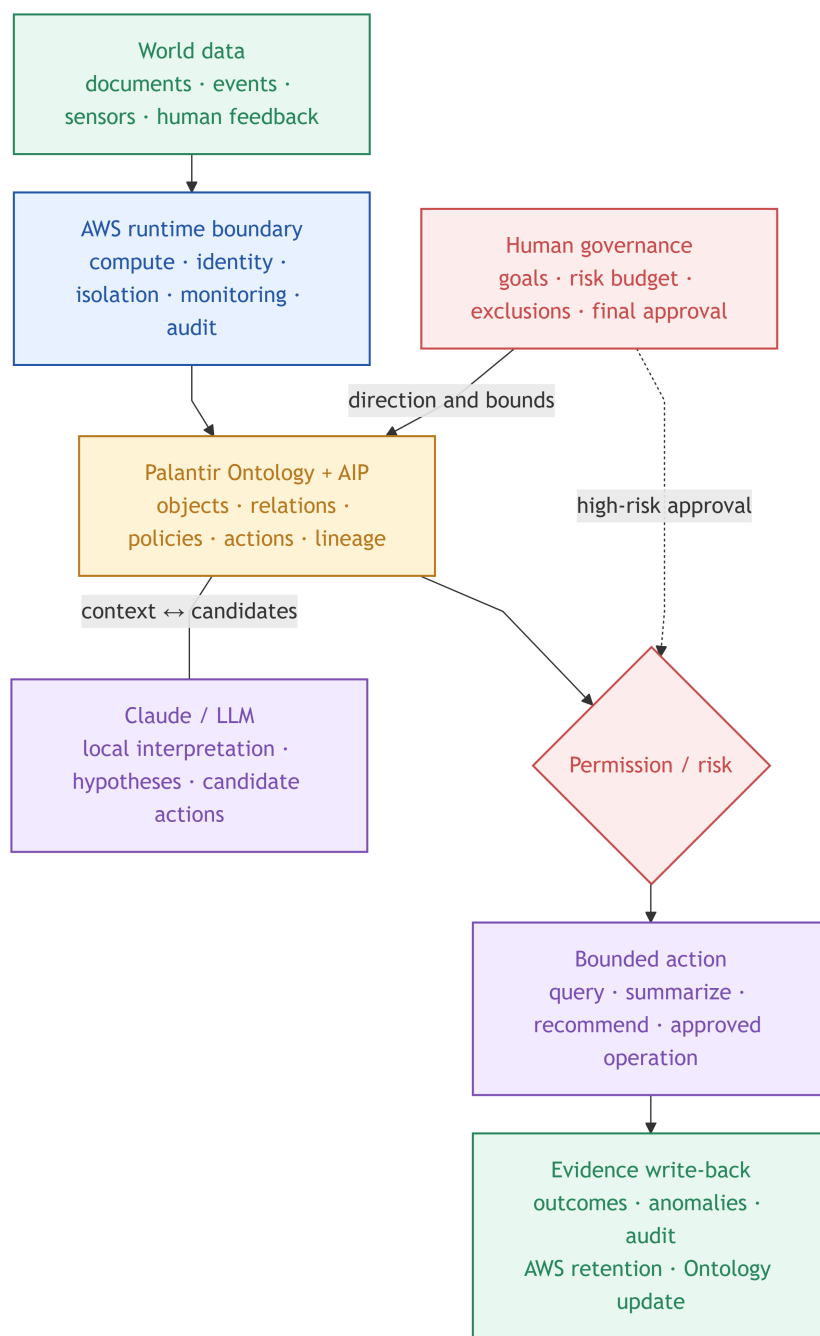


Figure 2.1: A control-plane reading of Anthropic–Palantir–AWS: local inference, business semantics, secure runtime, and human accountability

Table 2.1: Complementary control planes for LLMs, ontologies, and runtime infrastructure

Control layer	Problem it handles well	It must not be mistaken for	Evidence passed to the next layer
Human governance	Define mission value, risk budget, forbidden actions, stopping, and escalation	Micromanaging every step, or transferring accountability to a model	Executable goals, constraints, approvals, and exception rules

Control layer	Problem it handles well	It must not be mistaken for	Evidence passed to the next layer
LLM local inference	Read unstructured material, combine local clues, generate hypotheses and candidate actions	Possessing complete world state, a stable permission model, or intrinsic correctness	Hypotheses, supporting provenance, confidence rationale, and open questions
Ontology / AIP	Align language to objects, relations, actions, policies, and workflows	A knowledge graph that automatically guarantees truth, or a substitute for open-ended semantics	Object versions, relationship provenance, action authorization, and lineage
AWS runtime boundary	Carry compute, isolation, identity, monitoring, and audit	Understanding business meaning by itself, or independently proving application effectiveness	Call records, access decisions, runtime measures, and anomalies
World feedback	Reveal action outcomes, state changes, and new anomalies	A static, complete, noise-free training set	Replayable observations, counterexamples, drift, and outcome labels

2.3.2 The Hard Part of Massive Data Is Not Storing It but Preserving the Ability to Act

Operational systems mix tables, documents, logs, images, sensors, and human judgments. The same entity may have several identifiers, timescales, and permission domains. More volume amplifies conflict, latency, duplication, missingness, and unknown provenance; it does not automatically create knowledge. A systematic digital-twin review likewise identifies fidelity, data ownership, technical implementation, and integration among virtual entities as open knowledge gaps (Jones et al. 2020). Organizing massive data must therefore answer four connected questions: Which real object does a record denote? When and under what conditions is it valid? Who may take which action because of it? How will post-action evidence update the prior judgment?

Table 2.2: Digital-twin control problems in a complex world

Real-world complexity	Why the data is hard to organize	AI's local advantage	Global control still required
Heterogeneity and fidelity gap	Sources use different identifiers, grains, clocks, and permission domains; noise and missingness propagate with scale	Extract candidate entities, relations, and anomalies from text and local samples	Master identity, provenance, temporal semantics, permissions, and verifiable object mapping
Nonlinearity, coupling, and drift	Wear, environment, and policy changes invalidate old relations; local correlation is not a mechanism	Surface patterns, analogous cases, and explanations worth testing	Versioned state, physical/business constraints, drift detection, and counterfactual validation

	Why the data is hard to organize	AI’s local advantage	Global control still required
Real-world complexity			
Sparse labels and rare events	Valuable failures are rare, ground truth is costly, and the twin itself must be calibrated	Compress limited evidence into prioritized investigative hypotheses	Expert review, simulation/testing, confidence calibration, and uncertainty escalation
Real-time and compute constraints	Full recomputation lags state changes, while complex models add latency and cost	Select the locally most informative next step	Budgets, timeouts, degradation paths, incremental updates, and stopping conditions

2.3.3 “Exploring a Game Map” Is a Control-Theory Analogy, Not a Vendor Capability Claim

Research on partially observable decisions formalizes this predicament: an actor cannot see the complete state and instead maintains a belief about the world from observations, then revises it after acting (Kaelbling et al. 1998). ReAct interleaves reasoning and action in language agents, while Voyager demonstrates an exploratory structure that accumulates skills from feedback in an open environment (Yao et al. 2023; Wang et al. 2024). None directly proves enterprise or government deployment outcomes, but together they show why a complex world resembles a map under fog of war more than a table that one SQL query can exhaust.

In this analogy, high-level guidance is not step-by-step remote control. It defines the destination, forbidden areas, exploration budget, and conditions for stopping or calling a human. Within those bounds, the agent selects the next observation, proposes a local hypothesis, executes a reversible action, and writes the result back to the map. **AI’s advantage is local compression, not global omniscience.** The system-level upgrade delegates local exploration while elevating global goals, stable relationships, risk gates, and final accountability into executable control layers.

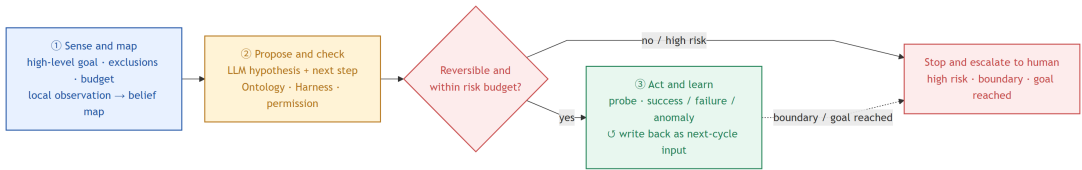


Figure 2.2: Bounded exploration of an unknown map: high-level direction, followed by observation–action–update cycles that reduce uncertainty

i Evidence boundary and implementation boundary

The partnership announcement is a vendor’s primary architecture statement, not an independent benchmark. POMDP, ReAct, Voyager, and the digital-twin review support a problem structure and research analogy, not deployed effectiveness. The control stack above is this book’s synthesis. The current demos still use local Ollama, Sakila, and layer-specific Python harnesses; they do not connect to Palantir, AWS, or Claude.

2.4 From Prompting a Model to Governing a Lifecycle: Anthropic's Year-by-Year Shift

Anthropic's official article on an AI-native SDLC argues that faster code generation shifts bottlenecks toward planning, review, testing, deployment, and governance (Claxton 2026). This book's distilled claim is not "let AI make every decision." The old control objectives remain, while execution, evidence, and enforcement must be redesigned as control migrates.

This is the meaning of "migration of control" in the chapter title. Final accountability does not migrate wholesale. Four control carriers change: tacit human knowledge becomes versioned context; line-by-line supervision becomes gates over plans and risk; verbal norms become tests, hooks, and sandboxes; and phase handoffs become machine-readable artifacts that can trigger the next step. Humans retain intent, risk acceptance, exception escalation, and final approval. Agents receive execution authority only inside observable, reversible, permission-bounded scopes.

2.4.1 2023–2026: Guidance Expands from Answers to Organizational Operation

When Anthropic launched Claude in 2023, its emphasis was a predictable and steerable assistant. The constitution published that year used explicit principles for model self-critique, revision, and training from AI feedback (Anthropic 2023b, 2023a). Control was primarily over **model behavior and a single interaction**: a human specified a task, a model produced a response, and explicit principles constrained the output.

In 2024 the guidance placed models inside runtime structures. *Building Effective Agents* distinguished predefined workflows from agents that dynamically choose their own steps and recommended starting with simple, composable patterns. MCP then recast connections to tools and data as standardized, bidirectional boundaries (Anthropic 2024a, 2024c). The control object expanded from "how should one prompt be written?" to **who chooses the next step and which resource can be reached through which interface**.

The 2025 shift occurred in repositories and harnesses. Claude Code guidance put `CLAUDE.md`, tool allowlists, plans, tests, and Git workflows into one development environment. Context engineering broadened prompting into the dynamic selection of system instructions, tools, MCP, external data, and message history. Skills packaged organizational procedures as discoverable resources; sandboxing used filesystem and network boundaries to reduce per-action approvals; and the long-running harness used progress files, Git history, and cross-session artifacts to preserve continuity (Anthropic 2025b, 2025c, 2025d, 2025a; Young 2025). Control moved out of moment-to-moment human operation and into **versioned repository knowledge, execution environments, and durable evidence**.

The 2026 material moves control to the lifecycle and organizational level. Agent-evaluation guidance treats tasks, trials, graders, and complete transcripts as regression objects. The application

harness uses planner, generator, evaluator, and structured handoff artifacts. Managed Agents separates session, harness, and sandbox into stable, replaceable interfaces. The AI-native SDLC applies these mechanisms across planning, design, build, test, deploy, and maintain (Anthropic 2026a, 2026b; Rajasekaran 2026; Claxton 2026). This is not a one-way increase in autonomy; **execution authority moves down while constraints, evidence, and accountability move up**.

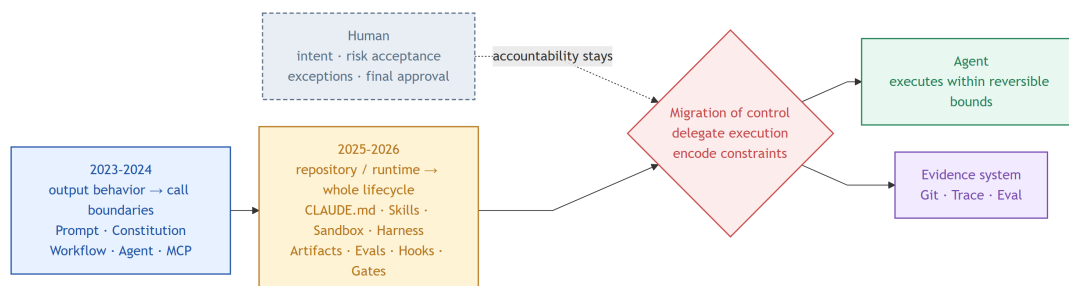


Figure 2.3: Anthropic’s 2023–2026 guidance: control moves from model output to the organizational lifecycle

Table 2.3: Year-by-year migration of control in Anthropic guidance and its evidence boundaries

Year	Primary object of official guidance	Change from the previous stage	Evidence boundary in this book
2023	Output behavior, steerability, and explicit value principles	Control resides mainly in model training, system/prompt instructions, and human turn-by-turn judgment	A constitution expresses normative intent; it does not prove behavioral compliance on every run
2024	Workflow/agent decision rights and MCP tool boundaries	Single responses become multi-step calls; simple patterns and standard connections are emphasized	A protocol exposes a boundary but does not supply authorization, recovery, or correctness by itself
2025	Repository context, skills, allowlists, sandboxes, and cross-session progress	Organizational knowledge and runtime restrictions become versioned, executable harness assets	Product practices are design evidence; gains depend on task, model, and environment
2026	Evaluations, session/harness/sandbox interfaces, and the six-stage SDLC	A coding assistant for one engineer expands into an artifact-driven organizational operating system	Official articles are practice guidance, not controlled evidence for every organization

2.4.2 The Artifact–Gate Loop

An AI-native SDLC can be abstracted as the following artifact–gate model: each stage commits a human-readable, machine-actionable artifact; accepting that artifact triggers the next stage; deterministic mechanisms enforce invariants; and a production anomaly produces the next intent, closing the loop (Claxton 2026).

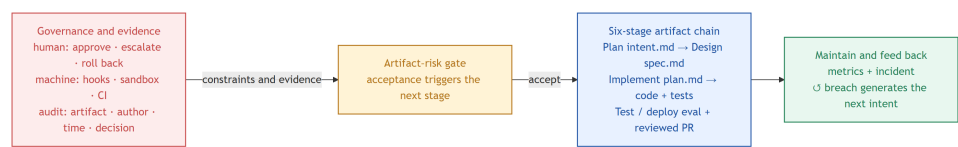


Figure 2.4: The AI-native SDLC artifact-gate loop: execution authority is delegated, risk accountability is not

Table 2.4: Reconstructed artifacts, enforcement, and accountability across the six SDLC stages

Rewritten stage	Versioned artifact and trigger	Deterministic machine control	Human responsibility retained	Mapping to this book
Plan	intent.md; acceptance triggers requirements and design	Template, schema, provenance, and revision-history checks	Confirm the problem, scope, constraints, and whether work should start	Level 1 fixed question and input contract
Design	spec.md; acceptance triggers plan mode	Load brand, security, compliance, and UX skills; report conflicts	Resolve policy conflicts and approve the specification	Levels 2 and 4 skills and conflict arbitration
Build	plan.md, code, and tests; writing starts only after plan approval	Read-only planning, path permissions, formatting/lint/secret hooks	Review high-risk approaches and deviations from plan	Levels 3–5 trace, policy harness, and loop
Test	Eval tasks, trials, graders, and transcripts	Tests, builds, static analysis, visual checks, and regression gates	Define success and calibrate subjective graders	Levels 5–6 regression and DeepEval boundary
Deploy	Reviewed PR with severity-ranked findings and a deployment record	Layered agent review, CI, approval hooks, and rollback path	Accept regulated or critical-code risk and approve production changes	Level 4 permissions and deterministic hooks
Maintain	Metrics, incidents, and the next intent.md	Control-band detection, diagnosis triggers, and gated action routes	Triage anomalies, decide rollback, and set next-loop priority	Levels 7–10 knowledge, causal questions, and experiment proposals

2.4.3 Trend Judgment: Speed Is Contextual; Migration of Control Is the Stable Variable

“Code is no longer the bottleneck” should be treated as a hypothesis to measure in each organization, not as a universal property of software work. A bounded controlled experiment reported that the Copilot group completed a JavaScript HTTP server faster, while a randomized study of developers working in familiar, mature open-source repositories reported that early-2025 AI tools increased completion time (Peng et al. 2023; Becker et al. 2025). The tasks, repository familiarity, quality standards, and tool forms differ, so neither percentage should replace the other.

Task structure supplies a more durable trend. SWE-bench combines real GitHub issues, repository context, and executable tests, with its early results showing that code generation alone could solve only a small fraction of problems (Jimenez et al. 2024). SWE-agent further shows that a purpose-built agent-computer interface materially changes repository navigation, file editing, and test execution (J. Yang et al. 2024). This book therefore does not conclude that “AI necessarily makes every engineer faster.” It concludes that **as action crosses more stages, quality increasingly depends on how context, interfaces, feedback, permissions, evidence, and accountability are orchestrated.**

! Migration of control does not remove humans from the loop

Low-risk, reversible, automatically verifiable execution can be delegated to agents. Irreversible deployment, regulatory exceptions, goal conflicts, and risk acceptance still require an identified human owner. Maturity is not the number of approval clicks removed; it is whether every delegation has a permission boundary, stopping condition, verifier, audit record, and escalation path.

2.5 A Typical AGI Evolution Narrative and This Book’s Position

This book uses “conversation → reasoning → agent → innovation → organization” to describe the engineering roles a system progressively assumes, while keeping that trajectory separate from the ANI → AGI → ASI narrative about breadth of capability. The role trajectory is an analytical frame, not a validated or unique AGI scale. The first five labels describe engineering capabilities and application forms; ANI, AGI, and ASI describe breadth. Names alone do not make the two progressions equivalent.

Primary research provides observable mechanisms for two capability stages. At the reasoning stage, intermediate steps can be elicited explicitly and reasoning behavior can then be trained with reinforcement learning (Wei et al. 2022; DeepSeek-AI 2025). Different studies establish components of the agent stage: Toolformer studies tool calls, ReAct interleaves reasoning and action, Reflexion uses linguistic feedback and episodic memory, Generative Agents combines memory, reflection, and planning, and Voyager accumulates reusable skills in Minecraft (Schick et al. 2023; Yao et al. 2023; Shinn et al. 2023; Park et al. 2023; Wang et al. 2024). Together, these components support a language-agent paradigm that places a model in an observation–reasoning–action–feedback loop; they do not imply that every study implements the complete loop. AgentBench also documents persistent gaps in long-horizon reasoning, decision-making, and instruction following (Liu et al. 2024).

“This project has entered narrow-domain agent engineering” must not be rewritten as “AGI has been achieved.” Legg and Hutter’s formalization associates intelligence with achieving goals across a wide range of environments (Legg and Hutter 2007). Chollet argues that evaluation should measure the efficiency of acquiring new skills rather than only scores on already trained

tasks (Chollet 2019). The Levels of AGI framework by Morris et al. further operationalizes these distinctions: it forms a two-dimensional capability classification from **generality (breadth)** and **performance (depth)**, while treating autonomy, risk, and human–AI interaction as deployment dimensions (Morris et al. 2024). The following figure explicitly separates the engineering narrative from an independent AGI evidence gate.

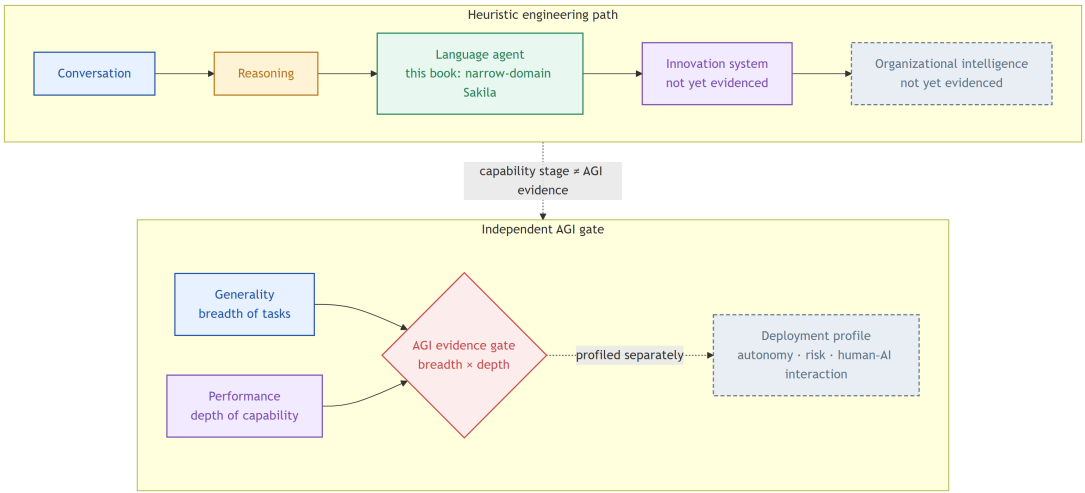


Figure 2.5: From an industry capability narrative to an independent AGI evidence gate: this book occupies narrow-domain language-agent engineering

The mapping below separates engineering roles from AGI evidence requirements so that stage labels do not substitute for capability evidence.

Table 2.5: Mapping the typical evolution narrative to this book’s ten levels and their boundaries

Typical stage or evidence gate	Increment in capability	Mapping to this book and its boundary
Task AI (prerequisite)	Complete one task inside fixed data and rule boundaries	Sakila is a fixed narrow domain; the read-only SQL invariant deliberately limits action scope
Conversation	Receive intent and generate responses in natural language	Level 1 supplies the interaction entry point; fluent expression is not autonomous action
Reasoning	Decompose multistep problems, generate candidates, and verify them	The levels may call reasoning models, but the book neither trains nor certifies general reasoning
Language agent (current engineering stage)	Combine tools, protocols, memory, feedback, and stopping conditions	Levels 2–7 demonstrate components for auditable narrow-domain agents; this supports “agent engineering,” not “AGI achieved”
Innovation system	Propose mechanisms and make them falsifiable with evidence or experiments	Levels 8–10 generate ontologies, causal questions, and experiment proposals; they do not autonomously conduct scientific discovery

Typical stage or evidence gate	Increment in capability	Mapping to this book and its boundary
Organizational intelligence	Coordinate roles, resources, permissions, and accountability over time	Harnesses and multi-agent orchestration are infrastructure, not organizational intelligence
AGI evidence gate	Jointly assess generality and performance; profile autonomy and risk separately	It is not a sixth engineering step; the book has no evidence of cross-domain transfer or open-world validation

Here, “mapping” means that the book studies engineering prerequisites for the corresponding capability stage; it does not mean that the stage has been achieved.

! This project has entered narrow-domain language-agent engineering, but it is not AGI

“Entered” is an architectural judgment here: models can now be placed inside control loops composed of goals, tools, observations, memory, and feedback. This book adds protocols, conflict arbitration, iteration, graphs, and knowledge governance. It nevertheless remains fixed to the Sakila domain, keeps database access read-only, substitutes offline paths for external services, and provides no cross-domain transfer or open-world autonomous validation. The outputs of Levels 8–10 are candidate knowledge, causal identification gaps, and experiment designs—not evidence that autonomous innovation or organizational intelligence has been achieved.

2.6 How Three Agent Paths Feed into the AGI Question

This book organizes agent engineering into three paths that can advance in parallel: **model-side agent RL**, **harness autonomy** composed of skills, memory, tools, and orchestration, and **ontology agents** that constrain the real world through business objects, relations, and actions. They may converge at the innovation and organization stages, but this is an analytical frame for control migration, not a consensus “three agent types” standard, and the paths are not three AGI levels.

Primary research makes the three paths more testable. AlphaZero shows that self-play, search, and outcome rewards can learn strong policies in games with complete rules; this is not general self-improvement in an open world (Silver et al. 2018). DeepSeek-R1 applies verifiable outcome rewards to reasoning tasks such as mathematics and code, while also reporting readability, language-mixing, and open-domain limitations (DeepSeek-AI 2025). Agent Lightning converts existing agent trajectories into reinforcement-learning transitions and separates training from agent execution, reporting improvements in text-to-SQL, RAG, and mathematical tool use (Luo et al. 2025). These results support “a model can update from controlled task feedback,” not “adding RL yields AGI.”

The harness path controls runtime context, tools, memory, verification, and stopping conditions. ReAct, Reflexion, Generative Agents, and Voyager offer partial evidence for interleaved action and reasoning, linguistic feedback, memory–reflection–planning, and skill accumulation, respectively (Yao et al. 2023; Shinn et al. 2023; Park et al. 2023; Wang et al. 2024). Product names such as OpenClaw and Hermes are time- and implementation-dependent, so this book discusses their underlying control surfaces rather than using popularity as evidence of autonomy. The ontology path controls world objects, relations, permitted actions, and invariants. An industrial digital-twin study validates ontology-based integration of domain knowledge and streaming data in an experimental setting (C. Yang et al. 2024), but semantic consistency does not automatically guarantee factual correctness, complete authorization, or safe physical action.

The following figure organizes model learning, runtime control, and real-world semantic constraints into one evidence-flow model.



Figure 2.6: Three agent-engineering paths cooperate through one evidence loop but do not automatically cross the AGI evidence gate

The following table compares the three paths by control object, acceptable evidence, and book boundary. It is not a product ranking.

Table 2.6: Control objects, evidence, and boundaries of the three agent paths in this book

Analytical path	Primary control object and acceptable evidence	Book location and boundary that cannot be crossed
Model-side agent RL	Control policy parameters, rewards, and credit assignment; inspect trajectories, reward changes, and held-out task performance	Level 5 demonstrates an iteration flywheel but does not train or <code>with-1.5:9b</code> ; controlled-task improvement is not cross-domain generality
Harness autonomy	Control context loading, tool permissions, memory, orchestration, verification, and termination; inspect traces, constraint rejection, and recovery	Levels 2–7 add skills, protocols, harnesses, loops, graphs, and a wiki; auditable execution is not complete autonomy
Ontology constraints	Control object types, relations, action semantics, and invariants; inspect mapping coverage, violations, and provenance	Levels 6 and 8–10 generate a schema graph, OWL/OBDA, causal questions, and experiment proposals; an ontology does not make knowledge true or action safe by itself
Three-path convergence	Let the harness collect evidence, the ontology define acceptable states, and the training loop update from feedback	The ten levels form a narrow-domain research scaffold; without cross-domain skill acquisition, open-world action, and durable accountability evidence, they do not establish AGI

⚠️ Forecasts are not capability evidence

Claims about DeepSeek V4, MT-OPD, DSec Sandbox, an OpenClaw/Hermes explosion, Ontology RL, and an “Agent OS” becoming organizational infrastructure are predictive or product-context statements. This book does not use those names as capability evidence. A more defensible conclusion is that the paths may advance simultaneously and supply

data to one another, while each capability still requires independent acceptance criteria for its environment, task, metrics, permissions, and failure record.

2.7 Two parallel pipelines

The most important mental model in this book is the separation of the data plane from the control plane:

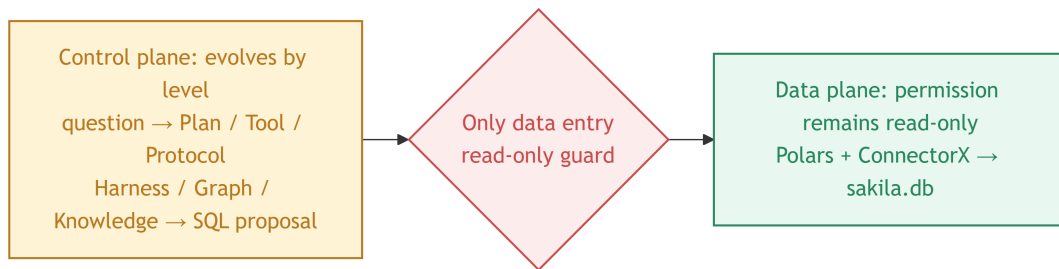


Figure 2.7: Separation of the evolving control plane and the read-only data plane

The control plane evolves from Level 1 through Level 10, while the data plane gains no additional permissions. At every level, data access must ultimately pass through `SakilaDB.query()`, which accepts exactly one `SELECT` or `WITH` statement and rejects writes, SQLite control statements, and stacked SQL.

This invariant matters. An agent that “knows more” should not thereby “receive broader database permissions.”

2.8 Offline mode does not mean synthetic data

The PDF build uses `offline=True` throughout so that it remains reproducible and independent of long-running services. Offline mode still:

- reads the real data/`sakila.db` database;
- uses Polars `read_database_uri` and `ConnectorX`;
- executes real `LangGraph`, `Pydantic`, `NetworkX`, `RDFLib`, and `Ralph` loops;
- generates real wiki, OWL, OBDA, and annotation files;
- passes through the same read-only SQL guard.

What it skips are nondeterministic or external dependencies: Ollama inference, the Codex app-server reviewer, the Omnigent server, and an explicit `DeepEval` judge. Every chapter marks this boundary.

2.9 One question throughout the book

Compare the two stores by revenue, rental volume, and customer count, and explain the observed differences.

Holding the question constant makes the evolution visible: the primary Level 1 artifact is workflow state; Level 3 introduces protocol traces; Level 7 introduces persistent documentation; Level 9 introduces identification gaps; and only at Level 10 is the business objective rewritten as an experimentally testable mechanism.

2.10 Reading conventions

The book uses four kinds of callout:

Implementation fact

Behavior that can be verified directly in the current repository code or execution output.

Teaching simulation

An alternative path used to keep execution offline, fast, and deterministic. It is clearly labeled and never presented as a real network protocol.

Optional external capability

A call that occurs only when the corresponding service is configured and running.

Production note

A requirement intentionally omitted from the current demo but necessary in a production system.

3 Environment and Reproducibility

3.1 Objective

This chapter connects the manuscript, executable code, and final PDF into one reproducible pipeline. Quarto passes Markdown, code cells, and execution output through Pandoc to produce HTML or PDF. This project uses .qmd chapters with the Jupyter execution engine (Quarto Project, n.d.-d).

3.2 1. Python project

Install all demo and book dependencies from the repository root:

```
python -m pip install -e "[harness,eval,dev,book]"
```

The core environment variables all have defaults:

```
$env:OLLAMA_OPENAI_BASE_URL = "http://localhost:11434/v1"  
$env:AGENT_LLM_MODEL = "ornith-1.5:9b"  
$env:SAKILA_DB_PATH = "D:\Python\agent_kungfu_levels\data\sakila.db"
```

The project deliberately uses AGENT_LLM_MODEL rather than the common but semantically unstable OLLAMA_MODEL, avoiding a name collision with host environments that use the latter for the model-storage directory.

3.3 2. Quarto and PDF

After installing Quarto, double-click the following file in the repository root:

```
render_book.cmd
```

You can also run the render directly:

```
quarto render my-book --profile zh --to pdf
quarto render my-book --profile en --to pdf
```

A Quarto Book declares its chapter order and output formats in `_quarto.yml`, and writes the complete rendered result to the `_book` directory (Quarto Project, n.d.-a). This project uses XeLaTeX with the Fandol CJK fonts bundled in TeX Live. The official Quarto PDF guide recommends explicitly selecting a font with coverage for the target language in Unicode documents (Quarto Project, n.d.-b).

3.4 3. Live Ollama path

The code in this book runs offline by default. To try the live model path, first verify the service:

```
ollama list
Invoke-RestMethod http://localhost:11434/v1/models
kungfu-agent run 2
```

Ollama exposes an OpenAI-compatible `/v1` interface. The project uses LiteLLM to send `ornith-1.5:9b` to this local endpoint. The OpenAI Agents SDK LiteLLM adapter is a third-party model path, so tool calling and usage reporting should be validated against the target provider (OpenAI, n.d.-b).

3.5 4. Three entry points with distinct purposes

Entry point	Purpose	Editor required?
<code>start_marimo.cmd</code>	Interactively select a level, question, and execution mode	No
<code>kungfu-agent run N</code>	Automation, scripting, or CI	No
This Quarto Book	Structured learning, executable notes, and PDF	No

3.6 Check

```
from assets.book_setup import DB, PROJECT_ROOT, SETTINGS

{
    "project_root": str(PROJECT_ROOT),
    "database_exists": SETTINGS.db_path.is_file(),
```

```
"sqlite_uri": SETTINGS.sqlite_uri,  
"tables": len(DB.table_names()),  
"model": SETTINGS.ollama_model,  
}
```

```
{'project_root': 'D:\\Python\\agent_kungfu_levels',  
 'database_exists': True,  
 'sqlite_uri': 'sqlite:///D:/Python/agent_kungfu_levels/data/sakila.db',  
 'tables': 16,  
 'model': 'ornith-1.5:9b'}
```

If `database_exists` is `True` and `tables` is greater than 0, the remaining offline chapters can execute. This check does not probe Ollama because PDF generation should not depend on whether a local service happens to be running.

4 Sakila: One Question, Ten Control Surfaces

4.1 Data source and security boundary

Sakila is a canonical relational database for a film-rental business. This project uses the SQLite file `data/sakila.db`. Every read converges on the same code path:

```
return pl.read_database_uri(query=clean, uri=self.settings.sqlite_uri)
```

`clean` is not a promise made by the model. It is the output of `validate_read_only_sql()`, which strips comments, requires exactly one statement beginning with `SELECT` or `WITH`, and rejects keywords such as `INSERT`, `UPDATE`, `DELETE`, `DROP`, `PRAGMA`, and `ATTACH`.

4.2 Data profile

```
from assets.book_setup import DB

key_tables = ["film", "customer", "rental", "payment", "inventory", "store"]
profile = DB.profile()
[{"table": table, "rows": profile[table]} for table in key_tables]
```

The database contains 16 business tables. `build_schema_graph()` extracts foreign keys from the DDL in `sqlite_master` rather than relying on a manually copied schema. The graph, wiki, and ontology therefore remain traceable to the actual database.

Table 4.1: Cardinality of key Sakila entities (read live from SQLite)

```
[{'table': 'film', 'rows': 1000},
 {'table': 'customer', 'rows': 599},
 {'table': 'rental', 'rows': 16044},
 {'table': 'payment', 'rows': 16049},
 {'table': 'inventory', 'rows': 4581},
 {'table': 'store', 'rows': 2}]
```

4.3 The baseline query used throughout the book

```
from assets.book_setup import DB
from agent_kungfu.database import QUERY_CATALOG

store_baseline = DB.query(QUERY_CATALOG["store_revenue"])
store_baseline
```

Table 4.2: Descriptive baseline for the two stores

store_id	revenue	rentals	customers	avg_payment
i64	f64	i64	i64	f64
2	33726.77	8121	599	4.15
1	33679.79	7923	599	4.25

In the current database, Store 2 has revenue of 33,726.77 and Store 1 has revenue of 33,679.79, a difference of 46.98. Store 2 has 198 more rentals, while both stores have 599 observed customers. Store 1’s average payment is 0.10 higher.

! Description is not causation

These values establish that the data contain differences. They do not prove that “making Store 1’s inventory resemble Store 2’s inventory would increase revenue.” Location, demand, pricing, title mix, and temporal trends may influence both inventory and revenue. Level 9 makes these identification problems explicit.

4.4 Why holding the question constant matters

If every level changed the data, question, and model, we could not determine where an improvement came from. This book holds constant:

- the database;
- the default question;
- the read-only boundary;
- the offline query catalogue;
- the `LevelResult` output contract.

Only the control mechanism changes from one level to the next. The shortest path to understanding the ten levels is to observe how answer, sql, trace, metadata, and artifacts evolve.

4.5 Validate the guard first

```
from agent_kungfu.database import validate_read_only_sql

attempts = [
    "SELECT COUNT(*) FROM rental",
    "DELETE FROM rental",
    "SELECT 1; SELECT 2",
]

guard_results = []
for sql in attempts:
    try:
        guard_results.append({"sql": sql, "result": validate_read_only_sql(sql)})
    except ValueError as error:
        guard_results.append({"sql": sql, "result": f"REJECTED: {error}"})
guard_results
```

```
[{'sql': 'SELECT COUNT(*) FROM rental',
  'result': 'SELECT COUNT(*) FROM rental'},
 {'sql': 'DELETE FROM rental',
  'result': 'REJECTED: Only SELECT or WITH queries are allowed'},
 {'sql': 'SELECT 1; SELECT 2',
  'result': 'REJECTED: Exactly one SQL statement is allowed'}]
```

This guard is neither a complete SQL parser nor a substitute for database-level permissions. It is one application-layer defense in a teaching project. A production system should also use a read-only database account, query timeouts, resource limits, and audit logging.

5 Experimental Method: Comparing Ten Levels Fairly

5.1 Research question

This book compares ten control mechanisms, not ten models. To avoid mistaking model randomness for an architectural benefit, the experiment fixes the data, question, model configuration, and result contract, changing only the dominant control surface.

The primary question is:

As a task evolves from one-shot SQL generation into multi-participant collaboration, iterative improvement, knowledge accumulation, and causal experimentation, which specific failure does each new level address?

5.2 Controlled variables

Variable	Fixed value
Data	data/sakila.db
Data access	Polars read_database_uri + ConnectorX
Default question	Compare the two stores by revenue, rental volume, and customer count
Online model	Ollama ornith-1.5:9b
Online endpoint	http://localhost:11434/v1
SQL permissions	One read-only SELECT/WITH statement
Output contract	LevelResult

5.3 Two experimental tracks

5.3.1 Track A: Reproducible offline experiment

Every executable cell in this PDF uses `offline=True`. This track validates the code structure, real SQL, database evidence, graphs, ontologies, and file artifacts. It is suitable for CI and reader reproduction.

5.3.2 Track B: Local live experiment

The live track exposes model decisions, tool selection, Codex protocol workers, the two-action Codex plan generated through OmniGenT, and the DeepEval judge. It requires additional services, and its output may vary with model version, context, and sampling.

Evidence levels

The book reports “successful offline cell execution” separately from “successful online integration validation.” The former cannot replace the latter, and an online smoke test cannot replace long-term reliability testing.

5.4 Observed measures

Every level records at least:

- answer: the final user-facing output;
- sql and row_count: the data evidence;
- trace: messages crossing responsibility boundaries;
- metadata: control evidence such as manifests, scores, graph summaries, and loop histories;
- artifacts: a wiki, OWL, OBDA, annotations, or candidate knowledge.

```
from agent_kungfu.contracts import LevelResult

[
    {
        "field": name,
        "annotation": str(field.annotation),
        "required": field.is_required(),
    }
    for name, field in LevelResult.model_fields.items()
]
```

5.5 Passing criteria

The deterministic SQL evaluator awards up to four components: 0.35 for a read-only statement, 0.35 for successful execution, 0.15 for at least one evidence row, and 0.15 for decision-relevant metrics. A total score of at least 0.9 is required to pass. For the store-comparison question, a query that does not group by store_id is capped at 0.74.

Table 5.1: Unified LevelResult contract

```
[{'field': 'level', 'annotation': "<class 'int'>", 'required': True},
 {'field': 'driver', 'annotation': "<class 'str'>", 'required': True},
 {'field': 'question', 'annotation': "<class 'str'>", 'required': True},
 {'field': 'answer', 'annotation': "<class 'str'>", 'required': True},
 {'field': 'sql', 'annotation': 'str | None', 'required': False},
 {'field': 'row_count', 'annotation': 'int | None', 'required': False},
 {'field': 'trace',
  'annotation': 'list[agent_kungfu.contracts.ProtocolMessage]',
  'required': False},
 {'field': 'artifacts', 'annotation': 'list[str]', 'required': False},
 {'field': 'metadata',
  'annotation': 'dict[str, typing.Any]',
  'required': False}]
```

This is not an “answer accuracy” score. It is an interpretable engineering gate that checks whether a query is safe, executable, nonempty, and decision-oriented. Levels 6–8 can explicitly enable DeepEval and use the same local model as a GEval judge, but the PDF build does not invoke it.

5.6 Current validation coverage and gaps

The repository’s automated tests cover offline smoke tests for all ten levels, real ConnectorX reads, the SQL guard, Level 4 bundle/skill validation and deterministic evaluation, the schema-graph path, two-round Ralph improvement, and ontology-file generation.

They do not yet automatically cover Ollama’s online stability, OpenAI Agents tool selection, the complete Codex app-server path, Level 4 bundle upload and Codex-harness execution on an OmniGenT server, the DeepEval judge, OWL consistency, or execution in an OBDA engine. Later chapters do not present these gaps as capabilities that have already been validated. Even when the Level 4 live path connects, a Windows Job Object demonstrates only process-tree and resource isolation; filesystem and network enforcement require separate verification.

5.7 How to interpret the results

When you encounter the output of a “higher” level, do not begin by asking whether it is smarter. Ask instead:

1. What new kind of structured artifact has appeared?
2. Which role gained or lost control?
3. Which failure can now be detected?
4. Once detected, is the failure rejected, degraded, arbitrated, or iterated upon?

5. What evidence is still missing before the conclusion can inform a real decision?

Part I

Part I — From Prompts to Tools

6 Level 1: Let Prompts Direct the Workflow

6.1 Capability Acceptance Experiment

Changing Top-N must change only the selected row count. Invalid N and model calls that change the requested N are rejected. Offline executes the workflow; boundary tests substitute the LiteLLM response.

```
kungfu-agent experiment 1
```

One-line takeaway: **The prompt carries task semantics, LangGraph defines step order, and code retains execution authority.**

6.2 From QA to Tasks: Why the Baseline Is Not Enough

The most basic approach is to concatenate the schema and the question into one large string, then trust the model to return the correct answer. This mixes together at least three distinct tasks: understanding the question, generating SQL, and executing SQL. If any one of them fails, all we learn is that “the model got it wrong.”

Agent engineering is shifting from demonstrations of basic generation toward reliable action in complex environments. That shift introduces control problems in long-horizon planning, memory, tool integration, risk control, and multi-agent governance. This is how the book organizes the engineering questions; it is neither a strict product chronology nor a uniformly validated maturity standard.

This book therefore does not present “QA → agent” as a history in which one model class replaces another. It rewrites the change as an expansion of the **delivery contract**. Few-shot language models could already perform question answering and other tasks from textual descriptions and examples, but the experimental endpoint remained generated text (Brown et al. 2020). InstructGPT improved adherence to user intent on its evaluated prompt distribution, yet instruction following by itself did not establish planning or execution (Ouyang et al. 2022).

The key system boundary adopted in this book appears beyond text generation. WebGPT added browsing, search, and evidence collection to long-form answering, illustrating an intermediate form of “acting in order to answer” (Nakano et al. 2021). WebShop instead required sequential

navigation and actions in an interactive website and evaluated task success rather than word-similarity (Yao et al. 2022). ReAct studied alternating reasoning and environmental action (Yao et al. 2023); Toolformer studied when and how to invoke external APIs (Schick et al. 2023); AgentBench extended evaluation to reasoning and decision making in multiple interactive environments (Liu et al. 2024). These studies validate different components and should not be collapsed into a claim that reliable, general autonomous agents have already been achieved.

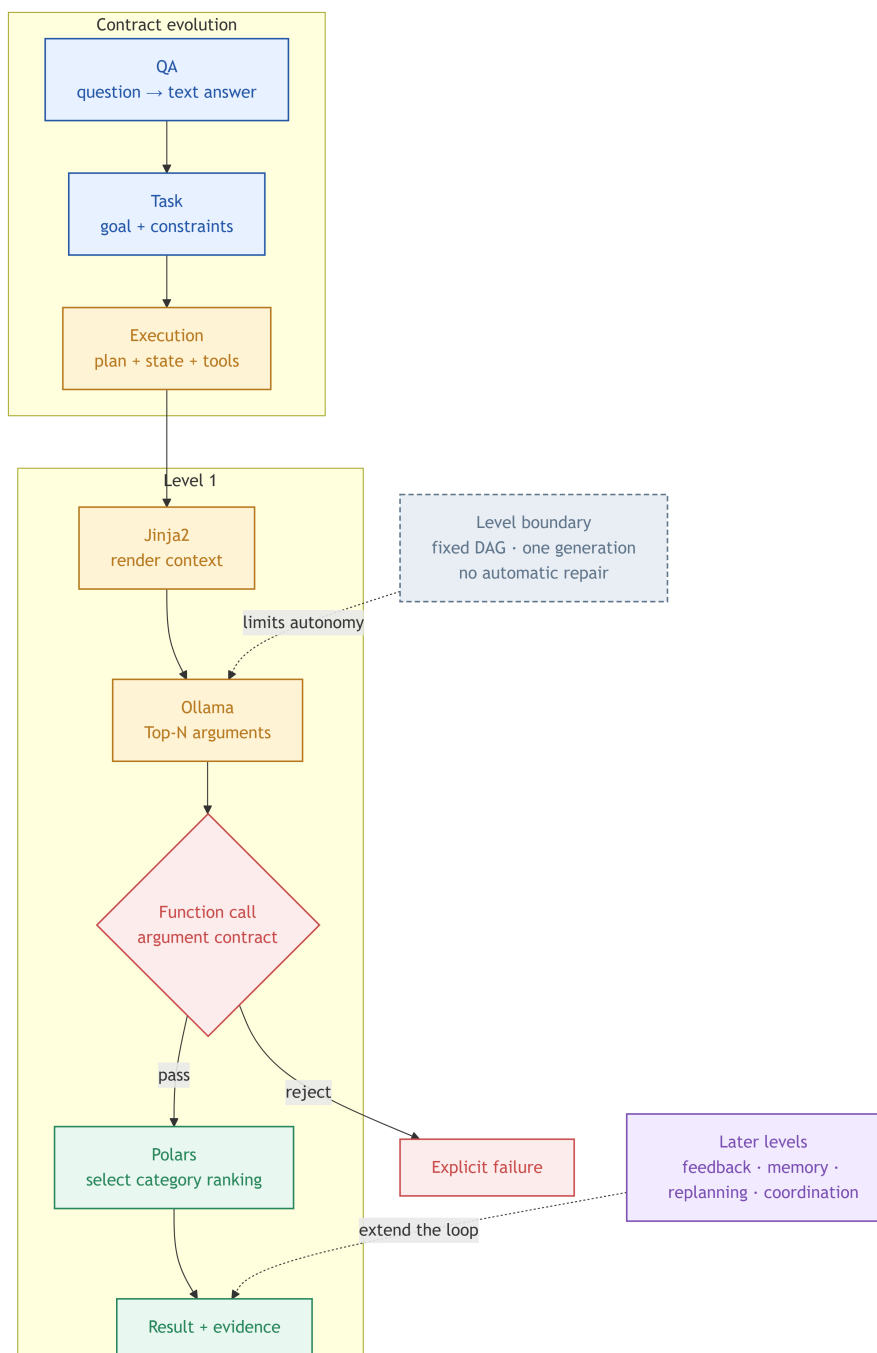


Figure 6.1: From an answer contract to the controlled task contract in Level 1

The figure shows a task contract rather than full execution order. Three LangGraph nodes first read five tables, join them, and aggregate revenue. Jinja2 then renders the question and N,

and LiteLLM/Ollama selects `top_n_categories(n)`. Code validates the function name, exact argument set, integer type, range, and agreement with the requested N. Reads use the SQL guard in `SakilaDB.query()` and `Polars/ConnectorX`.

Coordination, persistent memory, domain specialization, coding, and self-evolution are often placed together on an “agent capabilities” list. To avoid presenting aspirations as current behavior, the following table reorganizes them as inspectable control questions: which capabilities enter Level 1, and which must remain in later levels?

Table 6.1: The QA-to-task control contract and its Level 1 boundary

Control dimension	This chapter’s one-shot text-QA baseline	Task-oriented systems and the Level 1 boundary
Input contract	A question that needs an answer	Goal and constraints; the prompt supplies the question and N; code fixes the five reads and joins
Completion definition	A text answer	Checked against environmental results; this level checks category revenue, Top-N arguments, and three-step execution evidence
Control flow	One generation	A general system may loop; this level fixes the LangGraph DAG, which the model cannot rewrite
External action	Delivery does not require an explicit tool	This level executes only Polars/SQLite reads that pass application validation; it does not claim a read-only database connection
State and evidence	The question and answer	<code>RevenueWorkflowState</code> retains question, tables, table_rows, merged, and revenue
Failure handling	A wrong answer usually requires a person to retry or rephrase the question	Failure is localized to a node; this level has no automatic retry, reflection, or repair
Autonomy boundary	The model controls text output	The model only supplies Top-N tool arguments; code controls tools, validation, execution, and END

In this mapping, “domain specialization” means only a Sakila environment restriction and SQL safety boundary, not acquired domain knowledge. Multi-agent coordination maps to Levels 3–4, persistent memory to Level 7, coding/tool agents to Levels 2–6, and self-evolution only to the feedback and experiment mechanisms studied in Levels 5 and 10. The book studies engineering prerequisites for those capabilities; it does not mean Level 1 has achieved them.

Execution evidence is not semantic correctness. A legal SQL query that successfully returns data may still misunderstand the user’s question. Level 1 shows only that the control path and data provenance are inspectable; semantic evaluation, repair, and escalation remain the responsibility of later levels or human judgment.

! The precise position of Level 1

This level does not show that “the model has acquired autonomy.” It demonstrates a smaller but important transition: the prompt no longer specifies only answer content; it begins to supply task semantics to a deterministic workflow. It is a task-oriented, agentic workflow, not an autonomous agent; programs still execute the real actions, while SQL validation, database results, and the explicit endpoint provide execution checks rather than semantic proof.

Level 1 does not pursue sophisticated autonomy yet. Instead, it decomposes the process into an observable state machine. LangGraph is a low-level orchestration framework that lets developers explicitly define state, nodes, and edges (LangChain, n.d.); Jinja2 injects the user’s question and requested N into a template (Pallets Projects, n.d.).

6.3 Mental Model

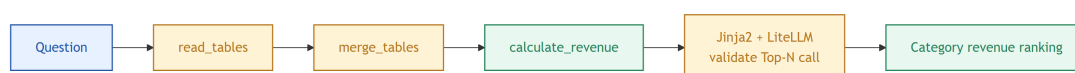


Figure 6.2: The prompt-directed workflow in Level 1

The arrows matter more than the word “Agent.” Each node reads only selected fields from the state and writes new fields back. If a read or join fails, we can locate it in `read_tables` or `merge_tables` instead of guessing what happened inside a monolithic black box.

6.4 Implementation Roadmap

`RevenueWorkflowState` is a `TypedDict` containing the question, five `DataFrames`, row counts, joined data, and revenue. `run_workflow()` executes exactly three nodes:

1. `read_tables` reads `category`, `film_category`, `inventory`, `rental`, and `payment` from configuration;
2. `merge_tables` joins on `category_id`, `film_id`, `inventory_id`, and `rental_id`;
3. `calculate_revenue` aggregates amounts by category name and sorts descending.

After the graph finishes, `live run()` asks LiteLLM/Ollama to call `top_n_categories(n)`; offline mode calls the same selector and labels the model step as simulated.

```

graph.add_edge(START, "read_tables")
graph.add_edge("read_tables", "merge_tables")
graph.add_edge("merge_tables", "calculate_revenue")
graph.add_edge("calculate_revenue", END)
  
```

i Implementation detail

Live Level 1 calls the model once to choose N. It neither generates SQL nor modifies the graph. `LevelResult.sql` is independent baseline SQL for comparison, not model output; actual results come from the Polars workflow.

6.5 Running Level 1 on the Shared Question

This section asks the default question: “Find the top 5 film categories by revenue.”

```
from textwrap import wrap

from assets.book_setup import DB, run_offline

level_01 = run_offline(1)
print(f"driver: {level_01.driver}")
print(f"rows: {level_01.row_count}")
print("nodes:")
for node in level_01.metadata["graph_nodes"]:
    print(f"  - {node}")
print("sql:")
print("\n".join(wrap(" ".join(level_01.sql.split()), width=68)))
```

driver: Prompt Agent: Polars 三步工作流 + LiteLLM function call

rows: 5

nodes:

- read_tables
- merge_tables
- calculate_revenue

sql:

```
-- Baseline: category revenue produced by the same inner joins as
Level 1. SELECT c.name AS name, ROUND(SUM(p.amount), 2) AS revenue
FROM category c JOIN film_category fc ON c.category_id =
fc.category_id JOIN inventory i ON fc.film_id = i.film_id JOIN
rental r ON i.inventory_id = r.inventory_id JOIN payment p ON
r.rental_id = p.rental_id GROUP BY c.name ORDER BY revenue DESC,
c.name ASC;
```

```
DB.query(level_01.sql)
```

Table 6.2: Independent SQL baseline for checking Level 1

name	revenue
str	f64
"Sports"	5314.21
"Sci-Fi"	4756.98
"Animation"	4656.3
"Drama"	4587.39
"Comedy"	4383.58
...	...
"Horror"	3722.54
"Children"	3655.55
"Classics"	3639.59
"Travel"	3549.64
"Music"	3417.72

Offline mode executes all three LangGraph nodes and real Polars operations, simulating only the model's Top-N selection. `metadata["rows"]` contains the selected N rows and `revenue_rows` records the full category count. The SQL table above is an independent aggregation baseline; passing offline does not establish prompt quality or real-model tool-calling ability.

6.6 SQL Guard Rejection Drill

This level does not let the model generate SQL. The following separate drill tests the shared database wrapper: textual instructions cannot replace a permission boundary when a future caller requests a write. Responsibilities are:

- The prompt narrows the space of intended behavior;
- The workflow isolates the location of a failure;
- The SQL guard applies application-level rejection rules;
- The database reports syntax and schema errors but does not judge whether the query answers the question correctly.

```
from textwrap import fill

try:
    DB.query("DELETE FROM customer")
except ValueError as error:
    print("rejected: True")
    print(fill(f"reason: {error}", width=68))
else:
    raise AssertionError("SQL guard accepted a write statement")
```

rejected: True

reason: Only SELECT or WITH queries are allowed

This drill directly tests the database wrapper and proves that application validation rejects a write statement. It does not prove that the SQLite connection or operating-system file permissions are configured as read-only, nor does it independently demonstrate LangGraph error state. During an actual graph run, the same class of exception propagates from `read_tables`.

This level has no automatic repair loop. If the live model produces invalid tool arguments, the selection step fails. That is exactly the class of problem Level 5 will address.

6.7 Reading Level 1's Limits Through an Evaluation Framework

An effective agent-evaluation chain begins when real tasks produce bad cases: failures must be recorded and classified before that evidence can drive optimization. Whole-task evaluation must also be separated from module-level evaluation of planning, tools, reflection, and memory. A related survey similarly distinguishes core capabilities, application scenarios, generalist agents, and evaluation frameworks while identifying realism, granularity, cost, safety, and robustness as open evaluation gaps (Yehudai et al. 2025). The book therefore rejects claims that an undefined level has “become mainstream” when neither the level definition nor supporting data is available.

Evidence has at least three layers: **outcome evidence** asks whether the task completed, **trajectory evidence** checks whether planning, tool use, and state transitions were valid, and **runtime evidence** covers safety, latency, and cost. When a gold answer exists, a reference-based check is usually the most reproducible. When no unique answer exists, a reference-free judge is only an evaluation signal with a recorded version and calibration. Synthetic cases expand boundary coverage, but they do not estimate the prevalence of failures in the real world.

6.7.1 Separate Whole-Task and Module Evaluation First

For this book’s Level 1, the whole-task question is whether the category ranking follows the requested metric and N—not whether the SQL merely runs. Module-level questions split into four parts:

1. **Planning:** live mode supplies Top-N arguments, without goal decomposition, alternative plans, or replanning.
2. **Tools:** code fixes one database read path; the model can only invoke one approved Top-N function and has no multistep function calling or complex parameter mapping.
3. **Reflection:** an exception propagates from `read_tables` and stops the run; `evaluate_sql()` is outside this level’s graph, and no feedback returns to the model.
4. **Memory:** `RevenueWorkflowState` serves one invocation; `LevelResult` retains result rows and call evidence, but not the full prompt, node latency, or cross-run bad-case history. Feedback text and episodic memory can form a different learning loop, but that is already beyond Level 1 (Shinn et al. 2023).

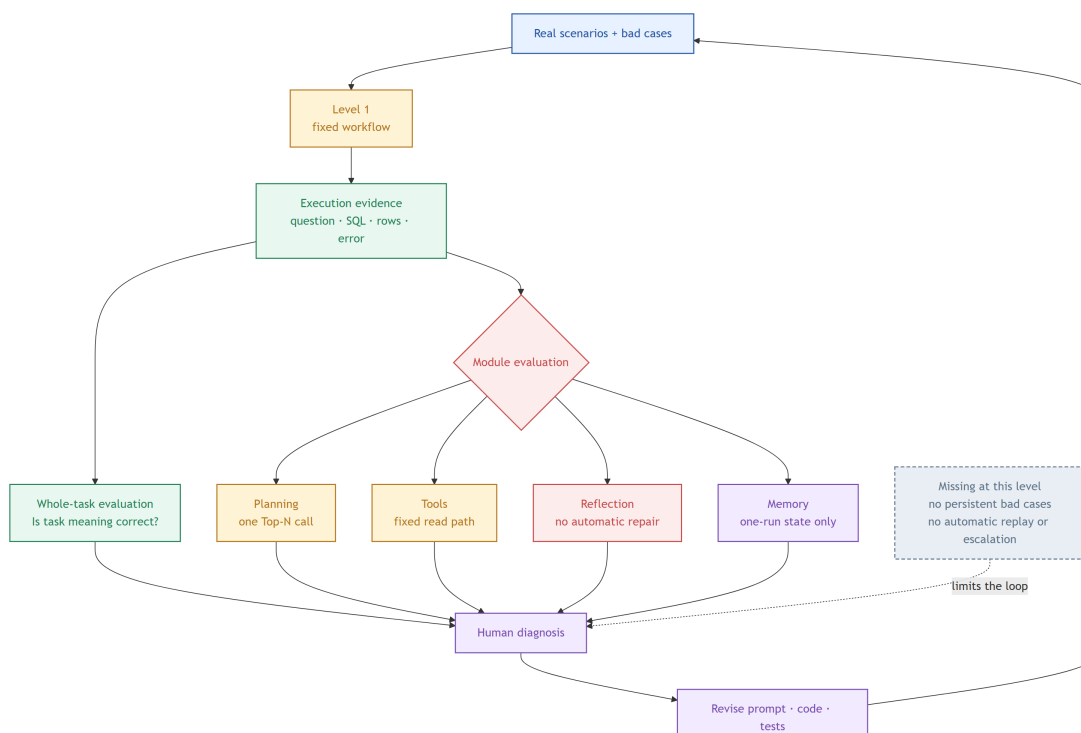


Figure 6.3: The Level 1 bad-case diagnosis flywheel

This diagnosis flywheel upgrades “the result was wrong” into structured evidence. Level 1 has observable nodes but no persistent evaluation record. Without a question, schema version, prompt, model parameters, SQL, exception, expected result, and decision rationale, a failure is not yet a replayable bad case.

6.7.2 Stress-Testing Five Scenarios

Table 6.3: Level 1 scenario limits and upgrade triggers

Scenario and associated evidence	What Level 1 can verify	Primary limit and upgrade trigger
Web applications: realistic sites require long-horizon interaction and functional correctness (Zhou et al. 2024)	Answer one read-only question about a site’s back-end data with SQL evidence	No browsing, multimodal state, or multi-turn recovery; requires Level 2 tool choice and the Level 5 feedback loop, while visual operation remains outside this book
Scientific assistance: evaluate individual workflow tasks, programs, execution results, and cost before end-to-end claims (Chen et al. 2025)	Query existing observational data and provide a traceable summary	No hypothesis testing, experiment design, or peer feedback; requires Levels 5 and 9–10, with human scientific review retained
Conversational service: dynamic users, APIs, databases, and domain policies must be evaluated together (Yao et al. 2025)	Handle one question through one fixed database path	No dialog state, policy conflict, clarification, or multitool transaction; requires Levels 2, 5, and 7
Software engineering: real issues require cross-file changes verified by tests (Jimenez et al. 2024)	Run fixed Polars aggregation; the shared SQL guard rejects writes	No repository reading, patch generation, test–repair loop, or project-level orchestration; requires Levels 2 and 5–6
Generalist agents: multi-turn, partially observable tasks need process metrics beyond final success (Ma et al. 2024)	Localize failure to read, join, aggregate, or Top-N selection along one path	No tool routing, trajectory progress, protocol correlation, or multi-agent conflict control; requires Levels 3–4 and then Level 5 for iterative improvement

The cross-scenario conclusion is not that “Level 1 performs poorly,” but that **its capability claim must use the same unit as its evaluation**. We may claim that a fixed data path is executable, localizable, and able to reject obvious writes. We may not claim multi-turn task completion, scientific autonomy, dynamic customer service, software engineering, or general tool orchestration. The historical offline query catalogue is no longer used here. The deterministic Top-N substitute supports workflow regression, but cannot replace real-model, interaction, and bad-case evaluation.

6.8 When to Stop at This Level

Level 1 is sufficient when the task has a single model-generation step, a fixed flow, very few tools, and failures that the caller can handle. Do not introduce multiple protocols or multiple agents merely to make the system look “more agentic.”

6.9 Upgrade Trigger

Level 1 has code call one fixed database path, and its TypedDict is only an internal state type. Move to Level 2 when the model must choose among a code-approved tool set and runtime schemas must validate each tool's inputs and outputs.

6.10 End-of-Chapter Exercises

1. Change Top-N from 3 to 5, then force the model to return the wrong N and observe contract rejection.
2. Why can't "do not write" replace `validate_read_only_sql()`?
3. If you remove the `read_tables` node, is this level still a data agent? What can it still provide?

7 Level 2: Move Capabilities from Prose into Code

7.1 Capability Acceptance Experiment

Offline supplies scripted model responses while running the real OpenAI Agents Runner, manager and specialist tools. Inspect `sdk_tool_calls` and verify revenue against the baseline.

```
kungfu-agent experiment 2
```

One-line takeaway: **The model chooses tools; code defines what those tools can actually do.**

7.2 From “Please Do This” to “You Can Only Do This”

Level 1 fixes reading, joining, and aggregation in code; the model only supplies Top-N arguments. Level 2 packages capabilities as skills: a QA leader delegates reading and calculation to two specialists through the Agents SDK, and each specialist can only invoke code-approved tools.

In the OpenAI Agents SDK, an Agent combines instructions, tools, and a model, while a Runner executes the model-tool loop. Python functions can be wrapped as function tools (OpenAI, n.d.-c, n.d.-d). The distribution used by this project is named `openai-agents`, and its import name is `agents`.

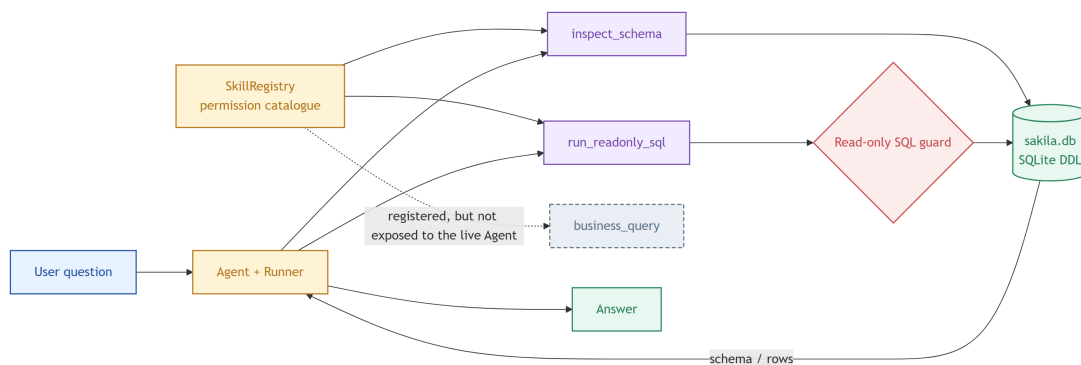


Figure 7.1: The code-defined skill permission boundary in Level 2

7.3 SkillRegistry Is a Permission Catalogue

The runtime registry exposes three capabilities:

Skill	Input	Output	Data permission
inspect_schema	None	Actual SQLite DDL	Read schema
run_readonly_sql	SQL	Up to 50 JSON rows	Read-only query
business_query	Query-catalogue key	Up to 50 JSON rows	Approved catalogue query

The independent registry example below exposes only its first two tools. The current main demo in `level2/agents.py` instead uses a QA manager with reader and calculator specialists exposed through `Agent.as_tool`. Code determines which tools are available.

```
@function_tool
def query_sakila(sql: str) -> str:
    """Execute one read-only SELECT/CTE and return up to 50 rows as JSON."""
    return registry.get("run_readonly_sql").invoke(sql)

agent = Agent(
    name="sakila-skill-agent",
    tools=[inspect_sakila_schema, query_sakila],
    model=model,
)

run = Runner.run_sync(agent, question, max_turns=6)
```

7.4 Running Code-Owned Skills

```
from assets.book_setup import DB, run_offline
from agent_kungfu.skills import SkillRegistry

registry = SkillRegistry(DB)
level_02 = run_offline(2)
{
    "registered_skills": registry.names(),
    "reported_sdk": level_02.metadata["sdk"],
    "rows": level_02.row_count,
    "sdk_tool_calls": len(level_02.metadata["sdk_tool_calls"]),
}
```

```
{'registered_skills': ['business_query', 'inspect_schema', 'run_readonly_sql'],
 'reported_sdk': 'openai-agents',
 'rows': 5,
 'sdk_tool_calls': 2}
```

```
try:
    registry.get("run_readonly_sql").invoke("DELETE FROM payment")
except ValueError as error:
    rejected_write = str(error)
rejected_write
```

```
'Only SELECT or WITH queries are allowed'
```

This failure occurs inside the Python tool. It does not require the model to “agree” or “understand.” That is what it means for code to own the permission boundary.

7.5 The Real Difference Between Offline and Live Modes

Teaching simulation

Offline Level 2 constructs the Agent team and calls the real Runner. LessonModel substitutes only model responses; the SDK executes nested specialists and real Polars tools. This verifies the SDK and data path, not the real model’s tool-selection quality.

Optional external capability

The live path uses LitellmModel with the same Agent team, allowing at most eight manager turns and two turns per specialist. `is_enabled` and workspace state enforce reading before calculation and reject repeated calculation.

In addition, `skills/sakila-analyst/SKILL.md` is a specification asset for humans and coding agents; the current Python runtime does not load it dynamically. This book treats it as a portable skill description, not as an implicitly active plugin.

7.6 OpenClaw: A General-Purpose Agent Engineering Breakthrough, Not Proof of General Intelligence

Progress in a general-purpose agent framework should not be judged by market narratives or product rankings. It should be separated into six testable themes: skills, continuous operation,

controlled action, collaboration, evolution, and generality. Each requires a corresponding run-time mechanism, permission boundary, and failure record; a complete component list is not evidence of general intelligence.

OpenClaw's main advance is not a new reasoning algorithm. It assembles previously separate capabilities into a continuously operating agent framework: `SKILL.md` carries discoverable procedural knowledge; the Gateway and session/queue machinery manage runs; the model-tool loop performs actions; Markdown files hold explicit memory; and tool policy plus sandboxing constrain side effects (OpenClaw Project 2026d, 2026a, 2026b, 2026c). This expands the task surface reachable by the same model without retraining its weights, which is an important breakthrough in **general-purpose agent engineering**.

Skill visibility, however, is not tool authorization. OpenClaw's documentation explicitly separates an agent allowlist from host-shell authorization, while memory files are state that can enter context rather than a mechanism that enforces policy (OpenClaw Project 2026d, 2026b). A skill description can tell the model *when and how* to act; code and the execution environment must still decide *whether the action is allowed, where it runs, and how it is stopped when it goes wrong*. A wider action radius expands both utility and blast radius, so permissions, budgets, timeouts, session isolation, and verification strength must tighten with it. That is precisely the Level 2 boundary.

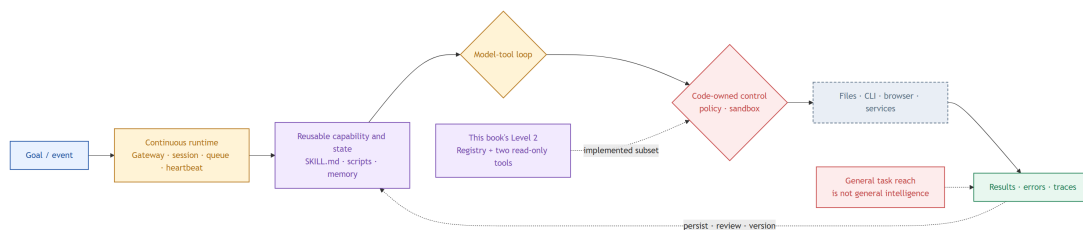


Figure 7.2: From skill packages to controlled action: OpenClaw's cross-layer lesson for general-purpose agent engineering

! Generality Has at Least Three Levels

1. A **general tool surface** can reach files, shells, browsers, and services.
2. A **general-purpose execution framework** can route, remember, schedule, recover, and constrain those tools.
3. **General intelligence** can reliably transfer, learn, and achieve goals on unseen tasks.

OpenClaw's architecture mainly advances the first two. A wider action radius does not by itself demonstrate the third.

The following engineering acceptance matrix uses this book's language of control surfaces, evidence, and upgrade triggers.

Table 7.1: An engineering acceptance matrix for OpenClaw

Rewritten theme	General-purpose progress and acceptance evidence	Book mapping and hard boundary
Reusable skills	Make procedural knowledge discoverable, filterable, and versioned; accept with selection records, argument validation, replay, and versions	Level 2; this project has an in-process registry and does not dynamically load SKILL.md
Continuous operation	Gateway, session, queue, heartbeat, and persistence carry work across turns; accept with timeouts, restart tests, compaction, and traces	Spans Levels 3, 4, and 7; a heartbeat is scheduler wake-up, not autonomy
Controlled action	Files, CLI, browser, and services create a broad tool surface; accept with allow/deny tests, isolation, least-privilege credentials, and side-effect logs	Levels 2 and 4; skill visibility is not authorization, and a sandbox is not an absolute security boundary
Multi-agent collaboration	Routing and child sessions expand parallel capacity; accept with correlation IDs, isolation, conflict rules, and total cost	Levels 3 and 4; more agents also amplify latency, cost, and inconsistency
Skill evolution	Distil candidate skills from successful and failed trajectories; accept with held-out gains, human approval, rollback, and version diffs	Level 5; writing a file is not learning and does not imply stable improvement
Generality claim	Cross-application coverage is broader than one chat box; accept only with execution benchmarks and failure classification	A general framework is not a generalizing agent, much less AGI

This separation also explains why adding more community skills does not directly imply general intelligence. ReAct established the value of interleaving reasoning traces with actions, and Voyager showed the usefulness of a reusable skill library in open-ended exploration (Yao et al. 2023; Wang et al. 2024). The newer SkillRL preprint studies hierarchical skill distillation from experience and joint evolution with a policy, but its evidence comes from ALFWorld, WebShop, and search tasks and therefore does not automatically transfer to arbitrary real environments (Xia et al. 2026). In the other direction, OSWorld reports that humans completed more than 72.36% of 369 real-computer tasks while the best model in that published evaluation achieved only 12.24%. The result exposes the large gap between connecting many tools and reliably completing open-ended work (Xie et al. 2024).

For enterprise practice, the safer conclusion is **SOP plus a bounded agentic path**. Frequent or high-risk tasks should follow deterministic workflows; infrequent, long-tail, and reversible tasks may explore within permission, budget, and review gates. Sakila Level 2 contains only a QA leader and two bounded specialists; writes are absent from the tool allowlist. OpenClaw

shows how this principle can expand into a cross-layer system; it does not establish that Level 2 already includes a Gateway, heartbeat, durable memory, or reinforcement learning.

7.7 What This Level Still Cannot Guarantee

- The model may select the wrong tool or provide invalid arguments;
- A function tool's docstring is not an authorization policy;
- LevelResult retains SQL, row counts, actual SDK tool calls and the model-boundary label; cross-process transport still needs protocol evidence;
- An in-process registry cannot explain where a cross-process message was lost.

The first two issues require guardrails and tests. The latter two push us toward protocols and observability.

7.8 When to Stop at This Level

A read-only data assistant that runs in one process, has only a few tools, and does not require cross-agent auditing can usually stop at Level 2. Investing in code-defined tools is generally more valuable than piling on more prompt text.

7.9 Upgrade Trigger

Move to Level 3 when the planner, reviewer, and worker belong to separate responsibility domains, or when you must trace which boundaries a unit of work crossed.

7.10 End-of-Chapter Exercises

1. Can the model call a skill that is registered in `SkillRegistry` but not exposed to the Agent? Why?
2. If you add an allowlist of tables to `run_readonly_sql`, should it live in the docstring or in code?
3. Is `max_turns=6` the same constraint as “invoke the query tool at most once”?

Part II

Part II — From Protocols to Improvement Loops

8 Level 3: Make Boundaries Traceable

8.1 Capability Acceptance Experiment

Tool actions cross serialized JSON-RPC requests and responses. Replayed IDs and unknown methods must be rejected before execution. A2A/MCP trace labels remain illustrative, not interoperability certification.

```
kungfu-agent experiment 3
```

One-line takeaway: **It is not enough for a capability to be callable; we must know who handed what to whom, and across which boundary.**

8.2 The Problem Beyond Tool Calls

Level 2 works well for a single-process tool loop. Level 3 moves the responsibility for reading tables and calculating revenue into `agent.md`; a factory dynamically generates Reader and Calculator Codex worker specifications from that file. Once responsibility crosses independent threads, function names alone are insufficient. Which manifest version produced a worker? Which Codex thread proposed an action? Did the host execute only an allowlisted Polars capability? Do the read and calculation evidence belong to the same request?

Level 3 no longer uses `openai-agents` as its runtime. It drives two independent threads through the Codex Python SDK and records each interaction in a unified `ProtocolMessage` envelope:

- `protocol`: this level records A2A, Codex app-server JSON-RPC, and MCP;
- `sender` and `recipient`;
- `kind`, such as `analysis.request`, `task.delegate`, `turn.completed`, or `tool.result`;
- `payload`;
- `correlation_id`;
- A timestamp and unique message ID.

8.3 Sequence of a Single Request

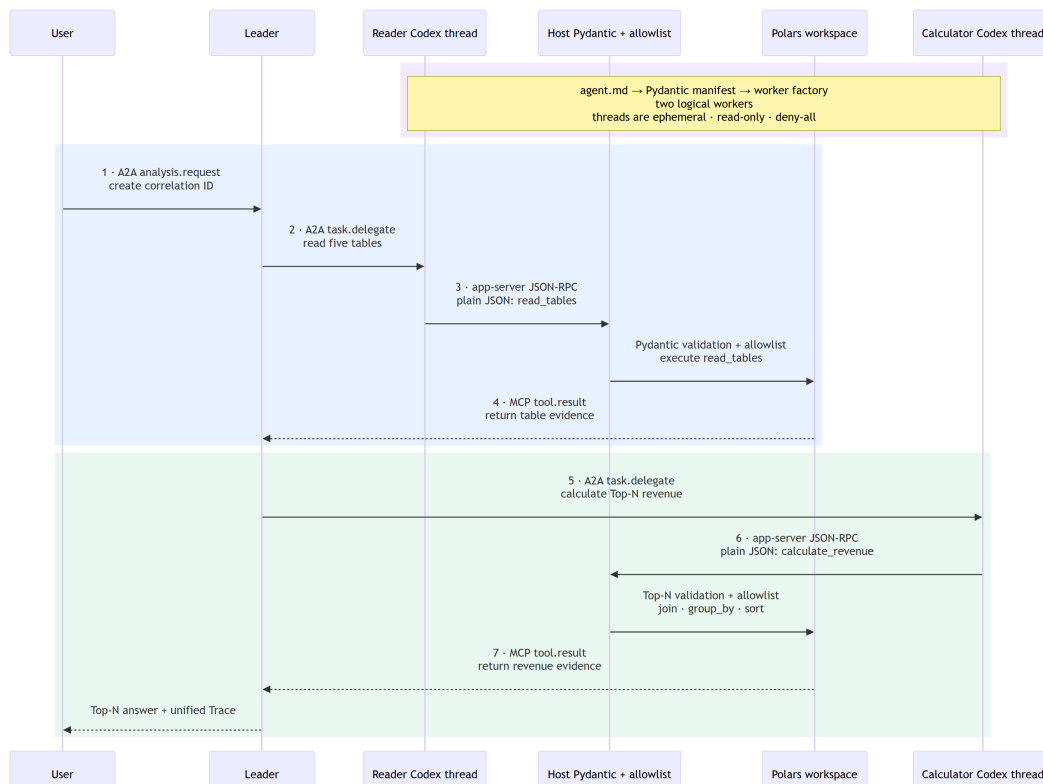


Figure 8.1: The actual Level 3 architecture: `agent.md` generates two Codex workers, while a host allowlist executes Polars

The purpose of a common envelope is not to erase protocol differences. It gives the observability layer a shared set of fields. A2A describes the task boundary between the leader and a worker, Codex JSON-RPC describes the thread/turn boundary between the SDK and the local app server, and MCP describes the evidence semantics of a tool result. They share one `correlation_id` in the trace while retaining different responsibility semantics.

8.4 Harness: Stability Is a System Output, Not a Model Property

Prompts, RAG, skills, memory, protocols, and orchestration are not mutually replacing layers; a runtime harness must govern them together. This yields an acceptance claim: **a harness is not “a pile of components around the model”; it is the evidence-and-control system that converts model capability into stable output.** Recent reviews similarly treat execution environment, tools, context, state, lifecycle, and validation as one governance object, while describing memory, skills, and protocols as externalized cognitive artifacts that require coordination (Meng et al. 2026; Zhou et al. 2026).

That stability cannot be inferred from “the model is strong.” Distributed-systems work used Dapper to demonstrate the diagnostic value of low-overhead traces across modules,

while OpenTelemetry uses trace IDs, span IDs, and context propagation to correlate signals produced in different processes (Sigelman et al. 2010; OpenTelemetry Project 2026). `ProtocolMessage.correlation_id` borrows the same idea, but it is a teaching envelope and does not claim to implement OpenTelemetry. Work on technical debt in machine-learning systems likewise shows that boundary erosion, hidden feedback loops, configuration, and changes in the external world make reliability a system-level concern rather than a property of one model or code fragment (Sculley et al. 2015).

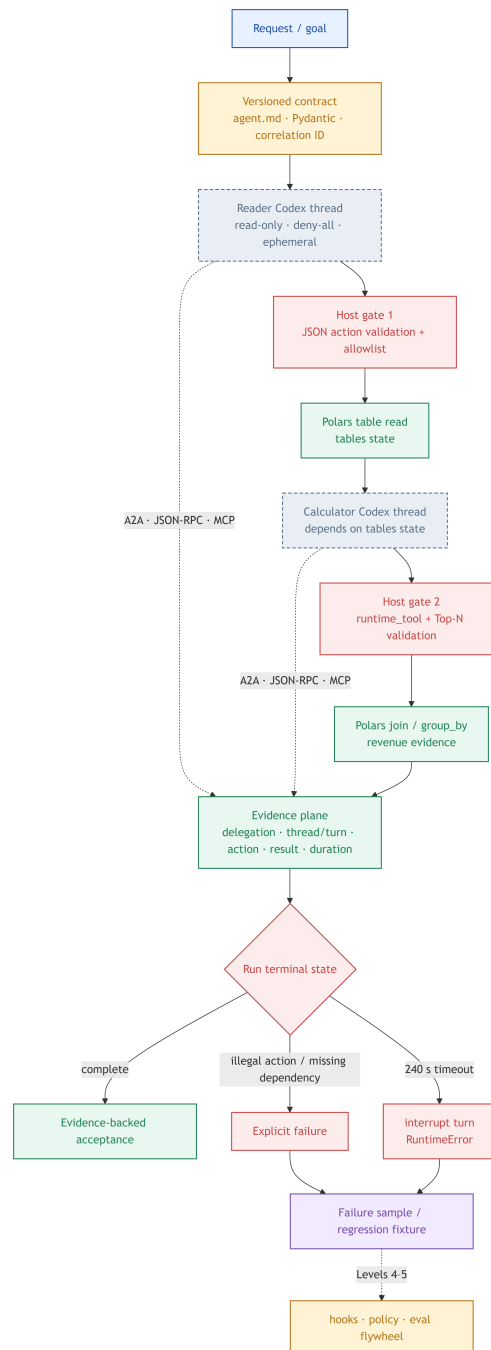


Figure 8.2: The harness stability supply chain: Level 3 first makes boundaries and failures traceable

“Stable output” in this figure does not mean every run succeeds. It means every run ends in an interpretable state: success has evidence, the host gate rejects an illegal action, and a timeout interrupts the turn and fails explicitly. The current Level 3 has no “continue with an old result after timeout” fallback, so it does not wrap failure as success. A final answer without stage evidence is one result. Once inputs, boundaries, status, and outputs are correlated, the system has a stability foundation that can be diagnosed, compared, and regression-tested.

⚠ Multi-Step Decay Is a Model, Not an Empirical Result

If all n strictly serial stages must succeed, and we temporarily assume independence, end-to-end success can be written as

$$P_{\text{end}} = \prod_{i=1}^n p_i.$$

With ten stages all set to $p_i = 0.8$, $0.8^{10} \approx 10.7\%$. This is a teaching model under an independence assumption, not an empirical estimate for real agents. Real stages are often correlated, and a harness changes effective reliability through validation, bounded retries, fallback, or human escalation. The formula’s real lesson is that a final result alone cannot reveal where the loss occurred.

The following matrix organizes runtime governance into seven control surfaces and checks each with six principles: progressive disclosure, architecture constraints, tests, hooks, independent audit, and failure regression.

Table 8.1: The harness stability control matrix

Rewritten control surface	Question for stable output and minimum evidence	Book level and hard boundary
Scope and context	What could this run read, and which context version was injected? Record scope, context hash, and provenance	Levels 1–3; Level 3 defines worker contracts in <code>agent.md</code> but does not yet record a manifest hash
Tools and protocols	Who called which tool, did arguments match the schema, and which request owns the result? Record sender, recipient, kind, protocol, and input/output summaries	Levels 2–3; this chapter implements <code>CodexAction</code> validation, a host allowlist, and a minimum correlation envelope
State and memory	Did state come from this turn, a session, or durable storage, and when was it committed? Record state version, read/write events, and commit outcome	Levels 3 and 7; the current demo has no cross-run durable memory
Lifecycle and scheduling	Which attempt ran, why did it retry or stop, and was fallback used? Record attempt, timeout, termination, and fallback reason	Levels 3–5; the current configurable limit is 240 seconds per Codex turn, after which it is interrupted and fails without automatic fallback

Rewritten control surface	Question for stable output and minimum evidence	Book level and hard boundary
Observability and cost	Which stage failed, and where were latency and tokens/cost spent? Record status, duration, usage, and parent/trace relationships	Level 3; the Codex turn event records status and duration, metadata can retain usage, but parent spans and cost conversion remain absent
Verification and feedback	Which test or judge passed, and can a failure enter the regression set? Record evaluator version, score, fixture, and disposition	Levels 5–6; Level 3 produces evidence but does not claim an evaluation flywheel
Environment and safety	In which snapshot, sandbox, and permission set did the run execute, and did side effects escape? Record environment digest, policy version, and side-effect log	Level 3 makes both Codex threads read-only, deny-all, and ephemeral, with a host allowlist; database identity, environment digests, and broader policies remain incomplete

The table defines what it means for a harness to become a system solution for stable output: it unifies execution contracts, state, tools, lifecycle, observation, verification, and environmental control so failures can be located and promoted into deterministic constraints. A 2026 survey describes the harness as a governance object spanning execution environment, tools, context, state, lifecycle, and verification, but its six-component decomposition is still conceptual and the authors explicitly identify empirical validation as future work (Meng et al. 2026). Another externalization review treats memory, skills, and protocols as external cognitive artifacts and the harness as their unification layer (Zhou et al. 2026). Both are recent preprints and should be treated as architectural hypotheses rather than mature standards.

Natural-Language Agent Harnesses attempts to make run-level policy an executable, comparable, and ablatability document object, suggesting that the harness itself can be inspected scientifically (Pan et al. 2026). Meta-Harness goes further by searching over harness code and reporting cross-model gains on several tasks, but that is a Level 5 optimization loop; it does not imply that a Level 3 trace can already self-improve (Lee et al. 2026). OpenAI’s Codex engineering retrospective likewise treats a legible environment, versioned knowledge, logs/metrics/traces, mechanical constraints, and feedback loops as foundations for reliable work. It is useful production evidence, not a controlled academic experiment (Lopopolo 2026).

i Removing a Homonymous Citation

Although arXiv:2605.23572 contains HARNESS - LM in its title, it studies small-model distillation and online latency for sponsored-search retrieval, not an agent execution harness, so this book does not use it as evidence for this chapter (Gupta et al. 2026).

8.4.1 Level 3 Is the Stability Foundation, Not a Complete Stability Guarantee

This chapter implements a common envelope, `agent.md` manifest validation, two independent ephemeral threads, correlation IDs, timestamps, message IDs, turn status/duration, host-side Pydantic action validation, a strict allowlist, and timeout interruption currently configured at 240 seconds. Reader and Calculator names are logical role labels in the host trace, not ephemeral-thread metadata. It does not yet provide parent spans, attempts, cost conversion, or environment digests, nor does it connect tests, hooks, judges, and failure samples into a loop. Level 3 therefore makes stability problems **locatable**; Level 4 deterministically arbitrates conflicts and permissions, and Level 5 continually converts failures into regression and improvement.

8.5 Tracing an Offline Run

```
import polars as pl
from assets.book_setup import run_offline

level_03 = run_offline(3)
roles = {
    "user": "User", "Dynamic Sakila QA Leader": "QA",
    "Dynamic Table Read Agent": "Reader",
    "Dynamic Revenue Calculate Agent": "Calculator",
    "codex-app-server": "Codex",
}
trace_rows = [
    {
        "protocol": message.protocol.value,
        "sender": roles.get(message.sender, message.sender),
        "recipient": roles.get(message.recipient, message.recipient),
        "kind": message.kind,
    }
    for message in level_03.trace
]
pl.DataFrame(trace_rows)
```

Table 8.2: Level 3 protocol trace

protocol str	sender str	recipient str	kind str
"a2a"	"User"	"QA"	"analysis.request"
"a2a"	"QA"	"Reader"	"task.delegate"
"codex-jsonrpc"	"Reader"	"Codex"	"turn.simulated"

Table 8.2: Level 3 protocol trace

protocol str	sender str	recipient str	kind str
"mcp"	"Reader"	"QA"	"tool.result"
"a2a"	"QA"	"Calculator"	"task.delegate"
"codex-jsonrpc"	"Calculator"	"Codex"	"turn.simulated"
"mcp"	"Calculator"	"QA"	"tool.result"

```
request_id = level_03.trace[0].id
{
    "messages": len(level_03.trace),
    "expected_seven_events": len(level_03.trace) == 7,
    "protocols": level_03.metadata["protocols"],
    "request_id": request_id,
    "all_followups_correlated": all(
        message.correlation_id == request_id for message in level_03.trace[1:]
    ),
}
```

```
{'messages': 7,
 'expected_seven_events': True,
 'protocols': ['a2a', 'codex-jsonrpc', 'mcp'],
 'request_id': 'f7c0c0ac-e7c8-4eea-a85b-b9fd31b5be3a',
 'all_followups_correlated': True}
```

The seven events comprise one root request plus two three-step worker groups: A2A task.delegate → Codex JSON-RPC turn → MCP tool.result. This assertion is stronger than inspecting log strings: a machine can verify that all six messages after the root belong to the same unit of work.

8.6 The Real Boundary of the Codex Workers

Live mode first validates the YAML front matter in `agent.md` as an `AgentManifest`, then dynamically constructs `Reader` and `Calculator` `CodexWorker` objects. They share one `Codex` app-server client context, but each worker starts its own ephemeral thread. `Reader` and `Calculator` are logical role names that the host retains in the manifest and trace:

```
manifest = load_agent_manifest()
workers = build_codex_workers_from_manifest(manifest, timeout_seconds=240)
config = CodexConfig(
```

```

    cwd=str(PROJECT_ROOT),
    env={
        "OPENAI_BASE_URL": settings.ollama_base_url,
        "OPENAI_API_KEY": settings.ollama_api_key,
    },
)

rpc = RevenueRpc(workspace, requested_n)
with Codex(config) as codex:
    for worker in workers:
        action, turn = worker.run(
            codex,
            settings=settings,
            requested_n=requested_n,
            workspace=workspace,
        )
        rpc.call(action.runtime_tool, action.n)

```

Inside `worker.run()`, the boundary is not an ordinary Python function tool call. The Codex Python SDK controls a local Codex agent through app-server JSON-RPC (OpenAI, n.d.-a):

```

thread = codex.thread_start(
    approval_mode=ApprovalMode.deny_all,
    base_instructions=worker.definition.instructions,
    model=settings.ollama_model,
    model_provider="ollama",
    sandbox=Sandbox.read_only,
    ephemeral=True,
)
handle = thread.turn(prompt, effort="none")

```

The Codex app server does not permit metadata updates for a thread created with `ephemeral=True`, so the implementation does not call `thread.set_name()`. Stable role identity comes from the worker name in `agent.md`, which becomes a host-side trace label rather than SDK thread metadata.

Both Reader and Calculator return only an ordinary JSON proposal with `runtime_tool`, `n`, and `summary`. The host parses it as `CodexAction`, checks that `runtime_tool` matches the manifest declaration, checks that the Calculator's `n` equals the user request, and only then maps the action through a strict allowlist to `workspace.load_tables()` or `workspace.calculate()`. This protocol does not expose SQL or Polars methods as worker actions; data execution accepted by the workflow can only cross the host allowlist. `read-only` prevents writes and `deny-all` prevents

privilege escalation, but neither means that Codex cannot attempt any read-only command. A production system should also narrow cwd/filesystem visibility and use a separate read-only database identity.

⚠ Structured-output compatibility with the local model

In the pinned Ollama + `ornith-1.5:9b` environment for this book, setting the SDK `output_schema` on a Codex turn produces a 400 error while initializing the sampler grammar. The current implementation therefore requests ordinary JSON and validates it with host-side Pydantic. This is a compatibility choice for the tested local combination, not a claim that all Codex models lack `output_schema` support.

This turn only converts a narrow routing intent into a JSON action and does not need expansive reasoning, so it uses `effort="none"`. The current configurable limit is 240 seconds per turn. On timeout, the host calls `interrupt()` and raises `RuntimeError`; it does not preserve an unvalidated old proposal. Timeout is therefore an attributable failure rather than an implicit degradation.

! Do not mistake labels for implementation

Offline mode starts neither a Codex app server nor real threads. It simulates the two actions under the same manifest and allowlist, marking Codex events as `turn.simulated`. The live path does start two ephemeral Codex threads, and both turns cross app-server JSON-RPC. In both modes, A2A and MCP are in-process teaching envelopes; no A2A or MCP network server is started.

8.7 What This Level Adds—and What It Does Not

It adds declarative worker specifications, dynamic Codex thread boundaries, read-only + deny-all + ephemeral constraints, host-side action validation and allowlisting, and a seven-event trace that correlates A2A, Codex JSON-RPC, and MCP. Together they form a minimal protocol harness that turns stability from a subjective impression into locatable evidence.

It does not add a selection rule for two valid but conflicting proposals. A protocol can faithfully record a conflict, but it cannot resolve that conflict by itself. That is the purpose of Level 4.

8.8 When to Stop at This Level

If responsibility domains are separated but each step produces only one candidate result, Level 3's trace harness provides sufficient diagnosability. Move to a Level 4 policy harness when competing proposals, permission rules, or mandatory hooks appear; move to Level 5 when failures must automatically enter an improvement loop.

8.9 End-of-Chapter Exercises

1. Why does a shared envelope for all messages not imply that every message uses the same real network protocol?
2. Why does one complete run produce exactly seven trace events? How are the six events after the root request grouped?
3. Even with `Sandbox.read_only` and `ApprovalMode.deny_all`, why does the host still need Pydantic action validation and a strict allowlist?
4. Which compatibility problem does “ordinary JSON + Pydantic” solve compared with SDK `output_schema`, and which validation responsibilities move to the host?
5. Why can one successful final answer not demonstrate harness stability? Name at least three kinds of stage evidence still required.

9 Level 4: Constrain Agent Behavior with a Harness

9.1 Capability Acceptance Experiment

Inject an executable, high-confidence proposal that doubles revenue and compare it with the correct proposal. Independent evidence must select the winner; reject delivery if no proposal qualifies. The conflict is an explicit experimental fixture.

```
kungfu-agent experiment 4
```

One move, one lesson: **OmniGenT asks exactly two Codex subagents to propose actions; the host executes only allowlisted actions and independently evaluates the run.**

9.2 The problem: traceability is not control

Level 3 can connect tasks, protocol envelopes, and results into a traceable chain. A trace, however, tells us what happened; it does not guarantee that an agent did only what was allowed. A model can still reorder steps, widen a parameter, omit a skill constraint, or return a plausible-looking result without verification.

Level 4 changes the unit of control from a protocol message to an action plan constrained by a harness. The current Sakila implementation establishes three consecutive boundaries:

1. **Generation boundary:** the OmniGenT bundle declares only a Reader and a Calculator, both backed by Codex;
2. **Execution boundary:** subagents may propose actions, while the host accepts only `read_tables` and `calculate_revenue`;
3. **Evaluation boundary:** the result must pass deterministic checks for structure, provenance, numeric validity, and SQL-baseline parity.

9.3 Architecture: one bundle, two subagents, one execution host

`omnigent-agent/config.yaml` defines the root harness and references two child bundles in a fixed order:

- **Polars Table Reader:** may propose only `read_tables`, with `n=null`;
- **Polars Revenue Calculator:** may propose only `calculate_revenue`, preserving the requested `n`.

Both declare `executor.config.harness: codex` and use the local Ollama model `ornith-1.5:9b`. The root and both child configurations also declare the same Codex `os_env: sandbox: auto`, `write_paths: []`, `allow_network: false`, and `start_in_scratch: true`; the OmniGenT 0.11 Codex executor starts turns with `approvalPolicy=never`. The host evaluator checks these configuration and runtime contracts. The portable hard capability boundary, however, remains the separation between proposal and execution: the protocol accepts only typed worker proposals, while the host authorizes only two allowlisted SQLite/Polars branches.

 Do not overstate the Windows boundary

Windows Job Objects constrain process trees and resources, but they do not enforce filesystem or network policy. On Windows, `sandbox: auto`, an empty `write_paths`, and `allow_network: false` are therefore defense-in-depth declarations; by themselves they do not prove an absolutely read-only filesystem or blocked network. Level 4's cross-platform safety claim rests on typed proposals, Pydantic validation, and the host action allowlist.

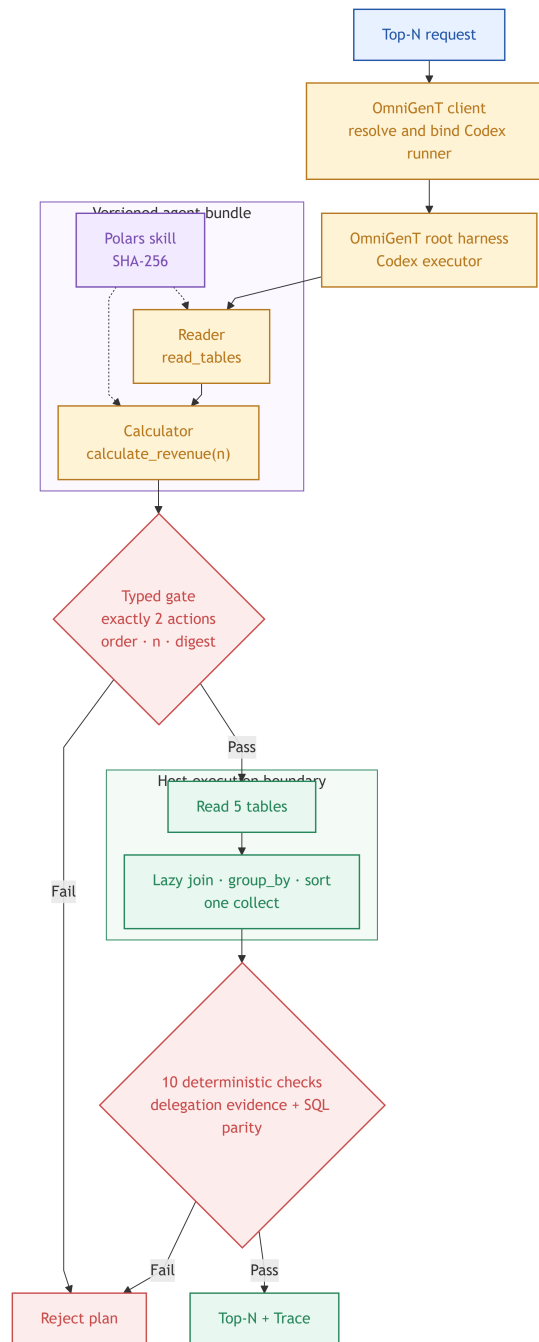


Figure 9.1: Level 4: OmniGenT generates two Codex subagents; the host allowlists execution and evaluates it independently

The design deliberately separates proposing an action from executing it. If the model invents a tool name, reverses the order, changes n , or returns the wrong skill digest, the plan is rejected before the host executes any database action.

9.4 The Polars skill is a runtime input, not a decorative label

`load_polars_skill()` reads `level4/polars/SKILL.md` and five required reference files. It validates frontmatter, directory naming, empty files, symlinks, and path escape, then computes a SHA-256 digest over names and content. The host-side worker definition constructs `base_instructions` from lazy API + contexts for the Reader and contexts + expressions + the insight recipe for the Calculator; the current runtime does not separately consume those role-specific subsets. In the actual bundle, `build_agent_bundle()` places the full skill and all references in the root-agent and both child-agent scopes, together with a source digest.

The level can therefore answer two provenance questions: **Which version of the skill informed the agents? Did both actions acknowledge the same digest?** If the skill changes, its digest changes and an old plan cannot be silently reused.

9.5 Host allowlisting: models plan, code authorizes

The model returns neither SQL nor Python. It returns a strict JSON action plan. `WorkerAction` limits the agent and runtime tool to two literal values and adds role invariants: the Reader's `n` must be null; the Calculator's `n` must be a positive integer. `_validate_plan()` then verifies that:

- the plan evaluation passed;
- the actions are exactly Reader followed by Calculator;
- both actions carry the current skill digest;
- the Calculator's `n` equals the user's request.

`_execute_plan()` has only two execution branches: `read_tables` and `calculate_revenue`. The host action layer has no arbitrary SQL, shell, file, or Python-eval branch. This is the harness boundary: the model has **proposal authority**, while host code retains **authorization and execution authority**.

9.6 One-collect Polars computation

The host's `read_tables` branch first loads `category`, `film_category`, `inventory`, `rental`, and `payment` through the read-only database adapter. The host's `calculate_revenue` branch then converts them to lazy frames, follows the four Sakila foreign-key joins, and groups, sums, and sorts:

```
summary = (  
    category.join(film_category, on="category_id")  
    .join(inventory, on="film_id")  
    .join(rental, on="inventory_id")
```

```

.join(payment, on="rental_id")
.group_by("name")
.agg(
    pl.col("amount").sum().round(2).alias("revenue"),
    pl.len().alias("_joined_rows"),
)
.sort(["revenue", "name"], descending=[True, False])
.collect()
)

```

The join, aggregation, and sort form one optimized plan with a single terminal `collect()`. Joined-row cardinality is computed inside the same aggregation, avoiding a second collection of the large intermediate frame merely for audit statistics.

9.7 Experiment: the offline lesson still runs the real data path

```

from assets.book_setup import run_offline

level_04 = run_offline(4)
meta = level_04.metadata
{
    "generated_agents": meta["generated_agents"],
    "agent_calls": meta["agent_calls"],
    "delegation": [call["name"] for call in meta["omnigent_tool_calls"]],
    "skill_digest": meta["skill_digest_sha256"][:12],
    "joined_rows": meta["joined_rows"],
    "evaluation_score": meta["evaluation"]["score"],
    "omnigent_connected": meta["omnigent_connected"],
    "mode": meta["mode"],
}

```

```

{'generated_agents': ['Polars Table Reader', 'Polars Revenue Calculator'],
 'agent_calls': ['read_tables', 'calculate_revenue'],
 'delegation': ['read_tables', 'calculate_revenue'],
 'skill_digest': '97494980cd1',
 'joined_rows': 16044,
 'evaluation_score': 1.0,
 'omnigent_connected': False,
 'mode': 'offline-simulated-omnigent-codex'}

```

offline=True simulates only **OmniGenT/Codex plan generation and delegation events**. It constructs the same two typed actions and two tool-call evidence records. Their executed_by field is "simulated", and their turns are marked simulated. The five-table read, Polars join, Top-N calculation, and SQL-baseline comparison still run against the real local data. Offline output therefore demonstrates reproducible data and control code, but it does not prove connectivity to the local OmniGenT server or Codex harness; codex_threads_started is 0.

9.8 Ten checks: the generator does not grade itself

The final evaluator is deterministic host code, not the model's self-assessment. It checks:

Check	Risk rejected
Mode-appropriate OmniGenT delegation evidence	Live server calls are missing/out of order, or offline simulated evidence has the wrong order
Exactly two workers	Hidden extra or missing roles
Skill provenance	Actions based on different skill versions
Least-privilege os_env declaration and approvalPolicy=never	Bundle configuration or Codex-executor constraints differ from the contract; on Windows this still does not prove hard FS/network isolation
Action order	Calculation before reading, or repeated calls
Five-table completeness	Continuing with missing or empty data
Top-N shape	Wrong row count or duplicate categories
Revenue domain and order	NaN, negative, or unsorted values
Join cardinality	Unexpected join explosion or row loss
SQL baseline parity	Polars output diverges from independent SQL

For the fixed dataset, the run must produce 16 categories from 16,044 joined records and agree with baseline/revenue.sql to the cent. Any failed check causes the harness to fail closed rather than return a plausible ranking.

9.9 Live mode: OmniGenT client connects to the Codex harness

The live path requires an explicit OMNIGENT_URL. The code first uses OMNIGENT_RUNNER_ID when supplied; otherwise omnigent-client resolves an online runner advertising the codex harness. Only then does it upload the reproducible tar.gz bundle and bind the new session to that runner. The bundle contains the root configuration, two child-agent configurations, three scoped copies of the Polars skill, and source digests. OmniGenT runs local ornth-1.5:9b through its Codex harness, while StreamHooks capture the two subagent tool calls actually emitted by the server. Only a Reader-then-Calculator sequence in which both calls were server-executed may proceed to host type validation, allowlisted execution, and the ten deterministic result checks.

```
$env:OMNIGENT_URL = "http://localhost:6767"  
# Optional: pin a runner when discovery is unavailable or ambiguous  
# $env:OMNIGENT_RUNNER_ID = "runner_..."  
kungfu-agent run 4 --question "top 5 film categories by revenue"
```

! No silent downgrade

Live mode fails explicitly when OMNIGENT_URL or an online/explicit Codex runner is absent. To run the reproducible lesson, select `offline=True`; the system never presents local simulation as a connected OmniGenT run.

9.10 Conclusion and promotion criterion

Level 4's new control is not merely "one more agent." It places role count, skill provenance, action set, order, parameters, and acceptance criteria inside one harness contract. The system can reject a bad plan or bad result, but it still runs only once: failure does not automatically become a repair action.

Move to Level 5 when evaluation feedback should enter the next plan and the system needs a retry budget, stopping conditions, and an auditable iterative trajectory.

9.11 End-of-chapter exercises

1. Why must an old plan's digest become invalid after any Polars reference file changes?
2. If OmniGenT returns a third action, should rejection happen before or after the database read?
3. If the SQL baseline and Polars plan share the same incorrect join assumption, can this evaluator detect the problem? What independent evidence is missing?
4. Why can `offline=True` validate the result without proving live Codex-harness connectivity?

10 Level 5: Make Failure the Input to the Next Iteration

10.1 Capability Acceptance Experiment

An early-calculation proposal is rejected, its feedback enters the retry, and execution completes sixteen inspections plus one calculation. Persistent errors exhaust a per-step retry budget. Rejections and accepted steps have separate audit records.

kungfu-agent experiment 5

One move, one lesson: **The value of a loop is not repetition, but evaluation, feedback, memory, and stopping conditions.**

10.2 The problem: how can an agent explore the full data space and stop at the goal?

Level 4 can constrain a fixed Reader → Calculator plan. Level 5 additionally controls the **cross-iteration trajectory**. This chapter gives the agent one explicit goal: inspect all 16 tables in the Sakila catalogue, use the discovered relationships to calculate category revenue, and stop immediately after successful verification. Evaluator output is not merely logged; the next iteration must prove that it consumed that state.

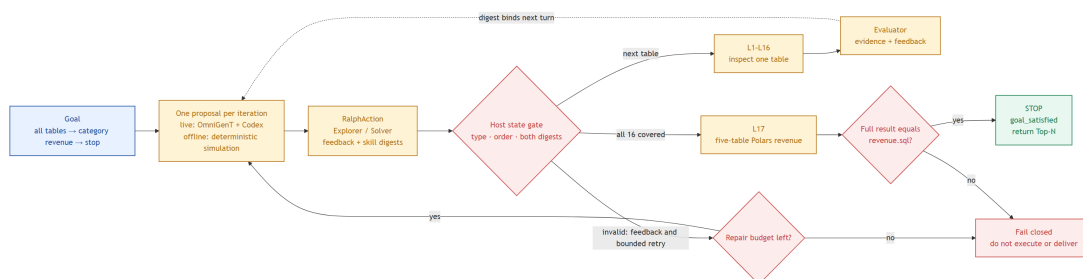


Figure 10.1: The Level 5 16+1 Ralph loop: table-by-table exploration, feedback binding, revenue verification, and immediate stopping

The executable state machine lives in `src/agent_kungfu/levels/level5/ralph_omnigent.py`. Every iteration stores a `RalphIterationRecord(iteration, action, evidence, evaluation,`

decision). A RalphAction can only be `inspect_table` or `calculate_revenue`: the model proposes, while only the host can read SQLite or execute Polars. The host canonically serializes the previous feedback as JSON and computes its SHA-256 digest, and it also obtains the loaded Polars skill digest. The next action must echo both digests exactly or be rejected before execution. This binds both loop state and capability version.

i The deterministic exit contract in this example

1. Freeze the name-sorted SQLite catalogue at startup; the current database must contain 16 unique tables.
2. Iterations 1–16 may only inspect the next unseen table. Here, `passed=false` means the global goal is incomplete, not that the table inspection failed.
3. Only after complete coverage may iteration 17 call `calculate_revenue(n)`.
4. Set `decision=stop` and `stop_reason=goal_satisfied` only when the complete 16-category Polars result equals the SQL baseline, the join contains 16,044 rows, and the Top-N shape is correct.
5. Do not run iteration 18 after success. The 17 iterations count accepted steps; each step has up to two additional proposal repairs. Invalid actions are rejected before execution and fed back; exhausted repair budgets, missing delegation evidence, or baseline mismatches raise errors, never returning a failed result.

A governable iteration loop requires a test, build, screenshot difference, or another verification signal so that a session can inspect and repair its work before a human sees it. When a production metric crosses its control band, the diagnosis should become the next intent.md and re-enter the lifecycle (Claxton 2026). Self-Refine shows iterative revision from explicit feedback by the same model, while Reflexion stores linguistic feedback as episodic memory for later trials (Madaan et al. 2023; Shinn et al. 2023). These studies demonstrate useful feedback mechanisms on their evaluated tasks; they do not establish that model self-critique equals external correctness. Level 5 therefore prefers database execution and deterministic rules as its gate.

10.3 From Item Repair to PRD and SDLC Loops

“Give an agent a feedback loop” cannot mean only “try again after failure.” A production generalization of this chapter’s narrow state machine can distinguish three timescales: the **item loop** turns verification failure into repair input within a session; the **PRD aggregation loop** declares Done only after every requirement passes separately; and the **SDLC operational loop** turns a post-release anomaly into new intent and priority. This repository implements only the first scale’s 16+1 Sakila instance; the other two are design extensions. Item acceptance does not complete the PRD, and PRD completion does not make production permanently safe.

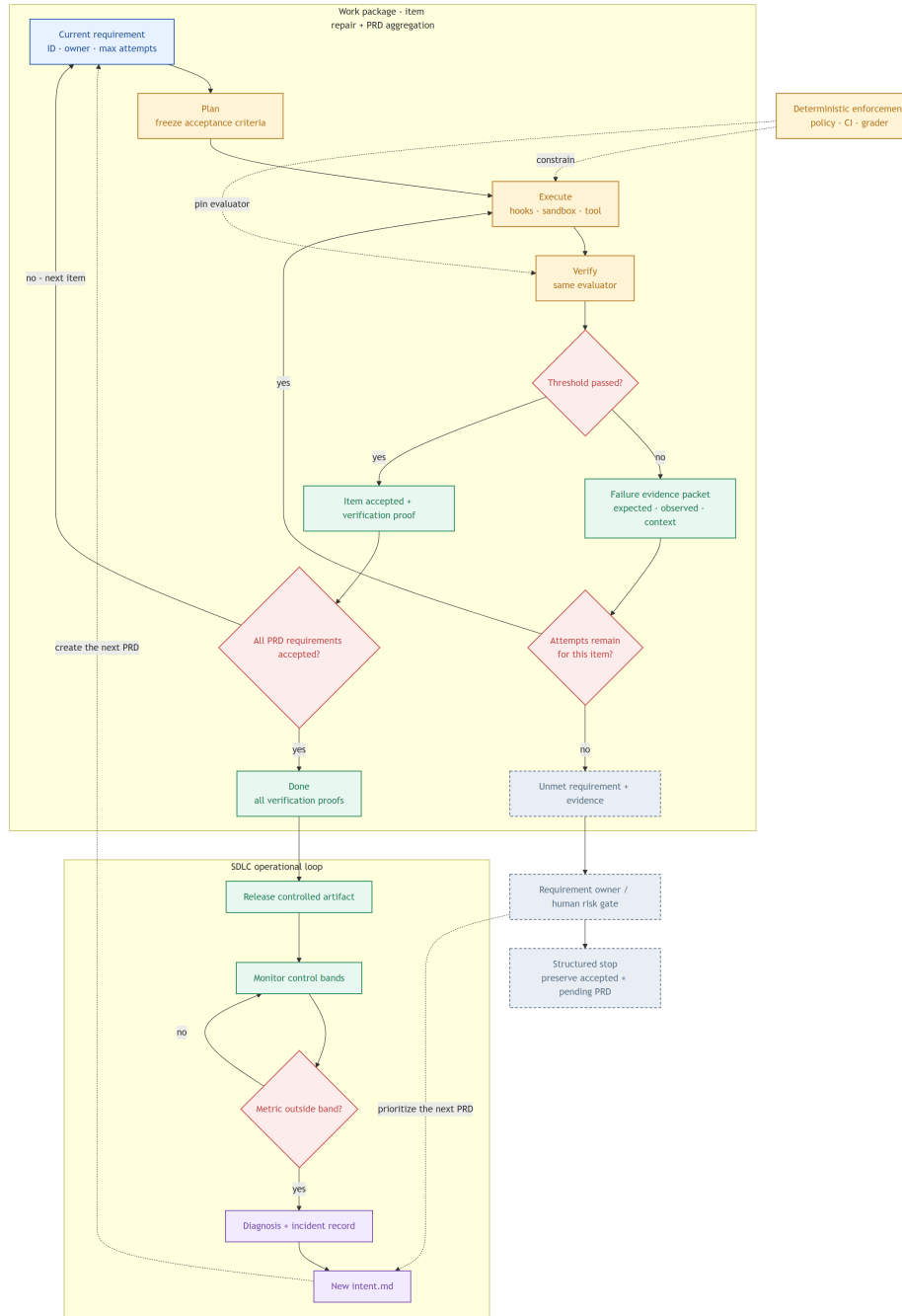


Figure 10.2: Three-scale feedback control: item evidence return, aggregate PRD completion, and SDLC operational write-back

A failure becomes consumable by the next iteration only after it is structured. A minimal failure evidence packet should retain `requirement_id`, owner, frozen acceptance criteria, current attempt and `max_attempts`, artifact and environment versions, expected and observed results, evaluator and evaluator version, failure class, retryability, accepted and pending requirements, and final disposition. SWE-bench combines real issues, repository states, and executable tests, illustrating why “task + environment + oracle” is closer to reproducible engineering feedback than a free-form criticism (Jimenez et al. 2024). Anthropic’s agent-evaluation guidance further separates a task, trial, transcript, outcome, and grader; this chapter therefore treats feedback

as an auditable record rather than a string casually appended to the next prompt (Anthropic 2026a).

The following table organizes six testing practices and the maintenance write-back mechanism as Level 5 control contracts with observable acceptance evidence:

Table 10.1: Control contracts for feedback loops

Loop practice	Control contract	Evidence that must persist	Current Sakila demo
Wrap checks in one target whose failure returns a non-zero code	A validator must be machine-callable and unambiguous	Command, exit code, logs, environment version	Any failure among ten deterministic harness checks raises an error
Provide healthy-output examples and measurable goals	Define the oracle and threshold before allowing action	Expected pattern, threshold, grader version	Fixes 16 tables, a 17-iteration budget, 16,044 joined rows, and the SQL baseline
Write a failing test first and do not edit it while fixing the bug	Freeze acceptance criteria so the agent cannot pass by changing the question	The test fails for the expected reason, then the same test passes	Configuration, typed contracts, and <code>revenue.sql</code> remain outside the loop
Close UI work with browser or screenshot checks	The evaluator must observe the modality actually delivered	Baseline, difference, tolerance, and human exception	Outside the read-only Sakila SQL scope
Run verification and show its output before reporting completion	Item progress only advances the trajectory; Done requires proof of the complete goal	Action, evidence, evaluation, decision, final checks, and stop reason	Every round stores that four-part record plus complete coverage and baseline evidence; there is no general PRD aggregator yet
Diagnose a production control-band breach and generate new intent	The outer loop writes operational failure back into requirements instead of silently patching production	Metric window, incident, <code>intent.md</code> , approver	Production monitoring and intent write-back are not implemented

10.4 Experiment: complete “explore—calculate—stop” in 17 iterations

The offline track does not ask a model to guess a SQL statement. It freezes the real `sqlite_master` catalogue and walks alphabetically from actor through store. Each `inspect_table` reads only a one-row sample boundary, the real `COUNT(*)`, and columns that might signal the revenue join. After all 16 tables have evidence, iteration 17 may execute `calculate_revenue`.

```
import polars as pl
from assets.book_setup import run_offline

level_05 = run_offline(5)
```

```

iteration_rows = []
for item in level_05.metadata["iterations"]:
    iteration_rows.append(
        {
            "iteration": item["iteration"],
            "action": item["action"]["runtime_tool"].split("_")[0],
            "table": item["action"].get("table") or "—",
            "score": item["evaluation"]["score"],
            "decision": item["decision"],
            "feedback": "; ".join(item["evaluation"]["feedback"][:28]),
        }
    )
pl.DataFrame(iteration_rows)

```

Table 10.2: Sixteen table inspections and the seventeenth revenue-verification iteration

iteration	action	table	score	decision	feedback
i64	str	str	f64	str	str
1	"inspect"	"actor"	0.0588	"continue"	"Coverage incomplete: inspect"
2	"inspect"	"address"	0.1176	"continue"	"Coverage incomplete: inspect"
3	"inspect"	"category"	0.1765	"continue"	"Coverage incomplete: inspect"
4	"inspect"	"city"	0.2353	"continue"	"Coverage incomplete: inspect"
5	"inspect"	"country"	0.2941	"continue"	"Coverage incomplete: inspect"
...	...	...	...	...	...
13	"inspect"	"payment"	0.7647	"continue"	"Coverage incomplete: inspect"
14	"inspect"	"rental"	0.8235	"continue"	"Coverage incomplete: inspect"
15	"inspect"	"staff"	0.8824	"continue"	"Coverage incomplete: inspect"
16	"inspect"	"store"	0.9412	"continue"	"All tables inspected; calcul"
17	"calculate"	"—"	1.0	"stop"	"Goal satisfied: every table "

With the default Top-5, Sports ranks first at 5314.21. The evaluator does not substitute that first row for a correctness proof: it compares the complete 16-category Polars result row by row with the independent baseline/revenue.sql result and also checks the 16,044-row join. The first 16 scores express coverage progress; the final pass stops immediately. Without an evaluator, “try again” is merely random resampling. Without a stopping condition, the system may keep consuming resources after success—or even replace a correct answer with a worse one.

10.5 How the live loop works

The live path requires `OMNIGENT_URL` and prefers `OMNIGENT_RUNNER_ID`; without an explicit ID, it resolves only an online Codex runner. `omnigent-client` uploads a reproducible bundle containing the root, Explorer, Revenue Solver, and Polars skill, binds the session, and

calls `chat.query()` per proposal. StreamHooks must observe exactly one expected tool call with `executed_by="server"`. Returned text must pass strict RalphAction parsing, current table/action validation, feedback-digest validation, and skill-digest validation before host execution. Invalid proposals become failure feedback, with up to two repairs per step. Rejections remain in repairs, separate from the 17 accepted steps. Service authority and delegation-evidence failures terminate rather than being bypassed by retries.

The offline track replaces only **proposal and delegation** with deterministic simulation and marks every tool-call record `executed_by="simulated"`. It starts no Codex turn and cannot prove that a local OmniGenT server, runner, or Ollama endpoint is connected. The table catalogue, per-table SQLite observations, five-table Polars calculation, and SQL-baseline comparison still run for real. A live environment without a URL or online runner fails explicitly and never falls back silently to an offline result.

`loops/sakila-improvement/RALPH.md` is the portable human-readable specification; the executable state machine lives in `src/agent_kungfu/levels/level5/ralph_omnigent.py`. This example implements one narrow 16+1 exploration task and its stop contract. It is still not a general PRD aggregator and not an already continuous AI-native SDLC.

10.6 The evaluator determines the flywheel's direction

The final harness runs ten deterministic checks: the catalogue snapshot, complete canonical order, typed action trajectory, feedback- and skill-digest binding, mode-appropriate OmniGenT/Codex call evidence, the two workers' read-only declarations, per-table observations, revenue shape, SQL-baseline parity, and bounded immediate stopping. This is suitable as a teaching gate, but it cannot verify whether:

- a join introduces duplicate counting;
- metric definitions match the business specification;
- the result is genuinely useful for the user's decision;
- the answer turns an observational difference into a causal claim.

Consequently, "the Ralph loop passed" means only that it passed this evaluator, not that it achieved universal correctness.

! Production warning

Invalid actions are rejected before execution and receive bounded feedback retries. Inadequate budgets, exhausted retries, invalid delegation evidence, or failure of any of the ten final checks raise an exception without constructing a `LevelResult`. A production PRD extension should additionally persist and escalate a stop record containing status, `unmet_requirement`, `owner`, `attempts`, `max_attempts`, `evidence`, `accepted_requirements`, and `pending_requirements`.

10.7 Conclusion and promotion criterion

Level 5 is appropriate for tasks that machines can evaluate and repair automatically within a bounded number of attempts. The most important engineering assets are often not prompts, but evaluators, failure taxonomies, budgets, and stopping rules.

Move to Level 6 when a repair requires understanding relationships among real tables, and when policy, row limits, and protocols must become typed constraints.

10.8 End-of-chapter exercises

1. Make iteration 5 inspect actor again. Explain why rejection must occur before another SQLite query.
2. Why must the budget be at least `table_count + 1`, and why must the system refuse to run iteration 18 after iteration 17 passes?
3. Design an exhaustion stop record. Explain which catalogue, feedback-digest, delegation, and baseline evidence should reach the owner.

Part III

Part III — From Data Contracts to Knowledge Production

11 Level 6: Encode Rules and Relationships in Types and Graphs

11.1 Capability Acceptance Experiment

Foreign-key edges determine the JoinPlan; a task DAG schedules reads, joins and aggregation. Removing `film_category→category` must block execution; the intact graph's Polars revenue must match the SQL baseline.

kungfu-agent experiment 6

One move, one lesson: **The harness schema governs rules, the schema graph governs relationships, and the evaluator governs minimum quality.**

11.2 The problem: feedback knows the result is poor, but not how tables connect

The Level 5 evaluator can report that “decision metrics are missing.” Yet if the SQL planner does not know how payment connects to film, it can only keep searching the schema text. At the same time, constraints such as `max_rows=50`, read-only mode, and a three-iteration limit are difficult to audit when they remain scattered parameters.

Level 6 upgrades two control planes at once:

1. It uses the Pydantic `HarnessManifest` to type protocols, row limits, iteration limits, and evidence requirements.
2. It builds a NetworkX directed graph from the foreign keys in the actual DDL to recover join paths.

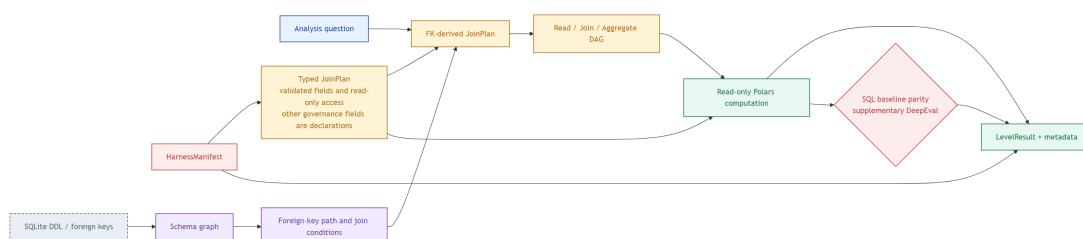


Figure 11.1: Level 6 contract, relationship-evidence, and evaluation control surfaces

Pydantic validates field constraints at object boundaries (Pydantic, n.d.), while NetworkX provides graph algorithms and structural queries (NetworkX Developers, n.d.).

11.3 From a Skill Catalogue to a Feature Graph

Once a task moves from “invoke one capability” to “deliver a business outcome,” one Skill is quickly insufficient; Skills, MCP, hooks, subagents, Agent Teams, and Plugins must cooperate around one feature contract. The relevant questions concern dependencies, types, permissions, and acceptance—not whether component counts or market estimates imply capability.

A function is not a longer Skill. A Skill is local, reusable procedural knowledge; a function is a promise about a user-visible outcome. A complex function must organize several skills as a dependency graph and specify:

1. the type consumed and produced by every skill;
2. which edges carry data and which impose control;
3. which nodes can run in parallel and which must wait for acceptance;
4. the permissions, budget, and side-effect boundary of each node;
5. how a local failure is rolled back, substituted, or escalated; and
6. the evidence that accepts the feature as a whole.

HuggingGPT decomposes a complex request, selects models by their function descriptions, executes subtasks, and summarizes the results. MetaGPT organizes multi-agent intermediate artifacts through roles and standard operating procedures. AutoGen lets developers define interaction patterns among agents, tools, and human participants in code and natural language (Shen et al. 2023; Hong et al. 2023; Wu et al. 2023). These studies support the mechanism that complex features need explicit orchestration; they do not imply that adding more skills automatically improves reliability. Errors can propagate along dependency edges, and incompatible handoff formats can make the overall feature fail even when each local skill reports success.

The following figure uses this book’s existing Pydantic types, Sakila DDL graph, SQL guard, Polars result, and evaluator to show the skill collaboration required by a “store inventory improvement recommendation” feature:

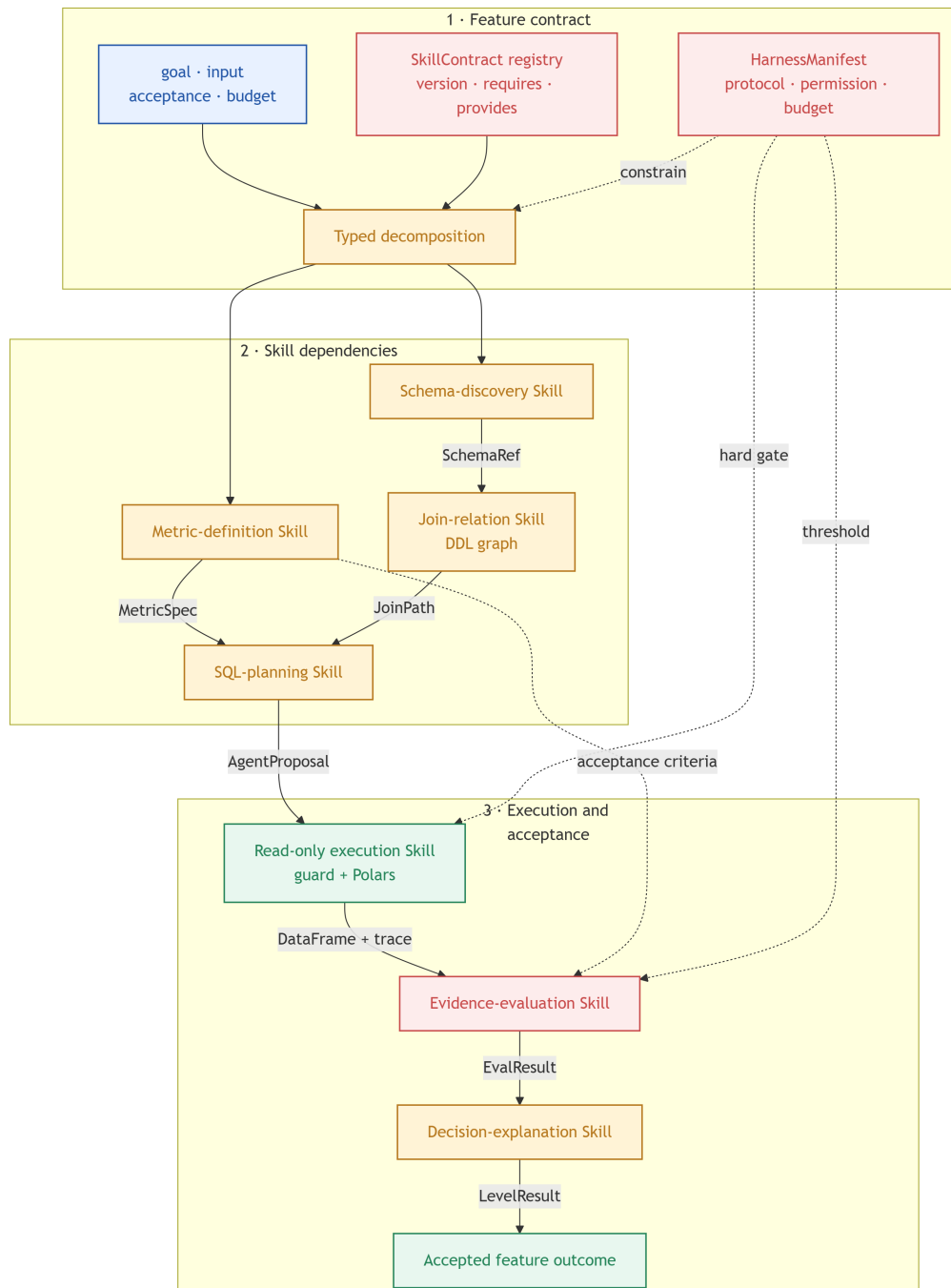


Figure 11.2: A complex feature dependency graph: local capabilities compose into a business outcome through typed edges and one acceptance gate

Granularity, packaging, and governance must be modeled separately. The following matrix defines five engineering objects so that a feature, plugin, and marketplace are not treated as synonyms:

Table 11.1: Skill–feature–distribution object matrix

Engineering object	Reconstructed definition in this book	Contract that must be machine-readable	Current Level 6 status
Skill	Reusable procedural knowledge for one local step	Input, output, version, permissions, failure modes	Skills and functions exist, but there is no unified SkillContract registry
Tool / MCP	An invocation edge that lets a skill reach an external capability	Protocol, parameter schema, identity, timeout, side effects	allowed_protocols is metadata only, not a hard gate
Function	A user-outcome contract implemented as a skill dependency graph	DAG, typed edges, budget, rollback, whole-feature acceptance	This section defines the contract model; the runtime does not execute a feature graph
Plugin / package	A versioned carrier that distributes skills, hooks, agents, and configuration	Dependencies, compatible versions, signature, upgrade, withdrawal	This project has no plugin packaging or installation lifecycle
Marketplace	Discovery, admission, and retirement governance across packages	Owner, review, permissions, cost, rating, deprecation	Outside Level 6; a marketplace does not guarantee correct composition

A function is not a plugin. A function states what the system must accomplish and how it will be accepted; a plugin states how code and configuration are distributed. One plugin can expose several functions, while one function may depend on skills distributed by several plugins. Only a feature graph and its typed edges can answer whether those skills can be composed safely.

For the question “which stores should improve inventory for which films?”, a minimal collaboration contract is:

Table 11.2: Typed multi-skill collaboration contract for a complex Sakila feature

Skill	Consumes	Produces	Failure gate and owner
Metric definition	Question, business definition	MetricSpec	Escalate when the metric is not computable or not approved
Schema discovery	SQLite DDL	SchemaRef	Reject references to missing tables or columns
Join relationships	SchemaRef, DDL graph	JoinPath	Do not guess when no path or several ambiguous paths exist
SQL planning	MetricSpec, JoinPath	AgentProposal	Replan on type incompatibility or missing decision metrics
Read-only execution	AgentProposal, policy	DataFrame + trace	Fail immediately on writes, timeout, or row-limit breach

Skill	Consumes	Produces	Failure gate and owner
Evidence evaluation	Result, acceptance criteria	EvalResult	Enter the Level 5 loop or exhaustion policy on failure
Decision explanation	EvalResult, evidence rows	LevelResult	Do not turn observational differences into causal claims

The current code already has `AgentProposal`, `EvalResult`, `LevelResult`, `HarnessManifest`, and a schema graph, but it has neither a `SkillContract` type nor an executor for a skill-dependency DAG. The figure is therefore an **implementable design based on existing components**, not a claim about present runtime behavior. The Level 6 extension is conceptual but precise: relationships among data tables belong in the DDL graph, relationships among skills belong in a dependency graph, and both graphs need typed edges and gates rather than prompt-only conventions.

11.4 Experiment 1: a typed manifest

```
from pydantic import ValidationError
from agent_kungfu.contracts import HarnessManifest, Protocol

manifest = HarnessManifest(
    name="sakila-graph-harness",
    allowed_protocols=[Protocol.SKILL, Protocol.MCP, Protocol.A2A],
)
try:
    HarnessManifest(
        name="unbounded",
        allowed_protocols=[Protocol.SKILL],
        max_iterations=11,
    )
except ValidationError as error:
    invalid_manifest_error = error.errors()[0]["msg"]

{
    "valid": manifest.model_dump(mode="json"),
    "invalid_max_iterations": invalid_manifest_error,
}
```

```
{'valid': {'name': 'sakila-graph-harness',
```

```

'version': '1',
'allowed_protocols': ['skill', 'mcp', 'a2a'],
'read_only': True,
'max_rows': 50,
'max_iterations': 3,
'conflict_policy': 'safe-executable-highest-confidence',
'evidence_required': True},
'invalid_max_iterations': 'Input should be less than or equal to 10'}

```

Typed constraints move the error from “discovered during the eleventh iteration” to configuration construction time. However, the current `allowed_protocols` value appears only in metadata; it is not yet connected to the `ProtocolBus` for runtime enforcement. It is an explicit contract, not a finished policy engine.

11.5 Experiment 2: recover relationships from DDL

```

from assets.book_setup import DB, run_offline
from agent_kungfu.graph_engineering import (
    build_schema_graph,
    graph_summary,
    shortest_join_path,
)

schema_graph = build_schema_graph(DB)
level_06 = run_offline(6)
{
    "summary": graph_summary(schema_graph),
    "payment_to_film": shortest_join_path(schema_graph, "payment", "film"),
    "evaluation": level_06.metadata["evaluation"],
}

```

```

{'summary': {'tables': 16,
'relationships': 21,
'components': 2,
'most_connected': [('store', 5),
('staff', 5),
('address', 4),
('customer', 4),
('film', 4)],

```

```

'cycles': [['staff', 'store']],
'payment_to_film': ['payment', 'rental', 'inventory', 'film'],
'evaluation': {'passed': True,
'score': 1.0,
'evaluator': 'graph-baseline-parity',
'feedback': []}]

```

The current database produces 16 nodes, 21 foreign-key edges, and two connected components. `payment → rental → inventory → film` is the shortest path after the graph is converted to an undirected graph. Direction is ignored because a SQL join can be expressed in either direction along a foreign-key relationship.

The current `level6/graph_agent.py` derives a `JoinPlan` from foreign-key paths and executes Polars joins and aggregation through a task DAG. Removing a required edge blocks planning. Results must match the independent SQL baseline because a shortest path alone does not establish the correct business definition.

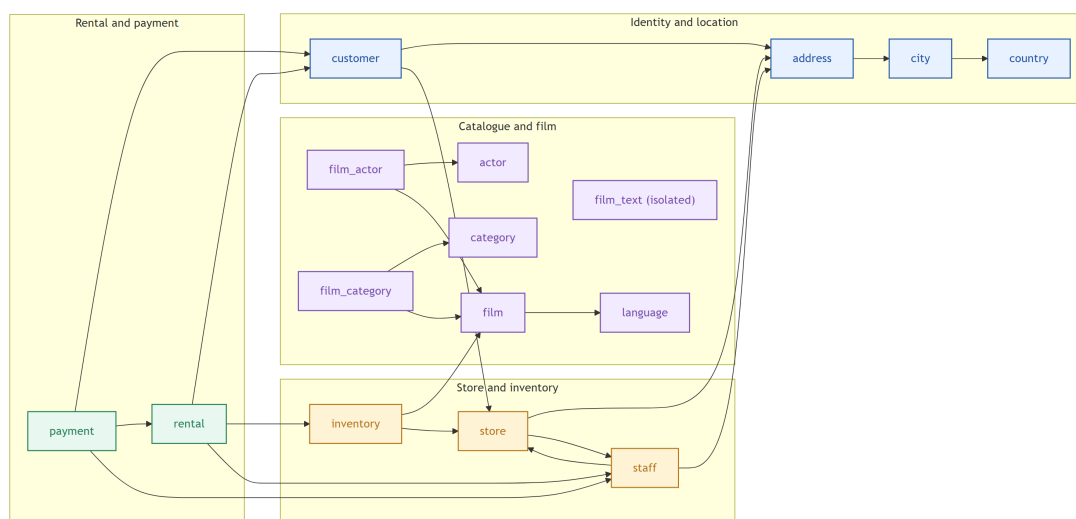


Figure 11.3: The Sakila schema graph extracted from SQLite DDL

11.6 Graphs, lineage, and ontologies are different

The graph at this level parses only explicit `FOREIGN KEY ... REFERENCES ...` clauses in the DDL that match a regular expression. It is not:

- SQL query lineage;
- a business concept graph;
- an ontology with inference rules;
- an optimizer that automatically proves a join is correct.

The graph can answer “which tables have declared relationships?” and “what is the shortest connection path?” It cannot answer “does this path match the business definition for the current question?”

11.7 Deterministic gates and DeepEval

Level 6 compares the complete revenue result with the SQL baseline by default. When `offline=False` and `use_deepeval=True`, it additionally runs GEval through a local DeepEvalBaseLLM using `ornith-1.5:9b`. Its separate `deepeval` field does not replace numeric acceptance (Confident AI, n.d.).

A deterministic gate is cheap, stable, and easy to diagnose. An LLM judge can assess natural-language relevance and causal phrasing, but it is slower, can drift, and may share biases with the model being evaluated. Production systems generally need a combination of both rather than choosing only one.

11.8 Conclusion and promotion criterion

Level 6 turns relationships into an executable join plan and organizes computation through task dependencies. This experiment covers category revenue; broader metrics and multi-agent task graphs still require their own design and validation.

The graph is still rebuilt from the schema on every run, and the system retains no explanations, metric definitions, or historical observations. Move to Level 7 when knowledge must be reused across runs.

11.9 End-of-chapter exercises

1. Why is the shortest join path not necessarily the correct business join path?
2. How would you turn `allowed_protocols` from metadata into a runtime hard gate?
3. What kinds of errors are DeepEval and a deterministic evaluator each better suited to detect?

12 Level 7: Let the System Remember Yesterday

12.1 Capability Acceptance Experiment

The second run retrieves verified query knowledge for the same task. Modifying an isolated database copy changes its fingerprint, triggers revalidation, and preserves a new knowledge version. Retrieval is scoped to the same task.

```
kungfu-agent experiment 7
```

One move, one lesson: **Turn volatile runtime context into inspectable, versionable Markdown knowledge snapshots.**

12.2 The problem: the graph can be rebuilt, but every explanation must be rebuilt too

Level 6 can reconstruct the schema graph from DDL, but it has no memory across runs. The model must receive the schema, table sizes, and relationship summary again each time. The team also has no way to diff, review, or cite the knowledge state from a particular run.

Level 7's `generate_wiki()` writes structured facts to ordinary Markdown:

- `README.md`: a table index and row counts;
- one page per table: row count, neighboring tables, and DDL;
- 17 foundational pages: one index plus 16 table pages;
- each verified task version adds experience - *.md and JSON; the live model reads retrieved experience to explain evidence.

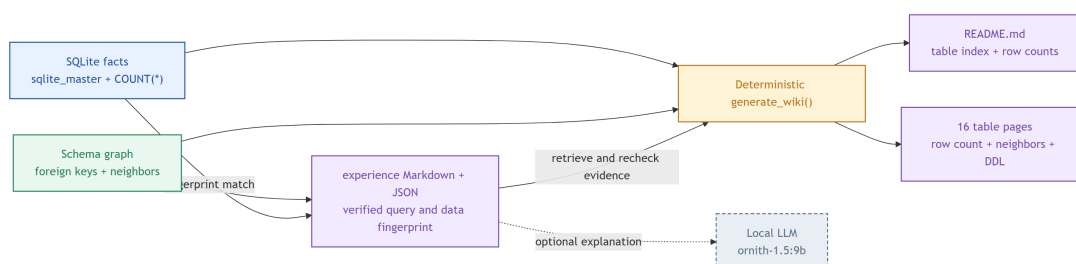


Figure 12.1: From database facts to a persistent wiki and an optional LLM overview

12.3 Three Open-Source Architecture Paths for Knowledge Production

This chapter samples OpenKB, SAG, axiom-wiki, deepwiki-open, Graphiti, GraWiki, Knowledge Catalog, llm-wiki-agent, llmwiki, obsidian-wiki, and Synthadoc. They do not share one reproduction, dataset, or cost definition, so they can reveal **architectural patterns**, not a performance ranking; project-reported numbers are not treated as independent effectiveness evidence.

The implementations can be organized into three paths. The first compiles documents or code into interlinked pages at ingestion time. The second writes events, entities, memories, and temporal relationships into graph structures. The third raises knowledge production to enterprise catalogues, governance, and federated data sources. Their common feature is not the storage medium; it is the promotion of knowledge from a by-product of one answer to a system asset with a maintenance lifecycle.

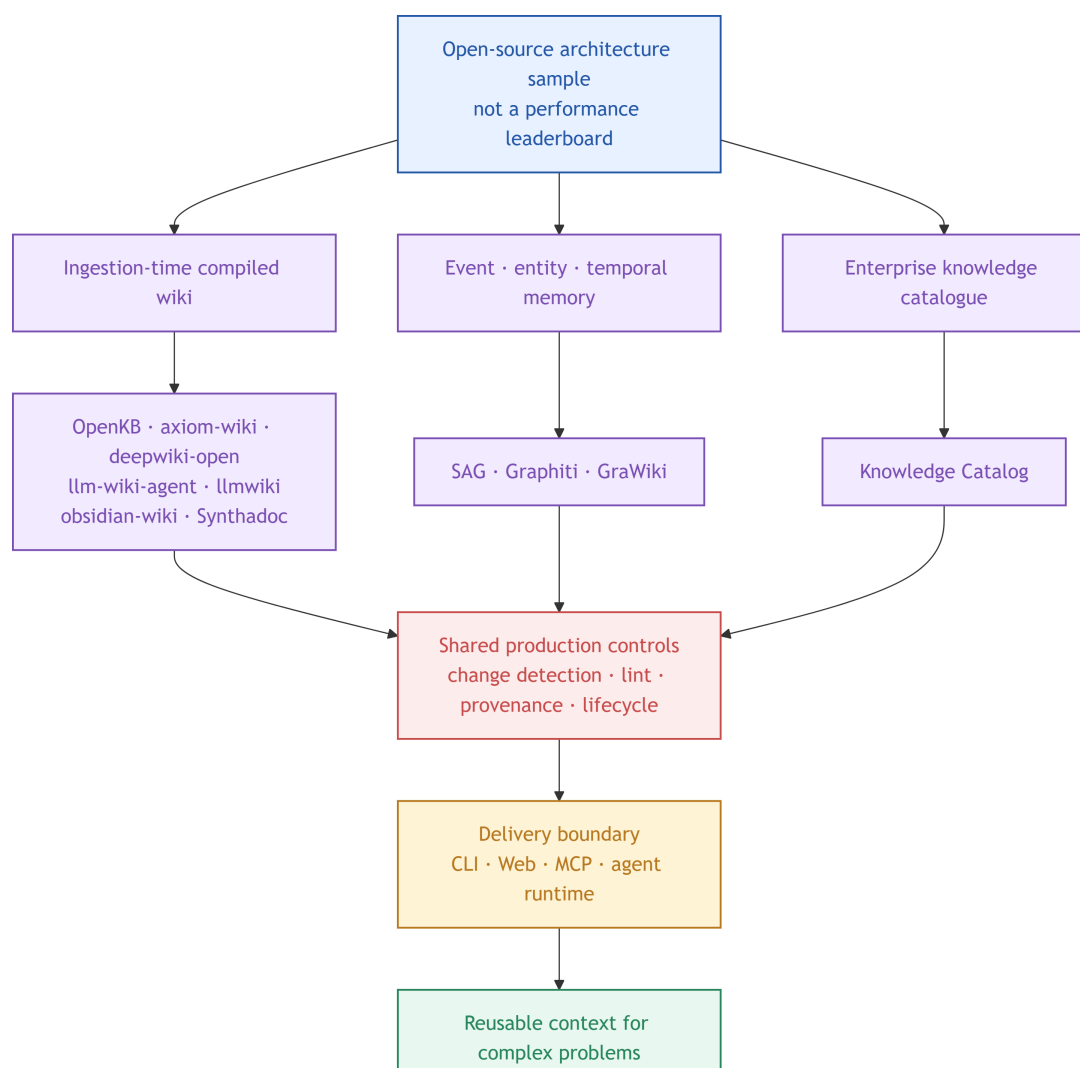


Figure 12.2: Three knowledge-production paths found across open-source LLM-Wiki implementations

The following matrix encodes positioning, retrieval structure, storage, interfaces, and maintenance controls around the Level 7 questions of how knowledge is produced, maintained, and delivered:

Table 12.1: Open-source project knowledge-production matrix

Project	Knowledge-production unit	Primary maintenance control	Delivery boundary	Architecture path
OpenKB	Hierarchical long-document index and interlinked pages	File watching and Wiki Foundation	CLI and Skill export	Compiled wiki
SAG	Event-entity retrieval unit	Retrieval traces and two retrieval modes	Web and MCP	Lightweight relational retrieval
axiom-wiki	Markdown pages and a hybrid index	Hash-based incremental and two-tier deterministic/agent maintenance	CLI, REPL, and MCP	Compiled wiki
deepwiki-open	Repository explanations and architecture pages	Repository analysis and multi-round research	Web and question answering	Code wiki
Graphiti	Entity relationships with validity time	Incremental graph construction and episode provenance	Python, MCP, and REST	Temporal memory graph
GraWiki	Graph entities and agent-memory nodes	Deduplication and backend abstraction	Python facade	Graph memory
Knowledge Catalog	Enterprise data semantics and metadata	Governance, policy, and lineage	Agent, MCP, and tools	Enterprise catalogue
llm-wiki-agent	Pages with front matter	Deterministic health checks separated from semantic lint	Host coding agent	Agent-as-Runtime
llmwiki	VaultFS pages and annotations	Scheduled maintenance and protected pages	Web, browser, and MCP	Productized wiki
obsidian-wiki	Vault pages, links, and manifest	Dream Cycle, dry-run, and staged writes	Multiple coding agents	Agent-as-Runtime
Synthadoc	Citation-bearing pages compiled at ingestion	Multi-agent orchestration, cost guards, and lifecycle	CLI, Obsidian, Web, and MCP	Trusted compiled wiki

This table is an architecture map, not a reproduction in one shared environment. Project versions, interface coverage, and product status can change; fields not reproduced under common conditions provide no evidence that one implementation is “more advanced” than another.

12.4 Why Knowledge-Driven Production Is Entering Complex Problem Solving

Knowledge production is not the generation of more text. It turns sources into reusable facts, links, summaries, conflict records, and versions, then allows later agents to select them for a problem. Nor is this a simple replacement for RAG. RAG separates parametric memory from external non-parametric memory and retrieves evidence at query time (Lewis et al. 2020). The LLM-Wiki path moves some structuring, consolidation, and checking work earlier, into ingestion and maintenance.

The research trajectory shows three successive changes. Generative Agents connects experience records, retrieval, reflection, and planning, demonstrating that behavior across time requires accumulated memory (Park et al. 2023). MemGPT uses tiered memory and context swapping to manage limited windows (Packer et al. 2023). GraphRAG builds an entity graph and community summaries before a query to support global questions over a corpus (Edge et al. 2024). Zep/Graphiti then incorporates validity time and historical relationships into the knowledge graph, making it possible to represent that something was true yesterday but is invalid today (Rasmussen et al. 2025).

This evidence supports a limited but important trend: the more a problem requires cross-document synthesis, cross-session continuity, global theme discovery, and change tracking, the less a system can rely on assembling all context inside the final prompt. It needs a pipeline that continuously produces knowledge assets and exchanges ingestion cost for reuse at query time. This does not mean that every problem needs a wiki. Short questions, low-reuse information, and rapidly changing sources that cannot be validated may still be better served by retrieval at query time.

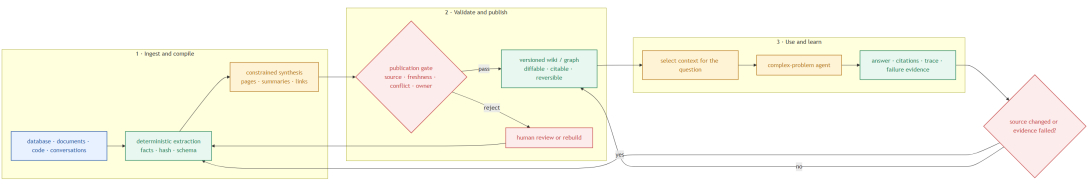


Figure 12.3: The knowledge production–use–feedback loop for complex problem solving

Table 12.2: Control objects for a knowledge-driven production loop and the Level 7 implementation boundary

Production stage	Required engineering object	Minimum publication gate	Current Level 7 status
Source registration	Source URI, time, and content hash	Source is locatable and readable	Only a local SQLite path
Fact compilation	Tables, row counts, DDL, and relationships	Recomputable from the database	Deterministically generated

Production stage	Required engineering object	Minimum publication gate	Current Level 7 status
Semantic synthesis	Page summary, business interpretation, and cross-page links	Cites facts and marks the interpretation layer	Overview only in live mode
Relationship organization	Page links or knowledge-graph edges	Traceable edges and no dangling targets	Schema adjacency only
Quality validation	Lint, conflicts, freshness, and owner	Failure cannot publish as trusted knowledge	Not implemented
Version publication	Snapshot ID, diff, and rollback	Validation passed and prior version retained	Same-name files are overwritten
Problem use and feedback	Context pack, citations, trace, and failure evidence	Select relevant pages only; failure can trigger rebuild	Retrieves experience by exact task and data fingerprint

The figure and table describe an engineering target that can evolve from the existing `generate_wiki()`. The current runtime has no retriever, version store, conflict checker, or feedback trigger. This is **an implementable upgrade target, not a complete loop already present in the current runtime.**

12.5 Experiment: generate a knowledge snapshot from the database

```
from pathlib import Path
from assets.book_setup import run_offline

level_07 = run_offline(7)
wiki_paths = [Path(path) for path in level_07.artifacts]
{
    "wiki_pages": level_07.metadata["wiki_pages"],
    "all_exist": all(path.is_file() for path in wiki_paths),
    "index": str(wiki_paths[0]),
    "evaluation": level_07.metadata["evaluation"],
}
```

```
{'wiki_pages': 18,
 'all_exist': True,
 'index': 'D:\\Python\\agent_kungfu_levels\\outputs\\book\\level07_wiki\\README.md',
 'evaluation': {'passed': True,
 'score': 1,
 'evaluator': 'retrieved-query-baseline-parity'}}
```

```

index_lines = wiki_paths[0].read_text(encoding="utf-8").splitlines()
index_excerpt = "\n".join(index_lines[:10])
customer_page = next(path for path in wiki_paths if path.name == "customer.md")
customer_lines = customer_page.read_text(encoding="utf-8").splitlines()
customer_excerpt = "\n".join(customer_lines[:12])
print(index_excerpt)
print("\n--- customer.md ---\n")
print(customer_excerpt)

```

Sakila Knowledge Wiki

Generated from the live SQLite schema.

- [actor](actor.md) — 200 rows
- [address](address.md) — 603 rows
- [category](category.md) — 16 rows
- [city](city.md) — 600 rows
- [country](country.md) — 109 rows
- [customer](customer.md) — 599 rows

--- customer.md ---

customer

Rows: 599

Related tables: address, payment, rental, store

Schema

```

```sql
CREATE TABLE customer (
 customer_id INT NOT NULL,
 store_id INT NOT NULL,

```

These texts are not cached model answers. They are generated deterministically from the current `sqlite_master`, the foreign-key graph, and live `COUNT(*)` queries. The 599 rows reported in `customer.md` can be checked against the database, and the table relationships can be checked against the DDL.

## 12.6 Why Markdown is a valuable intermediate layer

Markdown is not magical. Its value comes from serving three kinds of readers at once:

1. People can read and discuss it during code review.
2. Version control can reveal schema and row-count changes.
3. An LLM can receive the index and relevant pages as explicit context.

Compared with placing the entire schema in every prompt, a wiki gives knowledge filenames, update times, and review boundaries. It also turns “model context” into an engineering asset that can be built offline.

## 12.7 “LLM Wiki” needs a precise explanation

### Implementation facts

The foundational 17 pages are generated deterministically. Experience records additionally store the task, database fingerprint, generator version, verified SQL and knowledge notes. Live mode reads the retrieved experience Markdown; offline mode performs the same retrieval and numeric validation.

On a miss, Level 6 derives a graph-based plan and the verified query is persisted. On a hit, the query is reused, re-executed and checked against the baseline. A changed task or data fingerprint triggers learning again.

## 12.8 New risks introduced by accumulated knowledge

Persistence also introduces knowledge drift:

- Foundational schema pages are refreshed, while experience versions are retained by task and database fingerprint.
- Retrieval currently covers exact tasks, not semantic similarity or cross-task transfer.
- New data invalidates old experience; human review and version merging remain additional work.
- An LLM overview is interpretive content and must not receive the same trust as deterministic facts.

A production system should add provenance, freshness, owner, and validation status to every page, and compare schema fingerprints before publication.

## 12.9 Conclusion and promotion criterion

Level 7 is appropriate for systems that must reuse schema explanations, business definitions, and analytical context across runs. Knowledge becomes a persistent asset independent of the prompt for the first time.

Markdown pages, however, primarily describe “what is in the database.” They cannot rigorously express classes, properties, domain/range declarations, and mapping constraints. Move to Level 8 when knowledge requires an exchangeable semantic structure.

### 12.10 End-of-chapter exercises

1. Design the minimum provenance front matter for a wiki page.
2. How would you detect that `customer.md` has fallen behind the database?
3. Which content should be generated deterministically, and which is suitable for an LLM draft followed by human review?

## 13 Level 8: From Recording Knowledge to Producing Knowledge

### 13.1 Capability Acceptance Experiment

Relation candidates must pass OWL class/domain/range checks, foreign-key mapping and database-count verification before semantic publication. An invalid range blocks publication. This is not full OWL reasoning or OBDA execution.

kungfu-agent experiment 8

One move, one lesson: **A wiki is easy to read; an ontology makes classes, properties, and mappings machine-exchangeable.**

### 13.2 The Open-Source Implementation Landscape of Ontology Engineering

The audited open-source implementations span generation, learning, alignment, collaborative requirements, information extraction, and review work. The audit checked eighteen GitHub repositories and treats the main OntoGenix application and its experiment repository as one project family. On August 31, 2026, all addresses were accessible. Accessibility and recent updates establish inspectability, not production maturity or performance leadership.

The following table compares the implementations through three questions: Which semantic artifact does each produce? Which stage of knowledge production does it change? Where does its evidence stop? The associated papers support several representative conclusions. LLMs4OL separates learning into term typing, taxonomy discovery, and non-taxonomic relation extraction, while its original paper also warns that foundation models are not yet sufficient to perform ontology construction requiring substantial reasoning and domain expertise on their own (Babaei Giglou et al. 2023). OntoGPT/SPIRES constrains zero-shot extraction with a predefined schema and external vocabularies (Caufield et al. 2024). DeepOnto and OLLM represent neural-symbolic tooling and whole-subgraph learning, respectively (He et al. 2023; Lo et al. 2024). OntoAligner packages lightweight matching, retrieval, RAG, and LLM alignment as replaceable modules (Babaei Giglou et al. 2025).

Table 13.1: Open-source ontology projects by semantic artifact, production stage, and evidence boundary

Project or project family	Rewritten primary task	Contribution to knowledge production	Evidence and applicability boundary
<a href="#">Ontologies_by_LLMs</a>	Compare one-shot, decomposed prompting, and X-HCOME human collaboration	Return expert feedback to the conceptualization loop	A clinical case study; not a universal automation ranking
<a href="#">LLM-Automatic-Ontology-Generation</a>	Classify broader, narrower, same-as, and other relations against a gold standard	Turn semantic resemblance into measurable relation decisions	A relation benchmark, not a complete ontology generator; model numbers depend on one experimental setting
<a href="#">OntoAligner</a>	Compose string, retrieval, LLM, RAG, few-shot, and KGE aligners	Produce replaceable, reproducible mapping candidates across heterogeneous concepts	A modular toolkit; proposed correspondences still require domain validation
<a href="#">OntoGPT / SPIRES</a>	Recursively extract against LinkML/schema and ground to external vocabularies	Turn passages into structured assertions with identifiers	Grounding reduces identifier ambiguity; it does not prove that the source, relation, or causal account is true
<a href="#">automatic-KG-creation-with-LLM</a>	Use competency questions to drive paper-to-KG/ontology construction and query-back checks	Make intended questions into design and acceptance conditions	Answering the CQs demonstrates task utility, not logical completeness or domain approval
<a href="#">Agent-OM / ontology-llm</a>	Divide work between a retrieval agent and a matching agent	Separate large-scale candidate recall from precise semantic judgment	Agent structure improves workflow boundaries; both retrieval and LLM judgment can omit or invent matches
<a href="#">Onto-Generation</a>	Generate local OWL for individual CQs and merge fragments	Use local context to reduce interference from a monolithic long prompt	A research generation strategy; the merged result still needs consistency and redundancy checks
<a href="#">OntoGenix + Experiments</a>	Guide CSV-to-OWL/RML work in a GUI and compare models and quality measures	Let non-experts revise entities, relations, and mappings step by step	A GUI lowers the barrier but does not replace an ontology engineer; main and experiment repositories count as one family
<a href="#">OntoLearner</a>	Standardize term typing, taxonomy, and relation learning behind one API	Turn scattered scripts into reproducible tasks and data interfaces	Broad dataset coverage does not prove cross-domain correctness; benchmark labels still govern results
<a href="#">OntoChat</a>	Elicit requirements through dialogue and competency questions	Reposition the LLM as an interviewer and modeling adviser	A dialogue produces requirement candidates, not approved axioms

Project or project family	Rewritten primary task	Contribution to knowledge production	Evidence and applicability boundary
LLMs4OL	Define term typing, hierarchy discovery, and non-taxonomic relation tasks	Supply shared task language and an evaluation entry point for ontology learning	The original study does not support the claim that a base LLM can independently build an ontology
DeepOnto	Combine OWLAPI reasoning, normalization, projection, and deep models	Connect formal semantics with learnable representations	A broad engineering package, not a prompt-only system; logical and statistical modules fail differently
llm4oe-slr	Organize papers, tasks, and evidence about LLM ontology engineering	Provide a meta-level taxonomy of the project ecosystem	A review dataset, not an executable generator
OLLM	Fine-tune whole taxonomy-subgraph generation and evaluate structure	Move from edge-by-edge decisions to joint structural learning	Focused on the taxonomic backbone, not complete axioms, governance, or instance truth
LLMs4OM	Use RAG and parent/child context for ontology matching	Establish concept-representation granularity as a matching control variable	A match remains a correspondence proposal, not authorization to merge ontologies
SovereignOntologyLLM	Query local RDF/SHACL before asking an LLM to verbalize	Illustrate an auditable “controlled facts first, expression second” design	A personal exploratory project without independent evaluation comparable to mature systems; quantum routing is not treated as effectiveness evidence
OntoEKG	Assign entity/property extraction and hierarchical entailment to different models	Split discovery from logical classification into replaceable stages	A notebook-scale pipeline; multiple models do not eliminate error propagation

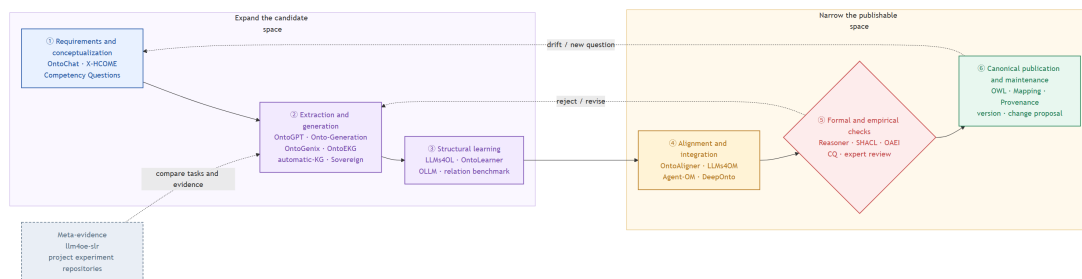


Figure 13.1: An ontology knowledge-production lifecycle across open-source implementations: projects control different stages rather than occupying one performance leaderboard

### 13.3 Trend: From Generating a File to Governing a Knowledge-Production Lifecycle

A 2026 systematic review analyzes 36 papers containing 49 task-level studies. It finds LLMs across requirements, implementation, publication, and maintenance, while task definitions, datasets, measures, and experimental protocols remain heterogeneous; direct comparisons across papers are generally not meaningful (Li et al. 2026). The following table therefore presents evolutionary directions rather than constructing a winners' table.

Table 13.2: Six migrations of ontology control

Migration from	Migration to	Representative project evidence	New control question
One prompt that emits an entire OWL file	CQ decomposition, recursive extraction, local generation, and iterative merge	OntoGPT, Onto-Generation, automatic-KG	Which question triggered each fragment, and how can it be merged or rolled back?
Isolated term or relation prediction	Joint task structure and taxonomy-subgraph learning	LLMs4OL, OntoLearner, OLLM	How is a hierarchy evaluated when individual nodes are correct but the whole structure is wrong?
One end-to-end model	Retrieval/LLM, two agents, neural-symbolic, and multi-model specialization	OntoAligner, Agent-OM, DeepOnto, OntoEKG	How does error propagate across stages, and which layer has final decision rights?
Generatability as the goal	Grounding, reasoners, SHACL, CQs, and expert review	OntoGPT, DeepOnto, OntoGenix	Who proves syntax, consistency, data conformance, and business correctness—and are these distinct proofs?
Automation as expert replacement	Non-expert participation, expert gates, and candidate/canonical separation	X-HCOME, OntoChat, OntoGenix	Which concepts are discoveries, and which are organizational or normative choices?
A static file as the deliverable	A mapped, queryable, traceable, continuously revised control plane	RML/OBDA, PROV, agent pipelines	Who published which version, when, and on what evidence?

Together these migrations show that ontology development is not about “having an LLM write more triples.” The model expands the candidate space; formal structure, validators, provenance, and people narrow the publishable space. Modularity becomes central because extraction, conceptualization, alignment, validation, and publication rely on different evidence and cannot all be proven by one act of language generation (Shimizu and Hitzler 2025).

### 13.4 Epistemic Reorganization: An Ontology Is Not Knowledge Itself but an Explicit Commitment About What Can Be Said

Gruber's classic definition treats an ontology as an explicit specification of a conceptualization. An ontology therefore states which classes, relations, and constraints a system recognizes; it does not claim to exhaust reality (Gruber 1993). Formal-ontology work further argues that once an information system interprets records through a set of categories, it has already made an ontological commitment. Knowledge organization consequently moves from arranging words to modeling identity, type, relation, time, and axioms together (Guarino 1998; Herre 2013).

The important epistemic change is to unbundle a fluent statement. A database value is an observation record. A sentence with time, subject, and provenance is an assertion. Types and relations place the assertion inside a shared conceptual frame. Logic, shapes, mapping tests, and expert review then provide different kinds of justification. This ladder is the book's operational definition; it does not claim to settle every philosophical dispute about knowledge. **An ontology reorganizes information; it does not automatically promote information to truth.**

Table 13.3: Ontological reorganization from data and information to publishable knowledge

Epistemic layer	Sakila example	Ontology-based organization	Question that must be answered before publication
Observation/data	<code>payment.amount = 5.99</code>	Preserve value, field, record identity, and acquisition time	Which system produced the record, and was it missing, duplicated, or revised?
Assertion/information	"Payment P recorded amount 5.99 at time T"	Bind a value to an event, participants, time, and provenance	Is this a source record, an inference, or a model-generated statement?
Conceptualization/commitment	Payment is an event; Customer and Store are participants	Define class, property, domain, range, and identity	Why this classification rather than Payment as an attribute or document?
Candidate knowledge	"Rental R corresponds to Payment P"; "the amount may constitute a revenue event"	An LLM, rule, or aligner emits a sourced candidate axiom or mapping	What supports it, and does it conflict with the existing ontology?
Justified canonical knowledge	An approved RevenueEvent definition and mapping	OWL reasoning, SHACL, mapping tests, CQs, and expert approval form separate gates	Did a check establish syntax, logic, data shape, task utility, or business truth?
Operational knowledge	Compute store revenue under the approved definition and trigger anomaly review	Queries, policies, and agent actions consume an explicit ontology version	How are action authority, stopping conditions, and outcome feedback recorded?

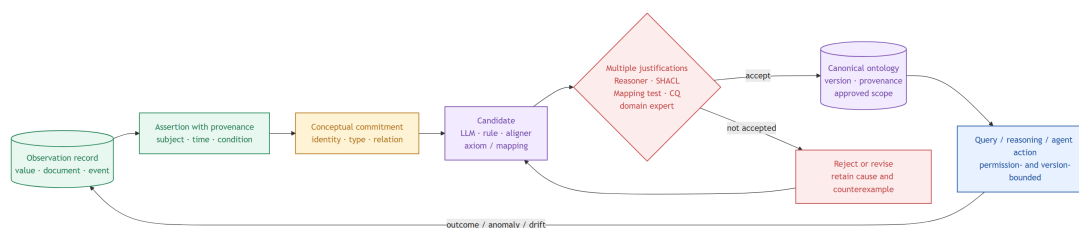


Figure 13.2: The ontology-driven epistemic loop: an LLM expands candidates, validation and governance narrow the canonical layer, and action feedback revises concepts

PROV-O supplies an exchangeable representation for the sources, activities, and responsible agents behind assertions, while SHACL provides a language for validating RDF data shapes. Their duties differ from OWL's open-world semantics (W3C Provenance Working Group 2013; W3C RDF Data Shapes Working Group 2017; W3C OWL Working Group, n.d.). Under OWL, a system may simply not know whether a relation exists, while a SHACL publication shape can require that a given record have exactly one amount. Calling both checks “ontology correctness” hides what was actually proven.

### ! The project map is not a performance leaderboard

Repositories, papers, datasets, models, domains, and metrics differ. This chapter compares knowledge-production stages and control mechanisms; it does not infer overall quality from GitHub stars, one reported F1 score, or author-reported results. Exploratory claims from projects such as SovereignOntologyLLM are retained only as design hypotheses, not verified effects.

## 13.5 The problem: a page states a relationship, but the system does not know its type

Level 7's `rental.md` can list neighboring tables, but it does not formally state that a particular foreign key is an `ObjectProperty` whose domain is `Rental` and whose range is `Customer`. Level 8 generates four kinds of assets from the live schema graph:

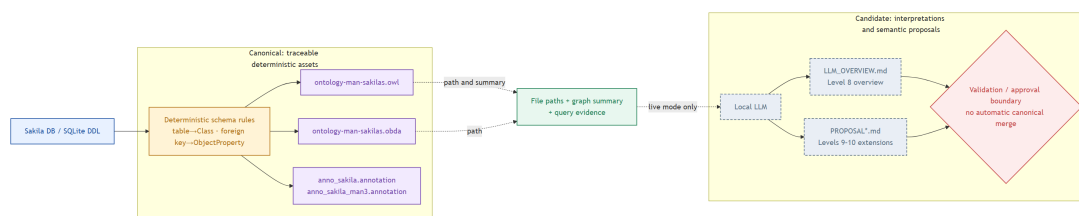


Figure 13.3: Level 8 canonical ontology pipeline and candidate boundary

OWL supplies the semantic structure for classes and properties (W3C OWL Working Group, n.d.). The OBDA text maps database tables to ontology class instances, while the annotation

files retain the correspondences for tables and foreign keys. The filenames follow the ontology-driven extraction project specified by the user, but the Sakila content comes from the local database; it does not copy the example ontology (Syamsiyah, n.d.).

### 13.6 Experiment: generate and parse the canonical OWL

```
from pathlib import Path
from rdflib import Graph, OWL, RDF
from assets.book_setup import run_offline

level_08 = run_offline(8)
artifact_paths = [Path(path) for path in level_08.artifacts]
owl_path = next(path for path in artifact_paths if path.suffix == ".owl")
ontology_graph = Graph().parse(owl_path)
{
 "artifacts": len(artifact_paths),
 "all_exist": all(path.is_file() for path in artifact_paths),
 "owl_classes": len(set(ontology_graph.subjects(RDF.type, OWL.Class))),
 "object_properties": len(
 set(ontology_graph.subjects(RDF.type, OWL.ObjectProperty))
),
 "rdf_triples": len(ontology_graph),
}
```

```
{'artifacts': 22,
 'all_exist': True,
 'owl_classes': 20,
 'object_properties': 21,
 'rdf_triples': 103}
```

The offline run produces 21 artifacts: four ontology/mapping files plus 17 wiki files. The code creates one `OWL.Class` for each table and one `OWL.ObjectProperty` for each foreign key. It also adds `RentalTransaction`, `RevenueEvent`, `CustomerValue`, and `FilmDemand`, together with an amount datatype property.

```
obda_path = next(path for path in artifact_paths if path.suffix == ".obda")
print("\n".join(obda_path.read_text(encoding="utf-8").splitlines()[1:18]))
```

```
[PrefixDeclaration]
```

sakila: <https://example.org/sakila#>

```
[MappingDeclaration] @collection [[
mappingId actor
target sakila:Actor/{rowid} a sakila:Actor .
source SELECT rowid, * FROM actor
```

```
mappingId address
target sakila:Address/{rowid} a sakila:Address .
source SELECT rowid, * FROM address
```

```
mappingId city
target sakila:City/{rowid} a sakila:City .
source SELECT rowid, * FROM city
```

```
mappingId category
target sakila:Category/{rowid} a sakila:Category .
```

## 13.7 Schema graphs, ontologies, and business truth

Generating valid RDF/XML proves only that the serialized structure can be parsed; it does not prove that the semantics are correct. The current transformation is a teaching projection: tables become classes and foreign keys become properties. A real domain ontology must still answer questions such as:

- Is payment an entity, an event, or a collection of attributes?
- How is the identity of the same customer defined across stores?
- What are the conceptual relationships among revenue, refunds, tax, and contribution?
- Which constraints belong in OWL axioms, and which belong in SHACL validation?

### ! Implementation boundary

The current runtime parses OWL classes and property domain/range, checks foreign-key mappings, queries relation counts, and publishes semantic knowledge only after validation. Revenue uses Level 6's graph-derived plan. This is not a full OWL reasoner, SPARQL endpoint, Ontop/OBDA engine or SHACL validator; the complete lifecycle above remains an analytical frame.

## 13.8 Canonical and candidate layers must remain separate

Level 8's canonical OWL is generated deterministically from schema rules. The live model receives the allowed relation vocabulary parsed from OWL and returns typed candidates. The host validates them, queries mapped database evidence and publishes `SEMANTIC_KNOWLEDGE.md`.

Levels 9–10 derive separate candidate OWL and knowledge pages from synthetic experiment variables and the revised objective. Their synthetic scope is explicit; they must not be merged directly into canonical OWL. Publication retains a validation boundary.

## 13.9 Evaluation

This level can optionally run DeepEval to assess whether the answer is relevant, grounded in evidence, and free of unjustified causal certainty. DeepEval does not validate OWL consistency or OBDA executability. Semantic assets require dedicated reasoners, shapes, and mapping tests.

## 13.10 Conclusion and promotion criterion

Level 8 upgrades knowledge from “readable documentation” to an “exchangeable, validatable, traceable candidate semantic structure.” These open-source implementations collectively show that knowledge production is not one-click OWL generation but a lifecycle of requirements, extraction, learning, alignment, validation, publication, and revision. It is appropriate for sharing concepts across systems, mapping data sources, or establishing a governed vocabulary.

A richer ontology still cannot answer whether increasing inventory causes revenue to rise. Move to Level 9 when the system must first express interventions, confounders, and identification gaps.

## 13.11 End-of-chapter exercises

1. Why might “one class per table” produce an anemic ontology?
2. Design a SHACL constraint for amount, then explain which class of business-semantic error it cannot solve.
3. What tests are still required between “RDFLib can parse the generated file” and “the business has approved it”?

## **Part IV**

# **Part IV — From Problems to Falsifiable Innovation**

## 14 Level 9: Ask Whether the Effect Is Identifiable Before Asking How to Optimize

### 14.1 Capability Acceptance Experiment

Generate seeded synthetic data with confounding and a known treatment effect; compare the naive difference with stratified adjustment. Reject estimation without overlap. The effect is not an estimate for real Sakila stores.

kungfu-agent experiment 9

One move, one lesson: **Problem-driven work is not about answering faster; it first makes the treatment, outcome, confounders, and missing evidence explicit.**

### 14.2 The problem: what can we infer from Store 2's higher revenue?

The first eight levels can answer “what happened” with increasing reliability, yet they can still turn a difference between stores into an intervention recommendation. The key to a causal problem is not drawing an attractive DAG, but defining:

- treatment: what we want to change;
- outcome: what we want to influence;
- confounders: which common causes affect both treatment and outcome;
- identification: whether the available data and assumptions are sufficient to separate the causal effect from the observational distribution.

Level 9 uses a hand-authored DAG to force the system to state these assumptions (Pearl 2009).

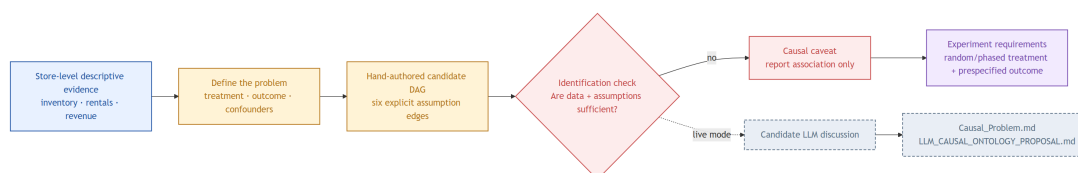


Figure 14.1: Level 9 pipeline from a descriptive difference to an identification problem

## 14.3 Experiment: broaden the descriptive evidence without crossing its boundary

```
from assets.book_setup import DB, run_offline

level_09 = run_offline(9)
DB.query(level_09.sql)
```

Table 14.1: Store-level observations of inventory breadth, rentals, and revenue

store_id	inventory_copies	rentals	customers	revenue
i64	i64	i64	i64	f64
1	2270	7923	599	33679.79
2	2311	8121	599	33726.77

Store 1 has 2,270 inventory copies, 7,923 rentals, and 33,679.79 in revenue. Store 2 has 2,311, 8,121, and 33,726.77, respectively. The observed direction is consistent with “more inventory may produce more transactions,” but it is equally consistent with “stores with greater demand receive more inventory.”

## 14.4 DAG: turn assumptions into objects of debate

The code fixes six edges:

```
causal_lab = level_09.metadata["causal_experiment"]
{
 "causal_dag": level_09.metadata["causal_dag"],
 "synthetic_known_effect": causal_lab["known_effect"],
 "synthetic_adjusted_effect": round(causal_lab["estimates"]["adjusted"], 3),
}
```

```
{'causal_dag': [(('inventory_breadth', 'rentals'),
 ('rentals', 'revenue'),
 ('customer_demand', 'rentals'),
 ('pricing', 'revenue'),
 ('store_location', 'customer_demand'),
 ('store_location', 'inventory_breadth'))],
'synthetic_known_effect': 2.0,
'synthetic_adjusted_effect': 2.007}
```

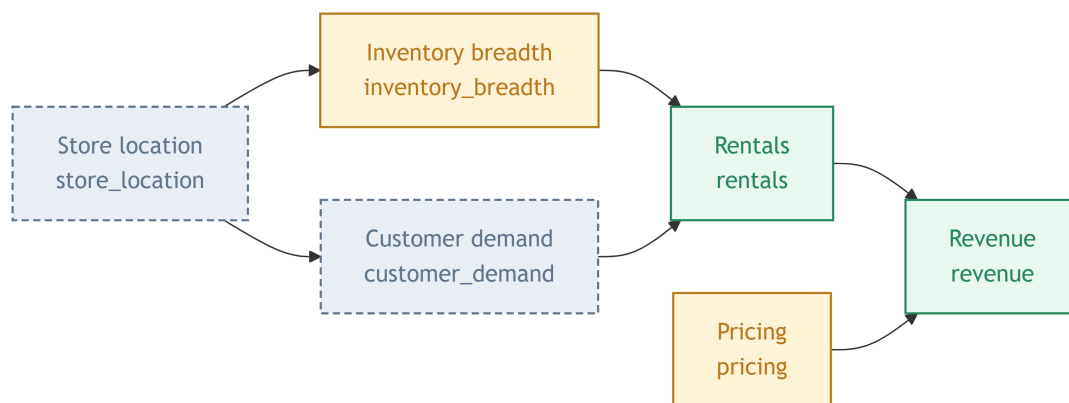


Figure 14.2: The candidate causal DAG in Level 9

store\_location affects both demand and inventory breadth, creating a potential backdoor path. customer\_demand affects rentals, while pricing directly affects revenue. Sakila lacks marketing exposure, random assignment, the competitive environment, and rich time-varying covariates. The current data therefore cannot identify the causal effect of inventory breadth on revenue.

## 14.5 Trust levels for this level's artifacts

Both modes retain observational context and produce explicitly labeled synthetic experiment artifacts:

- synthetic\_experiment.json: seed, ground truth, DAG, estimates and intervals;
- Identified\_Synthetic\_Effect.md: synthetic experiment knowledge;
- synthetic-causal.owl: a separate candidate vocabulary derived from simulation variables.

The live model explains the computed experiment report. The candidate vocabulary is scoped to the synthetic SCM and is not automatically merged into canonical ontology.

### ⚠ Implementation limitation

Real Sakila SQL remains a fixed observational store comparison. Effect estimation applies only to synthetic data with a known structure, adjusting for the declared demand confounder. It does not discover adjustment sets or establish real-store intervention effects. The 95% interval conditions on observed stratum weights.

## 14.6 From an identification gap to the next experiment

An honest output should have three parts:

1. **Association:** Store 2 has slightly more inventory, rentals, and revenue at the same time.

2. **Assumptions:** location, demand, and pricing may confound the relationship.
3. **Next step:** change inventory randomly or in phases, and collect treatment assignment, available-inventory time, and a prespecified outcome.

## 14.7 Conclusion and promotion criterion

Level 9 is appropriate for any analysis that could prompt a “correlation implies causation” decision. Its output is a better problem, explicit assumptions, and experimental requirements.

Move to Level 10 when the team must also derive falsifiable mechanisms from business invariants, rather than settling for unconstrained LLM brainstorming.

## 14.8 End-of-chapter exercises

1. Add promotion to the DAG and explain which paths it might open or close.
2. Define the treatment, unit, outcome, and estimand for an inventory experiment.
3. What additional data would still be insufficient to identify the causal effect, and why?

## 15 Level 10: Derive Falsifiable Innovation from Invariants

### 15.1 Capability Acceptance Experiment

Enumerate transfers from inventory conservation and compare purchasing with retaining the baseline. Contribution per inventory period falsifies a revenue-maximizing but costly proposal. A balanced-demand negative control must return no supported improvement. Conclusions are limited to the synthetic simulator.

kungfu-agent experiment 10

One move, one lesson: **First-principles reasoning is not “bold imagination”; it derives mechanisms from objectives, invariants, and constraints that data can prove wrong.**

### 15.2 The problem: the DAG tells us what we do not know—what comes next?

Level 9 prevents excessive causal interpretation, but listing identification gaps alone does not produce actionable learning. Level 10 decomposes an innovation problem into five layers:

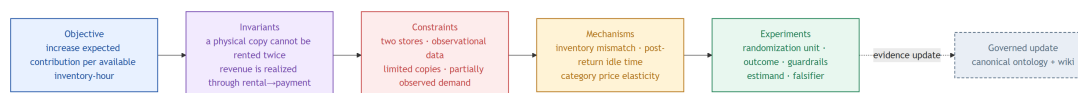


Figure 15.1: Five-layer derivation from a first-principles objective to a falsifiable experiment

The objective is not the vague “increase revenue,” but “increase expected contribution per available inventory-hour.” The invariants include that one physical inventory copy cannot serve two rentals at the same time, revenue is realized through payments associated with rentals, and demand is not fully observable. The constraints include only two stores, historical observational data, limited copies, and customer experience.

## 15.3 Experiment: inspect the first-principles structure

```
from assets.book_setup import run_offline

level_10 = run_offline(10)
principles = level_10.metadata["first_principles"]
principles
```

```
{'objective': 'contribution per inventory period, rather than revenue alone',
 'invariants': ['one copy serves at most one rental per period',
 'served rentals cannot exceed demand',
 'contribution = revenue - operating cost - intervention cost'],
 'constraints': ['two simulated stores',
 'nonnegative integer copies',
 'rebalancing conserves total copies',
 'explicit intervention costs'],
 'mechanisms': ['reallocate idle copies',
 'purchase additional copies',
 'retain baseline'],
 'experiments': ['evaluate all feasible transfers',
 'falsify revenue-only purchasing',
 'balanced-demand negative control']}
```

The executable experiment compares three mechanism families: rebalancing, purchasing and retaining the baseline. Inventory conservation derives a finite set of transfers; explicit cost and demand assumptions determine contribution. Reminder and pricing experiments remain re-search extensions.

## 15.4 From a mechanism to a falsifiable experiment

Consider the “inventory mismatch” mechanism:

Design element	One executable definition
Randomization unit	film × two-store pair; weeks are repeated observation windows
Treatment	Within the pair, move one inventory copy from the store with lower predicted stockout risk to the store with higher predicted stockout risk
Control	Matched films retain their existing inventory allocation
Primary proxy	Pair-level rentals or revenue per total copy-day

Design element	One executable definition
Secondary outcomes	Report stockouts, rentals, and revenue separately for donor and recipient stores
Guardrails	Donor-store stockouts, overdue returns, repeat-rental rate, and customer complaints
Estimand	Intention-to-treat average difference in the pair-level proxy under reallocation assignment
Falsifier	Pair-level output does not improve, or any donor guardrail deteriorates materially

First match eligible films using historical demand and title characteristics, then randomize the complete two-store pair to “reallocate” or “do not reallocate” within matched groups. This keeps interference between donor and recipient stores inside one experimental unit. Substitution across films can still violate no-interference assumptions, so category-level spillovers should be prespecified and, where feasible, randomization should be clustered by category. Freeze the outcome, sample window, and stopping rules before the experiment to avoid re-defining success after observing the results.

### ! A proxy is not the final objective

Sakila contains rentals, returns, and payments, but not complete inventory costs, transfer costs, store operating status, or an explicit “available inventory-hours” field. This experiment can therefore falsify or support only the proxy mechanism that reallocation improves observable throughput or revenue; it cannot establish an increase in contribution per available inventory-hour. That remains a design objective. Before making the latter claim, the system must capture available inventory-hours, net contribution, and transfer costs, and prespecify how the proxy maps to the objective.

## 15.5 Implementation facts: how Level 10 inherits Level 9

Level 10 reuses Level 9’s observational context and synthetic causal experiment offline, then enumerates feasible inventory interventions. Every candidate is evaluated under resource constraints and contribution accounting, followed by a balanced-demand negative control. The live model explains calculated evidence; it does not replace the evaluator’s selection.

Both modes write:

- `hypothesis_tests.json`: candidates, evaluations and the negative control;
- `Tested_Principles.md`: objective revision and experiment knowledge;
- `candidate-principles.owl`: separate candidate concepts.

The simulator assumes fixed demand, explicit costs and at most one rental per copy per period. It supports comparisons within a finite hypothesis space, not claims of real-world innovation or deployment returns. Candidate concepts are not automatically merged into canonical OWL.

## 15.6 How first-principles reasoning differs from unconstrained brainstorming

Unconstrained brainstorming asks, “What else could we do?” First-principles design asks:

1. Which facts must hold under every proposed solution?
2. Which constraint causes the current solution to fail?
3. Which mechanism links the intervention to the outcome?
4. What result would falsify that mechanism?
5. After failure, which ontology or wiki knowledge must be revised?

Falsifiability brings innovation into an engineering loop. A hypothesis is not a position to defend; it is an asset that should face a high-quality test as soon as possible.

## 15.7 Conclusion: the highest level still receives no additional privilege

Level 10’s innovation artifacts inherit the same read-only SQL boundary. What advances is the structure of problems, knowledge, and experiments—not database write privilege.

The next production step is not Level 11. It is to connect experiments to approval, randomization, monitoring, and causal-evaluation infrastructure, then feed the evidence into a controlled publication process for the canonical ontology and wiki.

## 15.8 End-of-chapter exercises

1. Describe an observation that would falsify “reducing idle turnaround time increases revenue.”
2. Why can revenue growth not be the only success metric for a pricing experiment?
3. Which new fields must be added to the data model to calculate “contribution per available inventory-hour”?

## 16 Conclusion: Choose the Lowest Sufficient Control Level

### 16.1 The ten levels are a selection matrix, not a ranking

Level	Controller	Primary artifact	Best-fit scenario	Residual risk
1	Prompt + workflow	state, SQL	Fixed, single-step analysis	No automatic repair
2	Code skill	tool calls	Single-process tool assistant	Incorrect tool selection
3	Protocol envelope	trace	Collaboration across responsibility boundaries	Records but does not adjudicate
4	OmniGenT + Codex harness	typed action plan, evaluation	Fixed two-role analysis	Shared blind spots in the plan and evaluator
5	Evaluator + loop	iteration history	Machine-repairable tasks	Metric gaming and exhaustion policy
6	Typed manifest + graph	contract, join path	Complex relational queries	A graph is not business semantics
7	Durable wiki	Markdown knowledge	Accumulation across runs	Drift and provenance governance
8	Ontology pipeline	OWL/OBDA/annotation	Semantic exchange	A valid structure is not necessarily correct knowledge
9	Causal problem	DAG, identification gap	Analysis before intervention decisions	No effect estimation
10	First principles	mechanisms and experiments	Falsifiable innovation	No experiment-execution infrastructure yet

## 16.2 The migration of control

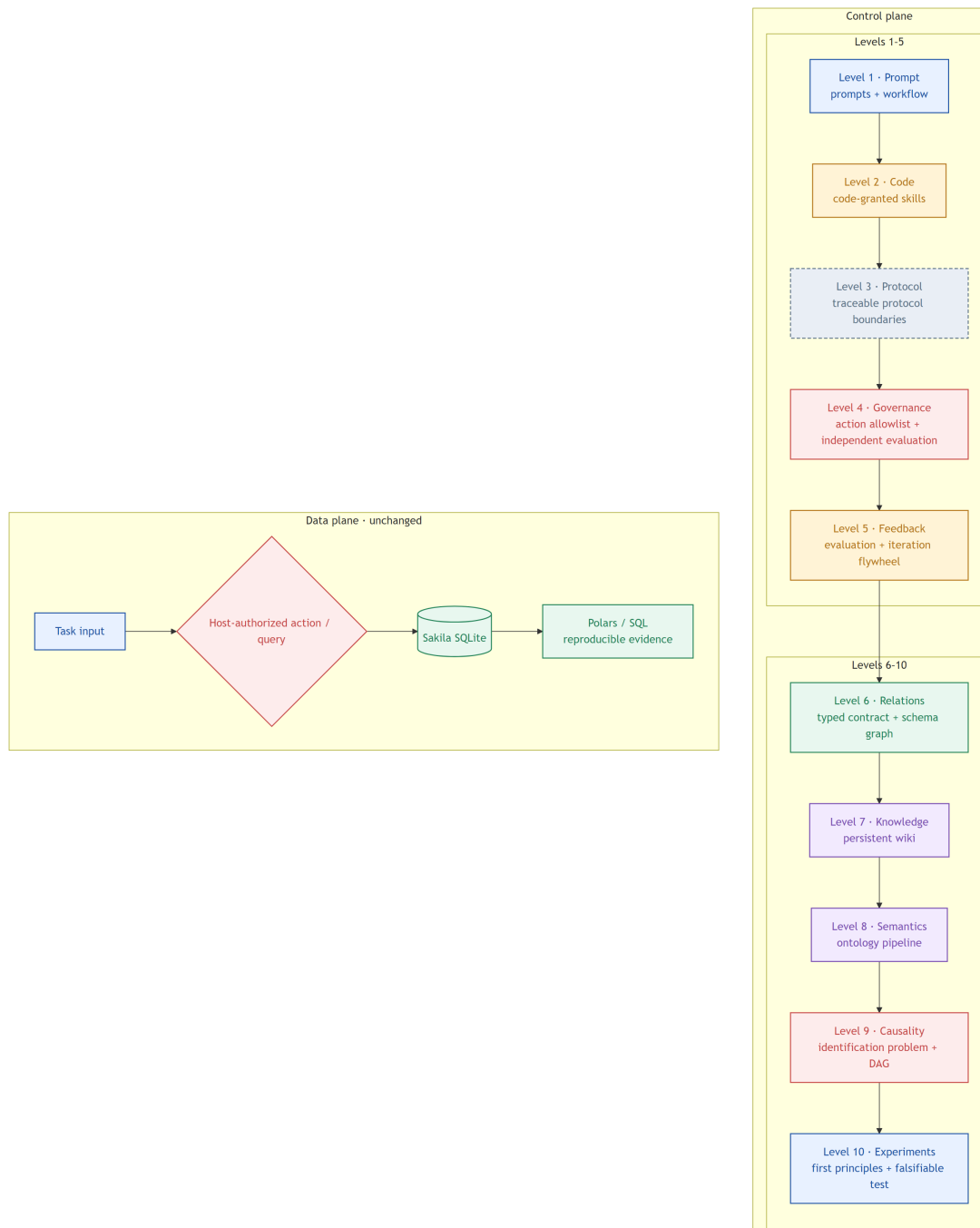


Figure 16.1: The cumulative migration of the control plane across ten levels

This does not mean that every level strictly contains the complete runtime of all preceding levels. It means that the number of control problems requiring management increases. To choose a level, first identify the failure that is currently most expensive, dangerous, or difficult to detect, then select the lowest level that controls it directly.

## 16.3 Ten engineering principles

1. **A prompt is a soft constraint.** Enforce permissions in code, a sandbox, or the database layer, and verify the sandbox's enforcement scope per operating system rather than inferring it from a configuration label.
2. **The tool surface is the capability surface.** A model should not acquire capabilities that the code has not exposed.
3. **A protocol label is not a protocol implementation.** State which boundaries are real networks and which are teaching envelopes.
4. **Model self-evaluation is not final evidence.** The host and an independent baseline must check action shape, skill provenance, and results.
5. **A loop without an evaluator is only repetition.** A loop without a stopping condition amplifies risk.
6. **A schema graph is not an ontology.** A shortest path is not necessarily the correct business path.
7. **Persistent knowledge needs freshness and provenance.** Writing a file is only the beginning of governance.
8. **Generating valid OWL does not create correct knowledge.** Canonical and candidate layers must remain separate.
9. **Identify before optimizing.** An observational difference does not automatically become an intervention effect.
10. **Innovation must be falsifiable.** A mechanism that cannot explain how it could fail is not yet a testable proposal.

## 16.4 A recommended learning path

### 16.4.1 A 90-minute rapid course

- Run Levels 1, 2, and 3 in Marimo and observe how the answer, tools, and trace change.
- Run Level 4 and inspect its delegation evidence, skill digest, allowlisted actions, and ten checks; then run Level 5 and read the evaluator feedback from both iterations.
- Run Level 8 and open the OWL and OBDA files.
- Run Levels 9–10 and rewrite the “conclusion” as an “identification gap + experiment.”

### 16.4.2 A one-day engineering workshop

1. Add a read-only database account and timeout to the SQL guard.
2. Add latency, fallback, and error category to the Level 3 trace.
3. Tamper with Level 4's action order, n, and skill digest, and verify that all three cases fail closed before host execution.

4. Design an exhaustion policy for Level 5.
5. Add a schema hash to the wiki.
6. Validate the ontology with SHACL or a reasoner.
7. Write a preregistered inventory experiment protocol.

## 16.5 Chinese–English synchronized maintenance policy

This book uses Quarto profiles to maintain two parallel editions (Quarto Project, n.d.-c). Chinese source files live at the project root and in `chapters/`; their English mirrors live under `en/`. Every conceptual change, implementation fact, limitation, exercise, or citation update should be applied to both editions in the same change. The two editions must preserve:

- a one-to-one correspondence between filenames and chapter numbers;
- identical heading anchors, cell labels, and citation keys;
- equivalent Python semantics, translating only explanatory output and captions;
- synchronized table rows, limitations, and implementation facts;
- a release gate requiring both PDFs to pass execution and text-extraction checks.

The shared `_quarto.yml` stores execution, format, and font configuration. `_quarto-zh.yml` and `_quarto-en.yml` define the respective chapter lists, languages, and output directories. Quarto profiles merge each profile configuration with the shared project configuration (Quarto Project, n.d.-c).

## 16.6 Glossary

Term	Meaning in this book
Data plane	The path that actually reads Sakila and returns evidence
Control plane	The mechanisms that determine plans, tools, protocols, adjudication, knowledge, and experiments
Skill / tool	A callable capability that code grants to the model
Envelope	A message wrapper containing sender, recipient, kind, payload, and correlation
Harness	The governance boundary for execution privileges, evaluation, adjudication, and budgets
Ralph loop	An improvement loop with an evaluator, feedback memory, and stopping conditions
Schema graph	A table-relationship graph reconstructed from foreign keys
Wiki	A readable, diffable, persistent knowledge snapshot
Ontology	A semantic model that explicitly represents classes, properties, relationships, and constraints
Canonical	A knowledge layer approved through a deterministic or governed process

Term	Meaning in this book
Candidate	A model proposal awaiting validation that cannot be published directly
Identification	The conditions required to recover a causal target from an observational distribution and assumptions
Estimand	The precisely defined quantity an experiment or analysis seeks to estimate
Guardrail	A constraint metric that prevents improvement in a primary metric from hiding serious side effects

## 16.7 The final reader test

If you can answer the following questions precisely, you understand the ten-level framework:

1. Why does a read-only SQL assistant usually stop at Level 2 rather than require Level 8?
2. Is the difference between Levels 2 and 3 the number of tools, or the explicitness of boundaries?
3. Can either Level 4 Codex subagent read the database directly? Who owns planning, authorization, execution, and evaluation?
4. Why must a Ralph loop have both an evaluator and a maximum iteration count?
5. How does a wiki differ from putting more schema text into a prompt?
6. Which ontology artifacts are deterministic canonical assets, and which are only LLM proposals?
7. Which descriptive questions can Sakila answer, and which causal questions can it not identify?
8. How does first-principles design differ from unconstrained brainstorming?
9. Why do database privileges remain unchanged even after advancing to Level 10?

## References

- Anthropic. 2023a. “Claude’s Constitution.” May 9. <https://www.anthropic.com/news/claude-constitution>.
- Anthropic. 2023b. “Introducing Claude.” March 14. <https://www.anthropic.com/news/introducing-claude>.
- Anthropic. 2024a. “Building Effective Agents.” December 19. <https://www.anthropic.com/engineering/building-effective-agents>.
- Anthropic. 2024b. “Expanding Access to Claude for Government.” June 26. <https://www.anthropic.com/news/expanding-access-to-claude-for-government>.

- Anthropic. 2024c. “Introducing the Model Context Protocol.” November 25. <https://www.anthropic.com/news/model-context-protocol>.
- Anthropic. 2025a. “Beyond Permission Prompts: Making Claude Code More Secure and Autonomous.” October 20. <https://www.anthropic.com/engineering/claude-code-sandboxing>.
- Anthropic. 2025b. “Claude Code: Best Practices for Agentic Coding.” April 18. <https://www.anthropic.com/engineering/claude-code-best-practices>.
- Anthropic. 2025c. “Effective Context Engineering for AI Agents.” September 29. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Anthropic. 2025d. “Equipping Agents for the Real World with Agent Skills.” October 16. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.
- Anthropic. 2026a. “Demystifying Evals for AI Agents.” January 9. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>.
- Anthropic. 2026b. “Scaling Managed Agents: Decoupling the Brain from the Hands.” April 8. <https://www.anthropic.com/engineering/managed-agents>.
- Babaei Giglou, Hamed, Jennifer D’Souza, and Sören Auer. 2023. “LLMs4OL: Large Language Models for Ontology Learning.” *The Semantic Web – ISWC 2023*, 408–27. [https://doi.org/10.1007/978-3-031-47240-4\\_22](https://doi.org/10.1007/978-3-031-47240-4_22).
- Babaei Giglou, Hamed, Jennifer D’Souza, Oliver Karras, and Sören Auer. 2025. “OntoAligner: A Comprehensive Modular and Robust Python Toolkit for Ontology Alignment.” *arXiv Preprint arXiv:2503.21902*, ahead of print. <https://doi.org/10.5281/zenodo.14533132>.
- Becker, Joel, Nate Rush, Elizabeth Barnes, and David Rein. 2025. “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.” *arXiv Preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>.
- Brown, Tom B., Benjamin Mann, Nick Ryder, et al. 2020. “Language Models Are Few-Shot Learners.” *Advances in Neural Information Processing Systems* 33: 1877–901. <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- Caufield, J. Harry, Harshad Hegde, Vincent Emonet, et al. 2024. “Structured Prompt Interrogation and Recursive Extraction of Semantics (SPIRES): A Method for Populating Knowledge

- Bases Using Zero-Shot Learning.” *Bioinformatics* 40 (3): btae104. <https://doi.org/10.1093/bioinformatics/btae104>.
- Chen, Zirui, Shijie Chen, Yuting Ning, et al. 2025. “ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery.” *International Conference on Learning Representations*. [https://proceedings.iclr.cc/paper\\_files/paper/2025/hash/f12b4df26344f3be803c06b555252efe-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2025/hash/f12b4df26344f3be803c06b555252efe-Abstract-Conference.html).
- Chollet, François. 2019. “On the Measure of Intelligence.” *arXiv Preprint arXiv:1911.01547*. <https://arxiv.org/abs/1911.01547>.
- Claxton, Louis. 2026. “The AI-Native SDLC Playbook.” Anthropic, August 21. <https://claude.com/blog/the-ai-native-sdlc-playbook>.
- Confident AI. n.d. “DeepEval Documentation.” Accessed August 30, 2026. <https://deepeval.com/docs/getting-started>.
- DeepSeek-AI. 2025. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” *Nature* 645: 633–38. <https://doi.org/10.1038/s41586-025-09422-z>.
- Edge, Darren, Ha Trinh, Newman Cheng, et al. 2024. “From Local to Global: A Graph RAG Approach to Query-Focused Summarization.” *arXiv Preprint arXiv:2404.16130*. <https://arxiv.org/abs/2404.16130>.
- Gruber, Thomas R. 1993. “A Translation Approach to Portable Ontology Specifications.” *Knowledge Acquisition* 5 (2): 199–220. <https://doi.org/10.1006/knac.1993.1008>.
- Guarino, Nicola. 1998. “Formal Ontology and Information Systems.” *Formal Ontology in Information Systems: Proceedings of the First International Conference*, 3–15. <https://iaoa.org/fois/1998.html>.
- Gupta, Vipul, Shikhar Mohan, Lakshya Kumar, et al. 2026. “HARNESS-LM: A Three-Phase Training Recipe for Harnessing SLMs in Sponsored Search Retrieval.” *arXiv Preprint arXiv:2605.23572*, ahead of print. <https://doi.org/10.48550/arXiv.2605.23572>.
- He, Yuan, Jiaoyan Chen, Hang Dong, et al. 2023. “DeepOnto: A Python Package for Ontology Engineering with Deep Learning.” *arXiv Preprint arXiv:2307.03067*. <https://arxiv.org/abs/2307.03067>.
- Herre, Heinrich. 2013. “Formal Ontology and the Foundation of Knowledge Organization.” *Knowledge Organization* 40 (5): 332–39. <https://doi.org/10.5771/0943-7444-2013-5-332>.

- Hong, Sirui, Mingchen Zhuge, Jiaqi Chen, et al. 2023. "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework." *arXiv Preprint arXiv:2308.00352*. <https://arxiv.org/abs/2308.00352>.
- Jimenez, Carlos E., John Yang, Alexander Wettig, et al. 2024. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" *International Conference on Learning Representations*. [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html).
- Jones, David, Chris Snider, Aydin Nassehi, Jason Yon, and Ben Hicks. 2020. "Characterising the Digital Twin: A Systematic Literature Review." *CIRP Journal of Manufacturing Science and Technology* 29 (Part A): 36–52. <https://doi.org/10.1016/j.cirpj.2020.02.002>.
- Kaelbling, Leslie Pack, Michael L. Littman, and Anthony R. Cassandra. 1998. "Planning and Acting in Partially Observable Stochastic Domains." *Artificial Intelligence* 101 (1–2): 99–134. [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X).
- LangChain. n.d. "LangGraph Overview." Accessed August 30, 2026. <https://docs.langchain.com/oss/python/langgraph/overview>.
- Lee, Yoonho, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. "Meta-Harness: End-to-End Optimization of Model Harnesses." *arXiv Preprint arXiv:2603.28052*, ahead of print. <https://doi.org/10.48550/arXiv.2603.28052>.
- Legg, Shane, and Marcus Hutter. 2007. "Universal Intelligence: A Definition of Machine Intelligence." *Minds and Machines* 17 (4): 391–444. <https://doi.org/10.1007/s11023-007-9079-x>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems* 33. <https://arxiv.org/abs/2005.11401>.
- Li, Jiayi, Daniel Garijo, and María Poveda-Villalón. 2026. "Large Language Models for Ontology Engineering: A Systematic Literature Review." *Semantic Web: Interoperability, Usability, Applicability* 17 (4): 1–45. <https://doi.org/10.1177/22104968261465514>.
- Lian, KJ, and Dan Krpata. 2024. "How AWS Can Help Mission-Focused Organizations Comply with the White House National Security Memorandum on AI." Amazon Web Services, November 15. <https://aws.amazon.com/blogs/publicsector/how-aws-can-help-mission-focused-organizations-comply-with-the-white-house-national-security-memorandum-on-ai/>.

- Liu, Xiao, Hao Yu, Hanchen Zhang, et al. 2024. "AgentBench: Evaluating LLMs as Agents." *International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Lo, Andy, Albert Q. Jiang, Wenda Li, and Mateja Jamnik. 2024. "End-to-End Ontology Learning with Large Language Models." *Advances in Neural Information Processing Systems* 37. <https://doi.org/10.5555/3737916.3740683>.
- Lopopolo, Ryan. 2026. "Harness Engineering: Leveraging Codex in an Agent-First World." February 11. <https://openai.com/index/harness-engineering/>.
- Luo, Xufang, Yuge Zhang, Zhiyuan He, et al. 2025. "Agent Lightning: Train ANY AI Agents with Reinforcement Learning." *arXiv Preprint arXiv:2508.03680*, ahead of print. <https://doi.org/10.48550/arXiv.2508.03680>.
- Ma, Chang, Junlei Zhang, Zhihao Zhu, et al. 2024. "AgentBoard: An Analytical Evaluation Board of Multi-Turn LLM Agents." *Advances in Neural Information Processing Systems* 37. <https://doi.org/10.52202/079017-2365>.
- Madaan, Aman, Niket Tandon, Prakhar Gupta, et al. 2023. "Self-Refine: Iterative Refinement with Self-Feedback." *Advances in Neural Information Processing Systems* 36. <https://arxiv.org/abs/2303.17651>.
- Meng, Qianyu, Yanan Wang, Liyi Chen, et al. 2026. *Agent Harness for Large Language Model Agents: A Survey*. Preprints.org. <https://www.preprints.org/manuscript/202604.0428/v1>.
- Morris, Meredith Ringel, Jascha Sohl-Dickstein, Noah Fiedel, et al. 2024. "Levels of AGI for Operationalizing Progress on the Path to AGI." *Proceedings of the 41st International Conference on Machine Learning*. <https://arxiv.org/abs/2311.02462>.
- Nakano, Reiichiro, Jacob Hilton, Suchir Balaji, et al. 2021. "WebGPT: Browser-Assisted Question-Answering with Human Feedback." *arXiv Preprint arXiv:2112.09332*. <https://arxiv.org/abs/2112.09332>.
- NetworkX Developers. n.d. "NetworkX Documentation." Accessed August 30, 2026. <https://networkx.org/documentation/stable/>.
- OpenAI. n.d.-a. "Codex SDK." Accessed August 30, 2026. <https://learn.chatgpt.com/docs/codex-sdk>.
- OpenAI. n.d.-b. "Models—OpenAI Agents SDK." Accessed August 30, 2026. <https://openai.github.io/openai-agents-python/models/>.

- OpenAI. n.d.-c. "Running Agents." Accessed August 30, 2026. [https://openai.github.io/openai-agents-python/running\\_agents/](https://openai.github.io/openai-agents-python/running_agents/).
- OpenAI. n.d.-d. "Tools—OpenAI Agents SDK." Accessed August 30, 2026. <https://openai.github.io/openai-agents-python/tools/>.
- OpenClaw Project. 2026a. "Agent Loop." <https://docs.openclaw.ai/concepts/agent-loop>.
- OpenClaw Project. 2026b. "Memory Overview." <https://docs.openclaw.ai/concepts/memory>.
- OpenClaw Project. 2026c. "Sandbox Vs. Tool Policy Vs. Elevated." <https://docs.openclaw.ai/gateway/sandbox-vs-tool-policy-vs-elevated>.
- OpenClaw Project. 2026d. "Skills." <https://docs.openclaw.ai/tools/skills>.
- OpenTelemetry Project. 2026. "Context Propagation." <https://opentelemetry.io/docs/concepts/context-propagation/>.
- Ouyang, Long, Jeffrey Wu, Xu Jiang, et al. 2022. "Training Language Models to Follow Instructions with Human Feedback." *Advances in Neural Information Processing Systems* 35: 27730–44. [https://proceedings.neurips.cc/paper\\_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract.html).
- Packer, Charles, Sarah Wooders, Kevin Lin, et al. 2023. "MemGPT: Towards LLMs as Operating Systems." *arXiv Preprint arXiv:2310.08560*. <https://arxiv.org/abs/2310.08560>.
- Palantir Technologies. 2024. "Anthropic and Palantir Partner to Bring Claude AI Models to AWS for u.s. Government Intelligence and Defense Operations." November 7. <https://investors.palantir.com/news-details/2024/Anthropic-and-Palantir-Partner-to-Bring-Claude-AI-Models-to-AWS-for-U.S.-Government-Intelligence-and-Defense-Operations/>.
- Palantir Technologies. 2026. "Foundry Platform Summary for LLMs." July 2. <https://www.palantir.com/docs/foundry/getting-started/foundry-platform-summary-llm>.
- Pallets Projects. n.d. "Jinja Documentation." Accessed August 30, 2026. <https://jinja.palletsprojects.com/>.
- Pan, Linyue, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 2026. "Natural-Language Agent Harnesses." *arXiv Preprint arXiv:2603.25723*, ahead of print. <https://doi.org/10.48550/arXiv.2603.25723>.

- Park, Joon Sung, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. "Generative Agents: Interactive Simulacra of Human Behavior." *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. <https://doi.org/10.1145/3586183.3606763>.
- Pearl, Judea. 2009. *Causality: Models, Reasoning, and Inference*. 2nd ed. Cambridge University Press.
- Peng, Sida, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot." *arXiv Preprint arXiv:2302.06590*. <https://arxiv.org/abs/2302.06590>.
- Pydantic. n.d. "Pydantic Documentation." Accessed August 30, 2026. <https://docs.pydantic.dev/>.
- Quarto Project. n.d.-a. "Creating a Book." Accessed August 30, 2026. <https://quarto.org/docs/books/>.
- Quarto Project. n.d.-b. "PDF Basics." Accessed August 30, 2026. <https://quarto.org/docs/output-formats/pdf-basics.html>.
- Quarto Project. n.d.-c. "Project Profiles." Accessed August 30, 2026. <https://quarto.org/docs/projects/profiles.html>.
- Quarto Project. n.d.-d. "Tutorial: Hello, Quarto—Jupyter." Accessed August 30, 2026. <https://quarto.org/docs/get-started/hello/jupyter.html>.
- Rajasekaran, Prithvi. 2026. "Harness Design for Long-Running Application Development." Anthropic, March 24. <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
- Rasmussen, Preston, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. "Zep: A Temporal Knowledge Graph Architecture for Agent Memory." *arXiv Preprint arXiv:2501.13956*. <https://arxiv.org/abs/2501.13956>.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, et al. 2023. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems* 36. <https://doi.org/10.52202/075280-2997>.
- Sculley, D., Gary Holt, Daniel Golovin, et al. 2015. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems* 28. <https://proceedings.neurips.cc/paper/2015/hash/86df7dcfd896fcdf2674f757a2463eba-Abstract.html>.

- Shen, Yongliang, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. "HuggingGPT: Solving AI Tasks with ChatGPT and Its Friends in Hugging Face." *arXiv Preprint arXiv:2303.17580*. <https://arxiv.org/abs/2303.17580>.
- Shimizu, Cogan, and Pascal Hitzler. 2025. "Accelerating Knowledge Graph and Ontology Engineering with Large Language Models." *Journal of Web Semantics*, 100862. <https://doi.org/10.1016/j.websem.2025.100862>.
- Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. "Reflexion: Language Agents with Verbal Reinforcement Learning." *Advances in Neural Information Processing Systems* 36. <https://doi.org/10.52202/075280-0377>.
- Sigelman, Benjamin H., Luiz André Barroso, Mike Burrows, et al. 2010. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google. <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>.
- Silver, David, Thomas Hubert, Julian Schrittwieser, et al. 2018. "A General Reinforcement Learning Algorithm That Masters Chess, Shogi, and Go Through Self-Play." *Science* 362 (6419): 1140–44. <https://doi.org/10.1126/science.aar6404>.
- Syamsiyah, Alifah. n.d. "Ontology-Driven Extraction Example." Accessed August 30, 2026. <https://github.com/alifahsyamsiyah/ontology-driven-extraction>.
- W3C OWL Working Group. n.d. "OWL 2 Web Ontology Language Document Overview." Accessed August 30, 2026. <https://www.w3.org/TR/owl2-overview/>.
- W3C Provenance Working Group. 2013. "PROV Overview." April 30. <https://www.w3.org/TR/prov-overview/>.
- W3C RDF Data Shapes Working Group. 2017. "Shapes Constraint Language (SHACL)." July 20. <https://www.w3.org/TR/shacl/>.
- Wang, Guanzhi, Yuqi Xie, Yunfan Jiang, et al. 2024. "Voyager: An Open-Ended Embodied Agent with Large Language Models." *Transactions on Machine Learning Research*. <https://openreview.net/forum?id=ehfRiF0R3a>.
- Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems* 35. <https://arxiv.org/abs/2201.11903>.

- Wu, Qingyun, Gagan Bansal, Jieyu Zhang, et al. 2023. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv Preprint arXiv:2308.08155*. <https://arxiv.org/abs/2308.08155>.
- Xia, Peng, Jianwen Chen, Hanyang Wang, et al. 2026. "SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning." *arXiv Preprint arXiv:2602.08234*, ahead of print. <https://doi.org/10.48550/arXiv.2602.08234>.
- Xie, Tianbao, Danyang Zhang, Jixuan Chen, et al. 2024. "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments." *Advances in Neural Information Processing Systems* 37. <https://doi.org/10.52202/079017-1650>.
- Yang, Chao, Yuan Zheng, Yuan Hua, Riku Ala-Laurinaho, Udayanto Dwi Atmojo, and Kari Tammi. 2024. "Knowledge-Enhanced Digital Twin for Industrial Production Process." *2024 IEEE 22nd International Conference on Industrial Informatics*, 94–99. <https://doi.org/10.1109/INDIN58382.2024.10774266>.
- Yang, John, Carlos E. Jimenez, Alexander Wettig, et al. 2024. "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering." *arXiv Preprint arXiv:2405.15793*. <https://arxiv.org/abs/2405.15793>.
- Yao, Shunyu, Howard Chen, John Yang, and Karthik Narasimhan. 2022. "WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents." *Advances in Neural Information Processing Systems* 35. [https://proceedings.neurips.cc/paper\\_files/paper/2022/hash/82ad13ec01f9fe44c01cb91814fd7b8c-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/82ad13ec01f9fe44c01cb91814fd7b8c-Abstract-Conference.html).
- Yao, Shunyu, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. " $\tau$ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." *International Conference on Learning Representations*. <https://openreview.net/forum?id=roNSXZpUDN>.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations*. [https://openreview.net/forum?id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X).
- Yehudai, Asaf, Lilach Eden, Alan Li, et al. 2025. "Survey on Evaluation of LLM-Based Agents." *arXiv Preprint arXiv:2503.16416*, ahead of print. <https://doi.org/10.48550/arXiv.2503.16416>.
- Young, Justin. 2025. "Effective Harnesses for Long-Running Agents." Anthropic, November 26. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>.

Zhou, Chenyu, Huacan Chai, Wenteng Chen, et al. 2026. "Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering." *arXiv Preprint arXiv:2604.08224*, ahead of print. <https://doi.org/10.48550/arXiv.2604.08224>.

Zhou, Shuyan, Frank F. Xu, Hao Zhu, et al. 2024. "WebArena: A Realistic Web Environment for Building Autonomous Agents." *International Conference on Learning Representations*. [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html).